



Note Desktop

Documentación técnica completa

Versión 0.2.3 + parche posrelease

Arquitectura, arranque, Wayland/XWayland, Rust/GTK4, shell, configuración, internacionalización, scripts, servicios, instalación, empaquetado, diagnóstico, modificación y roadmap hacia un compositor Smithay propio.

Manual para desarrolladores y mantenedores

Julio de 2026

Prefacio

Este documento describe de forma técnica y honesta el estado del proyecto **Note Desktop** en la rama 0.2.x. La versión tomada como base es la 0.2.3, junto con el parche posrelease que corrige el movimiento parcial de `controls.page` en `note-settings`. El manual no presenta al proyecto como un reemplazo terminado de GNOME: documenta una **alpha funcional**, útil para aprender, experimentar, refactorizar y construir la siguiente etapa.

La intención del proyecto es combinar dos ideas:

- un flujo de trabajo inspirado en GNOME: vista general, búsqueda central, escritorios, pocas decisiones visibles y navegación por teclado;
- una identidad visual inspirada en macOS: barra superior, dock flotante, controles compactos, transparencias, separación visual y jerarquía limpia.

La implementación actual usa **labwc** como compositor temporal. El panel, dock, vista general, centro de control y aplicación de configuración son propios y están escritos en Rust con GTK4. El objetivo posterior es sustituir labwc por **note-compositor**, construido con Smithay.

Qué significa “completo” en este manual

“Completo” significa que se explica la totalidad de la arquitectura existente, las responsabilidades de cada archivo, el flujo de arranque, las decisiones de código, los puntos débiles, los procedimientos de compilación, instalación y depuración, y una ruta concreta para extender el sistema. No significa que todas las funciones de un escritorio maduro ya estén implementadas.

Cómo usar esta documentación

Para empezar a modificar el proyecto, conviene leer en este orden:

1. [Capítulo 1](#), para obtener una copia compilable y un ciclo de trabajo seguro;
2. [Capítulo 2](#) y [Capítulo 4](#), para entender quién inicia a quién;
3. [Capítulo 6](#), [Capítulo 7](#), [Capítulo 8](#) y [Capítulo 9](#), para entender el código Rust;
4. [Capítulo 18](#), para agregar funciones sin desacomodar todo el condenado escritorio;
5. [Capítulo 21](#), antes de probar en hardware real.

El directorio `source_snapshot/` incluido junto con este manual contiene una fotografía de las fuentes utilizadas. El objetivo es que los fragmentos del documento y el código consultado correspondan entre sí.

Convenciones

Componente se escribe como **note-shell**.

Ruta se escribe como `/usr/bin/note-session`.

Comando se escribe como `cargo build --workspace --release`.

Estado actual describe lo que existe en 0.2.3.

Arquitectura objetivo describe lo planeado para el compositor Smithay y versiones posteriores.

Recomendación de seguridad

Haz las primeras modificaciones dentro de una máquina virtual y conserva acceso a una TTY. Un error en el panel suele ser recuperable; un error en el compositor o en greetd puede dejar la sesión gráfica en negro. La recuperación se documenta varias veces a propósito.

Índice general

Prefacio	I
Inicio rápido para desarrollo	XI
0.1. Requisitos del sistema de desarrollo	XI
0.2. Dependencias mínimas para compilar el código Rust	XI
0.3. Aplicar el parche posrelease de 0.2.3	XI
0.4. Ciclo rápido de compilación	XII
0.5. Instalación de desarrollo	XII
0.6. Recuperación inmediata	XII
1. Visión, objetivos y alcance real	1
1.1. Problema que intenta resolver	1
1.2. Principios de producto	1
1.2.1. Sencillez visible, complejidad encapsulada	1
1.2.2. Wayland como sesión nativa	1
1.2.3. Separación entre compositor y shell	1
1.2.4. Empaquetado nativo como destino	2
1.3. Qué existe en 0.2.3	2
1.4. Qué todavía no existe	2
2. Arquitectura general	4
2.1. Vista por capas	4
2.2. Procesos principales	4
2.3. Fronteras de responsabilidad	5
2.4. IPC y coordinación	5
2.5. Arquitectura objetivo	6
3. Árbol del proyecto y organización del repositorio	7
3.1. Estructura de alto nivel	7
3.2. Criterio de separación	8
3.3. Regla para decidir dónde poner código nuevo	8
3.4. Artefactos instalados	8
4. Arranque, inicio de sesión y ciclo de vida	10
4.1. Del kernel al escritorio	10
4.2. Configuración de greetd	11
4.3. Greeter con fallback gráfico	11
4.4. Variables establecidas por note-session	11
4.5. Bus D-Bus de sesión	12
4.6. Autostart y target de usuario	12
4.7. Cierre de sesión	12

5. Wayland, XWayland y labwc	13
5.1. Modelo Wayland de la sesión	13
5.2. Aplicaciones Wayland nativas	13
5.3. XWayland rootless	13
5.4. Por qué labwc es temporal	14
5.5. Configuración de labwc	14
5.6. Configuración combinada	15
5.7. Layer shell	15
5.8. Limitaciones de compatibilidad	15
6. Workspace Rust y dependencias	16
6.1. Manifiesto raíz	16
6.2. Resolver y edición	16
6.3. Dependencias principales	16
6.4. Dependencias por crate	17
6.4.1. note-core	17
6.4.2. note-shell	17
6.4.3. note-settings	17
6.5. Perfiles de compilación	17
6.6. Convenciones del código	18
6.7. Comandos de calidad	18
7. Crate note-core	19
7.1. Módulos públicos	19
7.2. Configuración: config.rs	19
7.2.1. AppearanceMode	19
7.2.2. NoteSettings	19
7.2.3. Carga y guardado	20
7.2.4. Variables CSS	20
7.2.5. Locale	20
7.3. Internacionalización: i18n.rs	20
7.4. Aplicaciones de escritorio: desktop.rs	21
7.4.1. Descubrimiento	21
7.4.2. Campos procesados	21
7.4.3. Lanzamiento	21
7.5. Adaptadores del sistema: system.rs	22
8. Crate note-shell	23
8.1. Estado global	23
8.2. GApplication e IPC	23
8.3. Reconstrucción y monitores	24
8.4. Panel superior	24
8.5. Dock	24
8.5.1. Lógica de cada icono	25
8.6. Overview	25
8.6.1. Búsqueda	25
8.6.2. Ventanas	25
8.6.3. Escritorios	25
8.7. Centro de control	25
8.8. CSS y preferencias GTK	26
8.9. Puntos de extensión recomendados	26
9. Crate note-settings	27

9.1. Estructura visual	27
9.2. Páginas	27
9.3. AppearanceControls	27
9.4. Flujo de guardado	28
9.5. Error E0382 corregido	28
9.6. Limitaciones del diseño actual	29
9.7. Ruta de evolución	29
10 Configuración, CSS, tema y recursos visuales	30
10.1 Jerarquía de configuración	30
10.2 settings.toml	30
10.3 CSS del shell	31
10.4 Transparencia frente a blur	31
10.5 Tema de ventanas	31
10.6 Fondos	31
10.7 Agregar un color de acento	32
10.8 Modificar estilos durante desarrollo	32
11 Internacionalización, idioma y región	34
11.1 Idiomas incluidos	34
11.2 Formato actual del catálogo	34
11.3 Selección y fallback	35
11.4 Locale de procesos	35
11.5 Agregar un idioma	35
11.6 Prueba automática de catálogos	35
11.7 Migración recomendada a Fluent real	36
12 Scripts y comandos auxiliares	37
12.1 Categorías	37
12.1.1 Entry points públicos	37
12.1.2 Scripts internos	37
12.2 Wrappers de aplicaciones	38
12.3 Capturas	38
12.4 Bloqueo	38
12.5 Aplicación de preferencias	38
12.6 Diagnóstico	38
12.7 Instalación y empaquetado	39
12.8 Principios para crear un script nuevo	39
13 systemd -user e integración con servicios del sistema	40
13.1 Por qué usar servicios de usuario	40
13.2 note-session.target	40
13.3 note-shell.service	40
13.4 Notificaciones y PolicyKit	41
13.5 Importación del entorno	41
13.6 PipeWire y WirePlumber	41
13.7 NetworkManager	41
13.8 BlueZ	41
13.9 PolicyKit	42
13.10 Power y batería	42
13.1 Portales XDG	42
13.12 Comandos de inspección	42

14GPU, DRM, NVIDIA y máquinas virtuales	43
14.1Ruta gráfica actual	43
14.2AMD e Intel	43
14.3NVIDIA	43
14.4Configuración gpu.conf	44
14.5Fallback a Pixman	44
14.6VirtualBox	44
14.7Diagnóstico observado	45
14.8Errores EGL comunes	45
14.9Diseño futuro multi-GPU	46
15Compilación e instalación	47
15.1Tres formas de trabajar	47
15.2Preflight	47
15.3Instalador directo	47
15.4Por qué se fuerza LC_ALL=C	48
15.5Paquetes core y opcionales	48
15.6install-files.sh y DESTDIR	48
15.7Instalación manual controlada	48
15.8Instalación limpia en VM	48
15.9Desinstalación	49
15.10Build reproducible	49
16Empaquetado y repositorios	50
16.1Por qué el instalador Bash no es el destino	50
16.2Paquete Debian local	50
16.3Metadatos actuales	50
16.4Scripts de mantenimiento Debian	50
16.5Debian packaging formal	51
16.6RPM	51
16.7Arch	51
16.8Paquetes planeados	51
16.9Repositorio APT propio	51
16.10Compatibilidad multiplataforma	52
17Pruebas, diagnóstico y depuración	53
17.1Pirámide de pruebas	53
17.2CI incluida	53
17.3Comandos básicos	53
17.4note-doctor	53
17.5Logs	54
17.6Prueba desde TTY	54
17.7Pruebas unitarias prioritarias	54
17.8Pruebas de instalación	54
17.9Pruebas visuales	55
18Guía práctica para modificar y extender Note Desktop	56
18.1Agregar una nueva preferencia	56
18.1.11. Modelo persistente	56
18.1.22. Control en Configuración	56
18.1.33. Uso en el shell	56
18.1.44. Pruebas	57
18.2Agregar una página de Configuración	57

18.3	Agregar un idioma	57
18.4	Cambiar favoritos del dock	57
18.5	Agregar una acción al centro de control	58
18.6	Agregar un nuevo wrapper	58
18.7	Agregar un servicio systemd -user	58
18.8	Modificar el panel	59
18.9	Agregar una acción GApplication	59
18.10	Crear un módulo nuevo en note-core	59
18.11	Trabajar sin reinstalar todo	60
18.12	Añadir logs	60
19	Calidad, refactorización y seguridad	61
19.1	Deuda técnica principal	61
19.2	Refactorización por etapas	61
19.2.1	Etapa 1: sin cambiar comportamiento	61
19.2.2	Etapa 2: backends abstractos	61
19.2.3	Etapa 3: eventos	61
19.3	Modelo de amenazas	62
19.4	Uso de shell	62
19.5	Privilegios	62
19.6	Bloqueo de sesión	62
19.7	IPC privado	62
19.8	Política de errores	63
19.9	Métricas útiles	63
20	Roadmap hacia note-compositor con Smithay	64
20.1	Objetivo	64
20.2	Fase 0.3: compositor anidado	64
20.3	Estructura propuesta	64
20.4	Fase 0.4: backend DRM/KMS	65
20.5	Fase 0.5: experiencia visual	65
20.6	Protocolo note-shell-v1	66
20.7	XWayland	66
20.8	Migración del shell	66
20.9	Criterios para retirar labwc	66
21	Problemas conocidos, historial de fallos y soluciones	68
21.1	policykit-1-gnome sin candidato	68
21.2	Todos los paquetes aparecían sin candidato	68
21.3	Fallo repetitivo de descarga de APT	68
21.4	greetd instalado pero no encontrado	68
21.5	API de gtk4-layer-shell	68
21.6	ApplicationFlags::DEFAULT_FLAGS	69
21.7	Movimiento parcial de controls.page	70
21.8	SnapToEdge combine	70
21.9	EGL en VirtualBox	70
21.10	note-doctor muestra salida Vulkan extensa	70
21.11	Herramientas externas ausentes	71
21.12	Lista de limitaciones funcionales	71
22	Flujo de contribución y mantenimiento	72
22.1	Repositorio Git	72
22.2	Formato de commits	72

22.3	Checklist de pull request	72
22.4	Versionado	73
22.5	Release	73
22.6	Matriz de pruebas recomendada	73
22.7	Documentación como requisito	73
A.	Referencia de rutas y variables	75
A.1.	Rutas del repositorio	75
A.2.	Rutas instaladas globales	75
A.3.	Rutas de usuario	76
A.4.	Variables de entorno	76
B.	Referencia de comandos	77
B.1.	Desarrollo	77
B.2.	Sesión	77
B.3.	systemd	77
B.4.	Gráficos	78
B.5.	Audio, red y Bluetooth	78
B.6.	Empaquetado	78
B.7.	Recuperación	78
C.	Referencia exhaustiva de archivos	79
D.	Listados técnicos seleccionados	84
D.1.	Configuración persistente completa	84
D.2.	Traductor completo	86
D.3.	Adaptadores del sistema completos	88
D.4.	Arranque de note-shell y acciones	90
D.5.	Construcción del panel	92
D.6.	Construcción del dock	94
D.7.	Overview	96
D.8.	Centro de control	100
D.9.	Configuración layer-shell y actualización de estado	103
D.10	Aplicación note-settings completa	105
D.11	Sesión	113
D.12	Aplicación de preferencias a labwc	114
D.13	Configuración completa de labwc	115
D.14	Unidades systemd -user	117
E.	Glosario	119
	Cierre	121

Índice de figuras

2.1. Capas de ejecución de Note Desktop 0.2.3.	4
2.2. Arquitectura objetivo cuando note-compositor sustituya a labwc.	6
4.1. Secuencia completa de inicio de sesión.	10
5.1. Convivencia de clientes Wayland y X11.	13
8.1. Superficies por monitor administradas por un proceso note-shell.	24
9.1. Aplicación de preferencias.	28
9.2. Error E0382 encontrado durante la compilación de note-settings.	29
10.1 Fuentes de configuración y precedencia conceptual.	30
10.2 Fondo principal note-wave incluido en el proyecto.	32
14.1 VirtualBox con VMSVGA y 128 MB; la opción 3D se habilita con la VM apagada.	45
14.2 Salida real de note-doctor durante el desarrollo.	45
16.1 Camino para llegar a una instalación APT de una línea.	52
21.1 Errores de tipos provocados por diferencias de API de gtk4-layer-shell. .	69
21.2 Error E0599 por una constante no disponible.	70

Índice de cuadros

2.1. Responsabilidades actuales de los componentes.	5
7.1. Adaptadores del sistema en note-core.	22

Inicio rápido para desarrollo

Este capítulo permite comenzar a moverle al proyecto sin leer todavía todos los detalles internos. El flujo recomendado separa cuatro actividades: preparar dependencias, aplicar el parche conocido, compilar, y probar desde una TTY.

0.1 Requisitos del sistema de desarrollo

Para Debian 13 o Ubuntu reciente se requiere:

- una instalación Linux funcional con systemd;
- acceso a Internet para APT y Cargo;
- un usuario normal con sudo, o acceso root;
- al menos 4 GiB de almacenamiento temporal;
- preferentemente 4 GiB de RAM y dos núcleos para compilar en una VM;
- acceso a una TTY mediante Ctrl+Alt+F2 o Ctrl+Alt+F3.

0.2 Dependencias mínimas para compilar el código Rust

Listing 1: Dependencias mínimas de compilación en Debian 13

```
1 sudo apt update
2 sudo apt install -y \
3     build-essential pkg-config rustc cargo \
4     libgtk-4-dev libgtk4-layer-shell-dev
```

Estas dependencias permiten compilar **note-core**, **note-shell** y **note-settings**. Para ejecutar la sesión completa se requieren además labwc, XWayland, greetd, cage, gtk-greet, servicios de audio, red y otras utilidades descritas en [Capítulo 15](#).

0.3 Aplicar el parche posrelease de 0.2.3

La fuente 0.2.3 original entregaba `controls.page` por valor y después intentaba clonar la estructura `controls`. Rust lo rechaza con E0382 porque se produjo un movimiento parcial. El snapshot de este manual ya trae el arreglo:

Listing 2: Corrección necesaria en note-settings

```
1 add_page(
2     &sidebar,
3     &stack,
4     "appearance",
```

```

5     "preferences-desktop-theme-symbolic",
6     &tr.text("appearance"),
7     controls.page.clone(),
8 );

```

En una copia sin corregir se puede aplicar:

```

1 sed -i \
2     's/controls\.page);/controls.page.clone());/' \
3     crates/note-settings/src/main.rs

```

0.4 Ciclo rápido de compilación

Listing 3: Comprobación y compilación del workspace

```

1 cargo fmt --all -- --check
2 cargo check --workspace --all-targets
3 cargo test --workspace
4 cargo build --workspace --release

```

Los binarios se generan en:

- target/release/note-shell;
- target/release/note-settings.

Durante desarrollo de interfaz no hace falta reinstalar todo después de cada cambio. Puedes ejecutar el binario desde el árbol de compilación dentro de una sesión Wayland compatible:

```

1 ./target/debug/note-settings
2 ./target/debug/note-shell --version

```

Para probar superficies layer-shell, **note-shell** necesita un compositor que implemente wl-layer-shell; labwc, Sway y otros compositores wlroots suelen servir.

0.5 Instalación de desarrollo

```

1 chmod +x scripts/*
2 ./scripts/preflight.sh
3 ./scripts/install.sh

```

El instalador compila en modo release, instala archivos en /usr, configura greetd y habilita el arranque gráfico. No reinicies a ciegas. Primero ejecuta:

```

1 note-doctor
2 note-test-from-tty

```

0.6 Recuperación inmediata

Si la pantalla de acceso no aparece o queda negra:

```
1 # Desde Ctrl+Alt+F2 o Ctrl+Alt+F3
2 sudo systemctl disable --now greetd
3 sudo systemctl set-default multi-user.target
4 sudo reboot
```

Tu ciclo de trabajo recomendado

Edita una pieza, ejecuta cargo check, compila sólo el crate afectado, prueba en una sesión anidada o VM, y sólo después instala en el sistema. Ese orden evita perder tiempo reiniciando por cada cambio y reduce el riesgo de dejar greetd en un bucle de fallos.

Visión, objetivos y alcance real

1.1 Problema que intenta resolver

Linux ofrece escritorios muy maduros, pero desarrollar uno propio es una forma excelente de estudiar sistemas operativos, gráficos, IPC, servicios de usuario, accesibilidad, empaquetado y diseño de interfaces. Note Desktop nace como un escritorio experimental con tres metas principales:

1. mantener una experiencia sencilla y centrada, evitando llenar la pantalla de controles;
2. adoptar una composición visual cuidada, con barra superior y dock flotante;
3. construir gradualmente una pila técnica moderna basada en Wayland y Rust.

1.2 Principios de producto

1.2.1 Sencillez visible, complejidad encapsulada

El usuario debería encontrar pocas opciones principales y rutas claras. La complejidad de PipeWire, NetworkManager, BlueZ, PolicyKit o portales no debe derramarse en cada pantalla. En la alpha, esa encapsulación todavía se logra mediante pequeños wrappers y herramientas externas; en versiones futuras deberá pasar a APIs D-Bus y servicios propios.

1.2.2 Wayland como sesión nativa

No existe una sesión Note sobre Xorg. Las aplicaciones Wayland se conectan directamente al compositor y las aplicaciones X11 se ejecutan mediante XWayland rootless. Esta decisión reduce duplicación de código y alinea el desarrollo con la arquitectura gráfica moderna.

1.2.3 Separación entre compositor y shell

El compositor administra superficies, entrada y presentación. El shell dibuja panel, dock, overview y controles. Incluso cuando exista **note-compositor**, **note-shell** debe seguir siendo un proceso separado; así una caída del dock no mata todas las aplicaciones.

1.2.4 Empaquetado nativo como destino

El script `scripts/install.sh` es una herramienta de bootstrap. El destino correcto es que las dependencias y actualizaciones sean gestionadas por APT, DNF o Pacman. El proyecto ya contiene esqueletos de empaquetado, pero todavía no un repositorio público firmado.

1.3 Qué existe en 0.2.3

Área	Implementación actual
Compositor	labwc, configurado y tematizado por Note.
Shell	Rust + GTK4 + gtk4-layer-shell.
Panel	Una superficie por monitor, reloj, aplicación activa, batería y acceso al centro de control.
Dock	Favoritos configurables, indicador de ejecución, enfoque o lanzamiento.
Overview	Búsqueda de aplicaciones, lista de ventanas y botones de escritorios.
Configuración	Aplicación propia con apariencia, idioma y accesos a herramientas externas.
Login	greetd + cage + gtkgreet tematizado.
X11	XWayland iniciado y administrado por labwc.
Audio	PipeWire/WirePlumber mediante wpctl.
Red	NetworkManager mediante nmcli y nm-connection-editor.
Bluetooth	BlueZ mediante bluetoothctl y Blueman opcional.
Notificaciones	mako.
Bloqueo	gklock, swaylock o fallback a loginctl.
Portales	xdg-desktop-portal-gtk y xdg-desktop-portal-wlr.

1.4 Qué todavía no existe

- compositor propio con Smithay;
- blur real detrás de superficies;
- animaciones de ventanas controladas por el compositor;
- miniaturas en vivo del overview;
- escritorios dinámicos como GNOME;
- configuración nativa de pantallas, red, Bluetooth y energía;
- greeter propio;
- backend de portal propio;
- accesibilidad completa;
- repositorios binarios públicos y firmados;
- migraciones de configuración entre versiones.

No confundas estética con compositor

Transparencia CSS no es blur. Redondear una caja GTK no redondea el contenido de una ventana arbitraria. Minimizar una aplicación con una animación hacia el dock requiere cooperación del compositor. Estas funciones no deben implementarse con trucos frágiles en el shell: pertenecen a **note-compositor**.

Arquitectura general

2.1 Vista por capas

La arquitectura actual puede leerse de arriba hacia abajo como cuatro capas: acceso, sesión, compositor y servicios/aplicaciones.

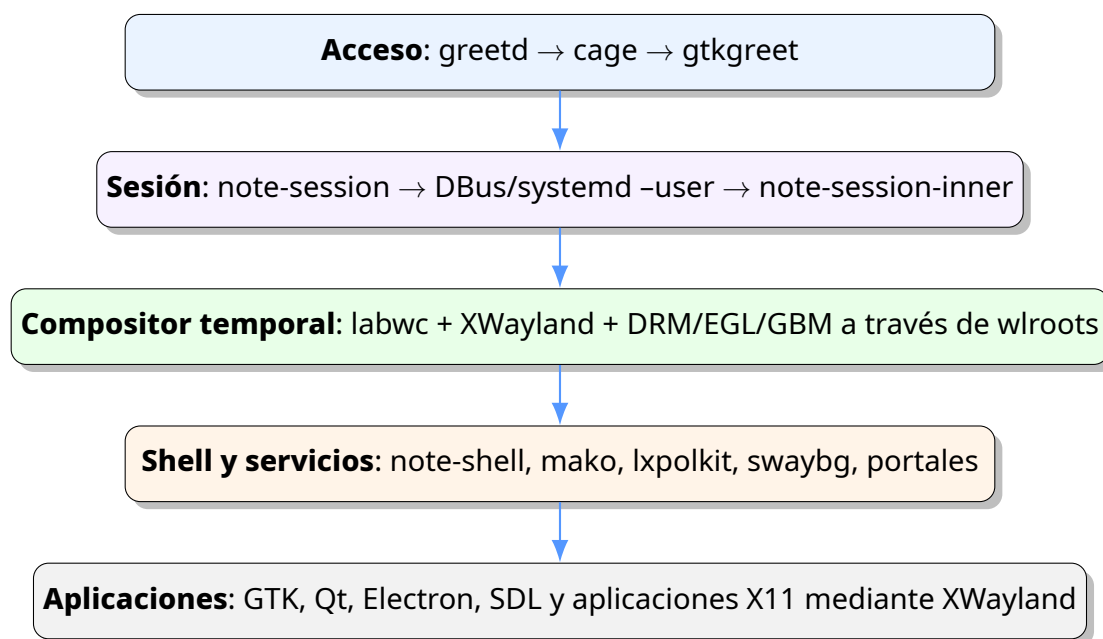


Figura 2.1: Capas de ejecución de Note Desktop 0.2.3.

2.2 Procesos principales

greetd daemon de acceso. Reserva una TTY y ejecuta una sesión de greeter como el usuario restringido greeter.

cage compositor Wayland de una sola aplicación, usado exclusivamente para contener gtkgreet.

gtkgreet interfaz de autenticación. Al aceptar credenciales pide a greetd ejecutar /usr/bin/note-session como el usuario seleccionado.

note-session prepara variables de entorno, locale, configuración de GPU y bus D-Bus.

labwc compositor temporal y administrador de ventanas.

note-shell panel, dock, overview y centro de control.

mako servidor de notificaciones.

lxpolkit agente gráfico de autorización PolicyKit.

swaybg fondo de pantalla.

2.3 Fronteras de responsabilidad

Una frontera clara evita que el proyecto se convierta en un programa monolítico imposible de mantener.

Componente	Debe hacer	No debe hacer
note-core	Modelos, configuración, traducción, descubrimiento de aplicaciones y adaptadores del sistema.	Crear ventanas GTK o decidir geometría del shell.
note-shell	Dibujar superficies y manejar interacción del shell.	Controlar DRM/KMS o implementar autenticación.
note-settings	Editar preferencias y activar recargas.	Reinventar NetworkManager, PipeWire o BlueZ en esta etapa.
labwc	Administrar ventanas, entrada, escritorios y XWayland.	Conocer detalles internos de la interfaz Note.
scripts	Unir herramientas existentes y ofrecer fallbacks.	Contener lógica de negocio compleja a largo plazo.

Cuadro 2.1: Responsabilidades actuales de los componentes.

2.4 IPC y coordinación

La alpha utiliza varias formas de comunicación:

- acciones GApplication para mostrar overview, centro de control y recargar el shell;
- wlrcctl para enumerar, enfocar y simular acciones de teclado;
- comandos de usuario como wpctl, nmcli, bluetoothctl y brightnessctl;
- systemd -user para iniciar y reiniciar servicios;
- variables de entorno para indicar toolkit, locale, portales y backend gráfico.

Deuda técnica intencional

Usar comandos externos permitió levantar rápido la alpha y observar el flujo completo. Para 1.0, las operaciones frecuentes deben migrar a APIs D-Bus o bibliotecas Rust: reducen procesos, eliminan parsing frágil y permiten devolver

errores precisos a la interfaz.

2.5 Arquitectura objetivo

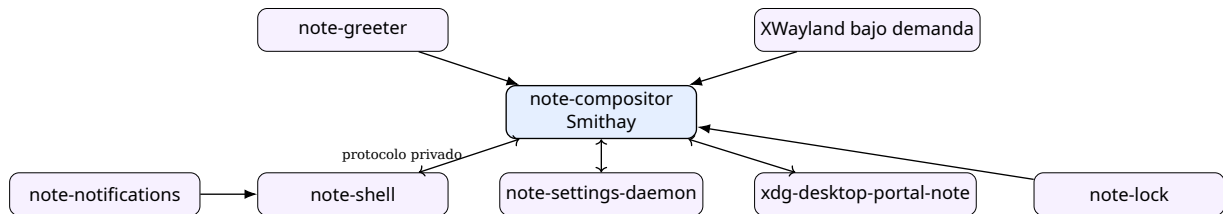


Figura 2.2: Arquitectura objetivo cuando note-compositor sustituya a labwc.

Árbol del proyecto y organización del repositorio

3.1 Estructura de alto nivel

Listing 3.1: Estructura principal del repositorio

```
1 note-desktop-0.2.3/|—
2   Cargo.toml|—
3   Makefile|—
4   VERSION|—
5   README.md|—
6   LICENSE|—
7   crates/|
8     |— note-core/|
9     |— note-shell/|
10    |— note-settings/|—
11  assets/|
12    |— icons/|
13    |— style/|
14    |— themes/|
15    |— wallpapers/|—
16  configs/|
17    |— greetd/|
18    |— labwc/|
19    |— wayland-session/|
20    |— xdg-desktop-portal/|—
21  locales/|—
22  scripts/|—
23  systemd/user/|—
24  packaging/|
25    |— debian/|
26    |— rpm/|
27    |— arch/|—
28  docs/|—
29  .github/workflows/
```

3.2 Criterio de separación

crates/ contiene código Rust compilable. El workspace se divide por responsabilidad, no por pantalla.

assets/ contiene recursos instalables que no son lógica: CSS, iconos, fondos y tema de decoración.

configs/ contiene configuración global predeterminada para servicios externos.

locales/ contiene catálogos de texto incorporados en tiempo de compilación.

scripts/ contiene entrypoints, adaptadores y procedimientos de instalación.

systemd/user/ describe el ciclo de vida de procesos de la sesión del usuario.

packaging/ adapta el mismo árbol a distintos gestores de paquetes.

docs/ contiene documentación breve del repositorio; este manual amplía y corrige esa información.

3.3 Regla para decidir dónde poner código nuevo

Antes de crear un archivo, pregunta qué responsabilidad tiene:

1. ¿Es lógica reutilizable sin GTK? Colócala en **note-core**.
2. ¿Dibuja o coordina una superficie del escritorio? Colócala en **note-shell**.
3. ¿Edita preferencias persistentes? Colócala en **note-settings** o en un futuro daemon.
4. ¿Sólo selecciona entre herramientas instaladas? Puede comenzar como script.
5. ¿Es configuración predeterminada de otro servicio? Colócala en **configs/**.
6. ¿Debe arrancar y reiniciarse con la sesión? Define una unidad **systemd -user**.

Evita seguir engordando main.rs

`crates/note-shell/src/main.rs` supera las 900 líneas y `note-settings/src/main.rs` ronda las 400. Antes de agregar funciones grandes, conviene dividirlos en módulos. En [Capítulo 19](#) se propone una refactorización concreta.

3.4 Artefactos instalados

El script `scripts/install-files.sh` transforma el árbol de fuentes en una jerarquía de sistema:

Destino	Contenido
<code>/usr/bin/note-shell</code>	Binario del shell.
<code>/usr/bin/note-settings</code>	Binario de configuración.
<code>/usr/bin/note-*</code>	Wrappers y comandos de sesión.
<code>/usr/libexec/note-desktop/</code>	Scripts internos que no son interfaz pública.
<code>/usr/share/wayland-sessions/note.desktop</code>	Entrada de sesión para display managers.

<code>/usr/share/applications/ note-settings.desktop</code>	Lanzador de Configuración.
<code>/usr/share/note-desktop/</code>	CSS, locales y fondos.
<code>/usr/share/themes/Note/</code>	Tema de decoración para labwc.
<code>/etc/xdg/note/labwc/</code>	Configuración global combinable de labwc.
<code>/etc/greetd/</code>	Configuración y CSS del greeter.
<code>/usr/lib/systemd/user/</code>	Unidades de sesión.

La referencia exhaustiva archivo por archivo aparece en el [Apéndice C](#).

Arranque, inicio de sesión y ciclo de vida

4.1 Del kernel al escritorio

Cuando el sistema tiene `graphical.target` como objetivo predeterminado, `systemd` inicia **greetd**. El archivo `/etc/greetd/config.toml` apunta al script interno `note-greeter-session`. Éste ejecuta `cage` y `gtk-greet`. Después de autenticar, `gtk-greet` solicita a `greetd` arrancar `/usr/bin/note-session` con la identidad del usuario.

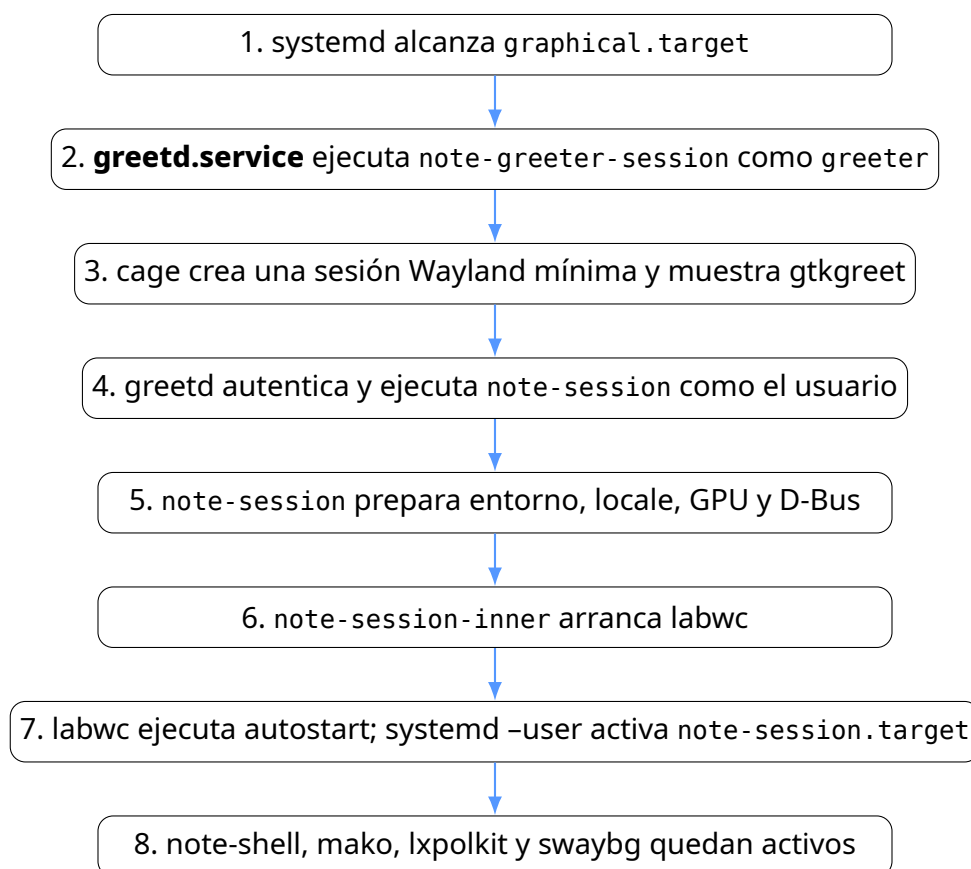


Figura 4.1: Secuencia completa de inicio de sesión.

4.2 Configuración de greetd

Listing 4.1: Configuración instalada de greetd

```

1 [terminal]
2 vt = 1
3
4 [default_session]
5 command = "/usr/libexec/note-desktop/note-greeter-session"
6 user = "greeter"

```

vt = 1 reserva la primera terminal virtual. default_session.command no llama directamente a gtkgreet; usa un wrapper que puede reintentar con renderizado por software. Esto fue necesario para VirtualBox y VMware, donde EGL puede fallar aun cuando el controlador virtual esté presente.

4.3 Greeter con fallback gráfico

El script note-greeter-session intenta primero la ruta acelerada. Si cage termina con error y WLR_RENDERER no era ya Pixman, configura:

```

1 export WLR_RENDERER=pixman
2 export WLR_NO_HARDWARE_CURSORS=1
3 exec cage -s -- gtkgreet ...

```

Este fallback aumenta compatibilidad, pero no debe ocultar errores indefinidamente. En un sistema de producción conviene registrar el fallo, marcar el modo seguro en la interfaz y permitir al usuario volver a intentar la ruta acelerada.

4.4 Variables establecidas por note-session

Variable	Propósito
XDG_SESSION_TYPE=wayland	Identifica la sesión como Wayland.
XDG_SESSION_DESKTOP=note	Nombre de la sesión para integración.
XDG_CURRENT_DESKTOP=Note: labwc:wlroots	Permite que portales y aplicaciones detecten Note y sus compatibilidades.
XDG_CONFIG_DIRS	Anteponer /etc/xdg/note a la búsqueda de configuración.
GTK_USE_PORTAL=1	Solicitar que GTK use portales cuando corresponda.
MOZ_ENABLE_WAYLAND=1	Preferir Wayland en aplicaciones Mozilla.
QT_QPA_PLATFORM=wayland;xcb	Preferir Wayland y permitir fallback XCB.
SDL_VIDEODRIVER=wayland,x11	Preferir Wayland en SDL.
ELECTRON_OZONE_PLATFORM_ HINT=auto	Permitir selección moderna de backend en Electron.
_JAVA_AWT_WM_ NONREParenting=1	Mejorar compatibilidad de Java con WMs no reparenting.

LABWC_UPDATE_ACTIVATION_ ENV=1	Actualizar variables de activación en el entorno de labwc.
-----------------------------------	---

4.5 Bus D-Bus de sesión

`note-session` verifica si existe `$XDG_RUNTIME_DIR/bus`. Si existe, reutiliza el bus administrado por `systemd`. Si no, usa `dbus-run-session`. Después `note-session-inner` importa variables al entorno de activación D-Bus y a `systemd -user`.

Este paso es indispensable: sin él, servicios activados por D-Bus pueden desconocer `WAYLAND_DISPLAY`, el locale o el escritorio activo.

4.6 Autostart y target de usuario

Labwc ejecuta `/etc/xdg/note/labwc/autostart`, que lanza `note-autostart`. Este último:

1. reinicia el proceso de fondo de pantalla;
2. ejecuta `swaybg` con el fondo Note;
3. importa el entorno Wayland a `systemd -user`;
4. inicia `graphical-session.target` y `note-session.target`;
5. usa un fallback manual si la instancia de `systemd -user` no está disponible.

4.7 Cierre de sesión

`note-logout` ordena a labwc salir. Al morir el compositor se cierran las conexiones Wayland y termina la sesión. `greeterd` vuelve a mostrar el greeter.

Por qué no se debe matar `note-shell` para cerrar sesión

note-shell es sólo la interfaz. Matarlo debería provocar que `systemd` lo reinicie y las aplicaciones permanezcan abiertas. El cierre de sesión debe terminar el compositor o utilizar una API de sesión explícita.

Wayland, XWayland y labwc

5.1 Modelo Wayland de la sesión

En Wayland, el compositor es simultáneamente servidor de ventanas, administrador de entrada y coordinador de presentación. Los clientes entregan buffers y metadatos; el compositor decide cuándo, dónde y cómo mostrarlos. Note Desktop no ofrece una sesión Xorg alternativa.

5.2 Aplicaciones Wayland nativas

GTK4, Qt, SDL, Electron y Firefox pueden abrir superficies Wayland directamente. Las variables de `note-session` intentan que los toolkits prefieran este camino. Una aplicación Wayland no recibe coordenadas globales arbitrarias ni controla el escritorio completo; las operaciones privilegiadas se realizan mediante protocolos y portales.

5.3 XWayland rootless

XWayland es un servidor X que funciona como cliente Wayland. Labwc lo administra y presenta cada ventana X11 como una ventana individual. El usuario no entra en una sesión Xorg: XWayland existe únicamente como compatibilidad.

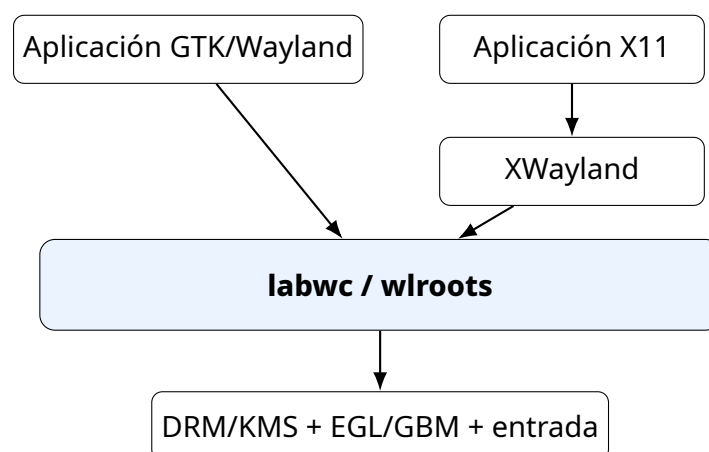


Figura 5.1: Convivencia de clientes Wayland y X11.

5.4 Por qué labwc es temporal

Labwc proporciona una base estable y ligera sobre wlroots. Permite concentrar el trabajo 0.2 en el shell. Sin embargo, las siguientes funciones dependen del compositor y no pueden resolverse de forma correcta desde GTK:

- blur real y sombras de ventanas;
- animaciones de apertura, cierre, minimización y cambio de escritorio;
- miniaturas vivas y transformaciones de superficies;
- integración profunda del dock con ventanas;
- escritorios dinámicos;
- gestos globales;
- administración directa de múltiples GPU, DMA-BUF y sincronización explícita.

5.5 Configuración de labwc

El archivo `configs/labwc/rc.xml` define:

- decoración del lado del servidor;
- separación de 10 píxeles entre ventanas y bordes;
- cuatro escritorios iniciales;
- tipografía Inter;
- botones de ventana en orden cerrar, maximizar y minimizar a la izquierda;
- atajos globales;
- comportamiento de mouse y touchpad;
- persistencia de XWayland desactivada.

Listing 5.1: Núcleo y tema de la configuración de labwc

```

1 <?xml version="1.0"?>
2 <labwc_config xmlns="http://openbox.org/3.4/rc">
3   <core>
4     <decoration>server</decoration>
5     <gap>10</gap>
6     <adaptiveSync>yes</adaptiveSync>
7     <reuseOutputMode>yes</reuseOutputMode>
8     <xwaylandPersistence>no</xwaylandPersistence>
9   </core>
10
11   <theme>
12     <name>Note</name>
13     <icon>Adwaita</icon>
14     <cornerRadius>12</cornerRadius>
15     <titlebar><layout>close,max,iconify:</layout></titlebar>
16     <font place="ActiveWindow"><name>Inter</name><size>10</size><weight>Semibold</weight></font>
17     <font place="InactiveWindow"><name>Inter</name><size>10</size></font>
18     <font place="MenuItem"><name>Inter</name><size>10</size></font>
19     <font place="OnScreenDisplay"><name>Inter</name><size>11</size><weight>Bold</weight></font>
20   </theme>
21
22   <desktops number="4">
23     <popupTime>650</popupTime>

```

```
24 <prefix>Escritorio</prefix>  
25 </desktops>
```

5.6 Configuración combinada

Labwc se ejecuta con `labwc --merge-config`. Esto permite combinar la configuración global de Note con `~/.config/labwc/rc.xml`. El script `note-apply-settings` genera un archivo de usuario pequeño que sólo cambia cantidad de escritorios y desplazamiento natural.

5.7 Layer shell

note-shell usa el protocolo `layer-shell` para colocar superficies en capas especiales:

Top panel y dock;

Overlay overview y centro de control.

El panel reserva una zona exclusiva para que las ventanas maximizadas no lo cubran. El dock usa zona exclusiva cero, por lo que flota sobre el contenido.

5.8 Limitaciones de compatibilidad

La alpha presupone un compositor compatible con `layer-shell` y herramientas de `wl-roots`. Aunque `GTK4` es portable, el shell no funcionará igual sobre `GNOME Shell` o `KDE Plasma` porque esos compositores no exponen necesariamente los mismos protocolos de administración de ventanas.

Diseño para el futuro

Cuando exista `note-compositor`, el shell debería hablar un protocolo privado versionado para enumerar ventanas, escritorios y estados. Las funciones básicas pueden seguir exponiendo protocolos estándar para terceros, pero la interfaz privilegiada no debe depender de parsing de texto de `wlrctl`.

Workspace Rust y dependencias

6.1 Manifiesto raíz

El archivo Cargo.toml define un workspace con tres crates y centraliza las versiones compartidas.

Listing 6.1: Cargo.toml del workspace

```
1 [workspace]
2 resolver = "2"
3 members = [
4     "crates/note-core",
5     "crates/note-shell",
6     "crates/note-settings",
7 ]
8
9 [workspace.package]
10 version = "0.2.3"
11 edition = "2021"
12 rust-version = "1.83"
13 license = "GPL-3.0-or-later"
14 authors = ["Gonzo M"]
15
16 [workspace.dependencies]
17 chrono = "0.4"
18 gtk = { package = "gtk4", version = "0.10.3", features = ["v4_10"] }
19 gtk4-layer-shell = "0.7.1"
20 serde = { version = "1", features = ["derive"] }
21 toml = "0.8"
```

6.2 Resolver y edición

resolver = "2" activa el resolovedor moderno de características de Cargo. La edición 2021 ofrece una base estable y compatible con el compilador disponible en Debian 13. El proyecto declara rust-version = "1.83"; Cargo debería rechazar compiladores anteriores antes de entrar en errores difíciles de interpretar.

6.3 Dependencias principales

Crate	Uso
chrono	Reloj, fecha, modo automático claro/oscuro y calendario.
gtk4 0.10.3	Widgets, aplicación, acciones, CSS, monitores y bucle principal.
gtk4-layer-shell 0.7.1	Superficies ancladas a bordes y capas del compositor.
serde	Serialización y deserialización de preferencias.
toml	Lectura y escritura de settings.toml.

6.4 Dependencias por crate

6.4.1 note-core

No depende de GTK. Esta separación es importante porque permite probar lógica de configuración y parsing sin una sesión gráfica.

6.4.2 note-shell

Depende de GTK, layer-shell, chrono y note-core. Contiene la mayor parte de la interfaz y actualmente concentra demasiadas responsabilidades en un solo archivo.

6.4.3 note-settings

Depende de GTK y note-core. No necesita layer-shell porque es una ventana de aplicación normal.

6.5 Perfiles de compilación

El proyecto usa los perfiles predeterminados de Cargo. Para desarrollo rápido:

```
1 cargo check -p note-shell
2 cargo run -p note-settings
```

Para instalación y paquete:

```
1 cargo build --workspace --release
```

Una optimización futura razonable para releases es:

Listing 6.2: Perfil release propuesto

```
1 [profile.release]
2 lto = "thin"
3 codegen-units = 1
4 strip = "symbols"
5 panic = "abort"
```

No conviene agregarlo antes de medir tiempos de compilación, tamaño y calidad de backtraces.

6.6 Convenciones del código

El código actual usa:

- `Rc<RefCell<T>>` para estado mutable compartido en el hilo principal de GTK;
- clones ligeros de objetos GTK, que son referencias contadas a objetos `GObject`;
- funciones libres para construir widgets;
- procesos externos para integración del sistema;
- fallbacks silenciosos en varias operaciones.

6.7 Comandos de calidad

```
1 cargo fmt --all
2 cargo clippy --workspace --all-targets -- -D warnings
3 cargo test --workspace
4 cargo doc --workspace --no-deps --open
```

Faltan pruebas unitarias

El workflow ejecuta `cargo test`, pero 0.2.3 casi no contiene pruebas. Antes de refactorizar el parser de archivos desktop, configuración o traducción, crea pruebas que congelen el comportamiento actual.

Crate note-core

note-core concentra lógica reutilizable y deliberadamente evita GTK. Exporta configuración, traducciones, descubrimiento de aplicaciones y adaptadores del sistema.

7.1 Módulos públicos

Listing 7.1: Módulos exportados por note-core

```
1 pub mod config;
2 pub mod desktop;
3 pub mod i18n;
4 pub mod system;
5
6 pub use config::{AppearanceMode, NoteSettings};
7 pub use desktop::{DesktopApp, discover_apps};
8 pub use i18n::Translator;
```

7.2 Configuración: config.rs

7.2.1 AppearanceMode

AppearanceMode representa tres modos:

- Auto: oscuro fuera del rango 07:00–18:59;
- Light: tema claro;
- Dark: tema oscuro.

La serialización usa nombres kebab-case. En TOML aparecen como auto, light y dark.

7.2.2 NoteSettings

Listing 7.2: Modelo persistente de preferencias

```
1 pub struct NoteSettings {
2     pub language: String,
3     pub appearance: AppearanceMode,
4     pub accent: String,
5     pub dock_size: i32,
```

```

6   pub panel_opacity: f64,
7   pub dock_opacity: f64,
8   pub animations: bool,
9   pub natural_scroll: bool,
10  pub workspaces: u8,
11  pub favorites: Vec<String>,
12 }

```

El atributo `#[serde(default)]` permite cargar archivos antiguos o incompletos. Los campos ausentes reciben valores de Default.

7.2.3 Carga y guardado

`NoteSettings::load()` lee `~/.config/note-desktop/settings.toml`. Cualquier error de lectura o parseo devuelve la configuración predeterminada. Esto evita que el shell se caiga, pero puede ocultar un archivo corrupto.

Mejora recomendada

Distingue entre “archivo no existe” y “archivo inválido”. En el segundo caso, conserva el archivo, registra el error y muestra una advertencia en Configuración. Sobrescribir después con defaults podría destruir preferencias recuperables.

7.2.4 Variables CSS

`css_variables()` genera tres variables GTK CSS:

```

1 @define-color note_accent rgb(...);
2 @define-color note_panel_bg rgba(...);
3 @define-color note_dock_bg rgba(...);

```

Las opacidades se limitan entre 0.25 y 1.0. Esta capa permite cambiar acento y transparencia sin reescribir el CSS base.

7.2.5 Locale

`write_locale_environment()` genera `~/.config/environment.d/90-note-locale.conf` con LANG, LANGUAGE y LC_MESSAGES. El idioma BCP-47 se transforma a nombre POSIX reemplazando guion por guion bajo.

7.3 Internacionalización: i18n.rs

Los catálogos `shell.ftl` se incorporan al binario con `include_str!`. Translator selecciona un catálogo y usa inglés como fallback.

Listing 7.3: Resolución de una clave traducida

```

1 pub fn text(&self, key: &str) -> String {
2     self.values
3         .get(key)
4         .or_else(|| self.fallback.get(key))
5         .cloned()
6         .unwrap_or_else(|| key.to_owned())
7 }

```

Aunque los archivos usan extensión `.ftl`, el parser no implementa Fluent completo. Sólo acepta líneas `clave = valor`, ignora comentarios y términos que comienzan con guion, y realiza reemplazo textual simple.

7.4 Aplicaciones de escritorio: desktop.rs

7.4.1 Descubrimiento

Se recorren en orden:

1. `~/.local/share/applications;`
2. `/usr/local/share/applications;`
3. `/usr/share/applications.`

El primer archivo con un desktop ID determinado gana. Esto respeta sobrescrituras de usuario.

7.4.2 Campos procesados

- Type;
- Name localizado;
- GenericName localizado;
- Icon;
- Exec;
- StartupWMClass;
- Terminal;
- Categories;
- NoDisplay y Hidden.

No se implementan todavía TryExec, OnlyShowIn, NotShowIn, DBusActivatable, acciones ni validación completa de códigos de campo.

7.4.3 Lanzamiento

`DesktopApp::launch()` limpia códigos de campo simples y ejecuta `sh -lc`. Para aplicaciones de terminal envuelve el comando en `note-terminal -e`.

Superficie de seguridad

Los archivos `.desktop` de usuario son datos ejecutables. El uso de `sh -lc` permite sintaxis de shell. No debes mostrar ni ejecutar archivos desktop descargados de fuentes no confiables. Una implementación madura debe analizar Exec según la especificación y ejecutar `argv` sin shell cuando sea posible.

7.5 Adaptadores del sistema: system.rs

Función	Herramienta externa
<code>list_toplevels / focus_toplevel</code>	<code>wlrcctl</code>
<code>volume_percent / set_volume</code>	<code>wpctl</code>
<code>brightness_percent / set_brightness</code>	<code>brightnessctl</code>
<code>wifi_enabled / set_wifi</code>	<code>nmcli</code>
<code>bluetooth_enabled / set_bluetooth</code>	<code>bluetoothctl</code>

Cuadro 7.1: Adaptadores del sistema en note-core.

Estos adaptadores son pequeños y legibles, pero lanzan procesos y dependen de salida textual. El siguiente paso es definir traits, por ejemplo `AudioBackend`, `NetworkBackend` y `ToplevelBackend`, con implementaciones reales y dobles de prueba.

Crate note-shell

note-shell es una aplicación GTK4 de una sola instancia. Crea varias `GtkApplicationWindow` y las convierte en superficies `layer-shell`. El estado se comparte mediante `Rc<RefCell<ShellState>` porque GTK ejecuta callbacks en el hilo principal.

8.1 Estado global

Listing 8.1: Estado compartido del shell

```
1 #[derive(Default)]
2 struct ShellState {
3     surfaces: Vec<gtk::ApplicationWindow>,
4     overview: Option<gtk::ApplicationWindow>,
5     control_center: Option<gtk::ApplicationWindow>,
6     settings: NoteSettings,
7     apps: Vec<DesktopApp>,
8 }
```

`surfaces` almacena paneles y docks por monitor. `Overview` y centro de control son superficies únicas creadas bajo demanda. Las preferencias y la lista de aplicaciones se recargan al reconstruir el shell.

8.2 GApplication e IPC

El ID de aplicación es `mx.note.desktop.shell`. Se registran cuatro acciones:

- `overview`;
- `control-center`;
- `reload`;
- `hide-overview`.

Desde shell se pueden invocar así:

```
1 gapplication action mx.note.desktop.shell overview
2 gapplication action mx.note.desktop.shell control-center
3 gapplication action mx.note.desktop.shell reload
```

Esto evita crear múltiples procesos para mostrar el overview. Los scripts `note-overview` y `note-control-center` arrancan el shell sólo si la acción falla.

8.3 Reconstrucción y monitores

`rebuild_shell()` cierra superficies anteriores, carga preferencias, descubre aplicaciones, instala CSS y consulta monitores desde GDK. Por cada monitor crea un panel y un dock.

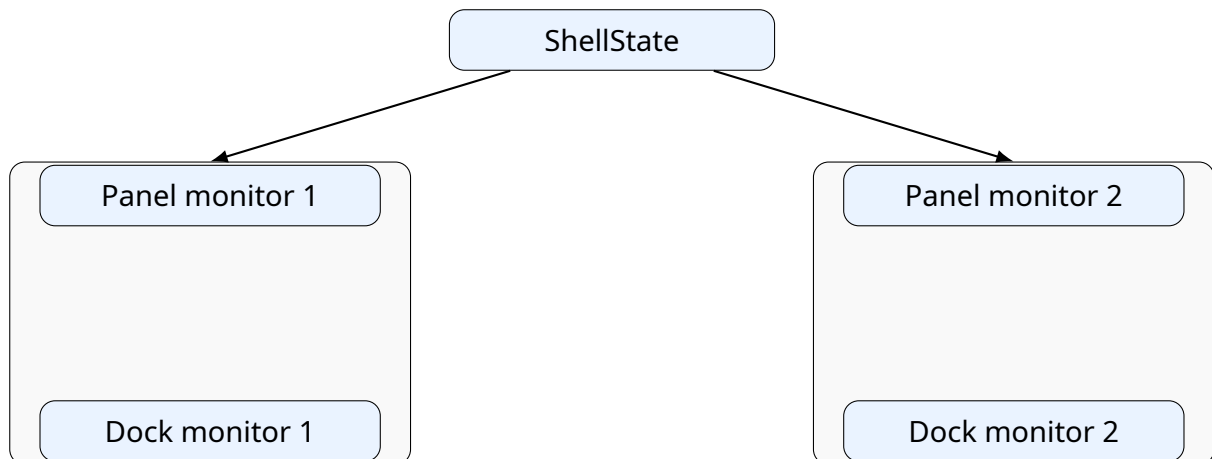


Figura 8.1: Superficies por monitor administradas por un proceso note-shell.

8.4 Panel superior

`create_panel()` configura una ventana `layer-shell` en capa `Top`, anclada a arriba, izquierda y derecha. `auto_exclusive_zone_enable()` reserva el alto del panel.

El panel se construye con `GtkCenterBox`:

- inicio: botón Note y aplicación activa;
- centro: reloj con popover de calendario;
- final: porcentaje de batería y clúster de estado.

La aplicación activa se consulta cada segundo mediante `wlrcctltoplevelliststate:active`. El reloj también se actualiza cada segundo y la batería cada 30 segundos.

Costo del polling

La alpha prioriza simplicidad. Sin embargo, lanzar `wlrcctl` cada segundo es caro y puede producir retrasos en máquinas virtuales lentas. El compositor propio debe emitir eventos de foco; la batería debe llegar por `UPower D-Bus`.

8.5 Dock

El dock se ancla abajo en capa `Top`, con margen de 14 píxeles y zona exclusiva cero. La lista de favoritos viene de `NoteSettings::favorites`.

8.5.1 Lógica de cada icono

Al hacer clic:

1. se enumeran ventanas abiertas;
2. se compara `app_id` con el identificador de la aplicación;
3. si existe, se enfoca con `wlrctl`;
4. si no existe, se lanza el archivo `desktop`.

El indicador de ejecución se actualiza cada dos segundos. La ampliación al pasar el puntero sólo cambia el tamaño del icono ocho píxeles; no existe magnificación vecinal como en macOS.

8.6 Overview

Overview es una superficie Overlay anclada a los cuatro bordes y con teclado exclusivo. Contiene:

- campo de búsqueda;
- lista de ventanas abiertas;
- cuadrícula de hasta 80 aplicaciones;
- botones para escritorios 1-N.

8.6.1 Búsqueda

Se normaliza a minúsculas y se compara contra nombre, nombre genérico y categorías. No hay indexación, ranking, fuzzy search ni proveedores externos.

8.6.2 Ventanas

Se usa `wlrctl` para obtener `app_id:title`. Las tarjetas no tienen miniaturas; muestran un icono genérico, título e ID. Al seleccionar una tarjeta se enfoca la ventana y se oculta el overview.

8.6.3 Escritorios

El botón llama a `note-workspace`, que simula `Super+n` mediante `wlrctl`. Es una solución temporal. El shell no conoce el escritorio activo ni la relación ventana-escritorio.

8.7 Centro de control

Es una superficie Overlay anclada arriba a la derecha. Incluye:

- usuario y acceso a Configuración;
- interruptores de Wi-Fi y Bluetooth;
- accesos a red y pantallas;
- sliders de volumen y brillo;
- botones de bloqueo, cierre de sesión, reinicio y apagado.

Los interruptores llaman funciones de **note-core**. No existe todavía manejo visual de errores ni estados intermedios.

8.8 CSS y preferencias GTK

`install_css()` concatena variables generadas con el CSS base. `apply_gtk_preferences()` configura modo oscuro y animaciones. El modo automático usa la hora local, no amanecer/anocheecer ni geolocalización.

8.9 Puntos de extensión recomendados

Antes de agregar más código a `main.rs`, divide el crate así:

Listing 8.2: Estructura sugerida para note-shell

```
1 src/|—
2   main.rs|—
3   app.rs|—
4   state.rs|—
5   actions.rs|—
6   surfaces/|
7     |— mod.rs|
8     |— panel.rs|
9     |— dock.rs|
10    |— overview.rs|
11    |— control_center.rs|—
12  widgets/|
13    |— app_tile.rs|
14    |— quick_tile.rs|
15    |— slider_row.rs|—
16  services/
17    |— window_tracker.rs
18    |— battery.rs
19    |— launcher.rs
```

Primera refactorización útil

Extrae el rastreo de ventanas a un objeto con caché y temporizador único. Panel, dock y overview pueden suscribirse al mismo estado, en vez de ejecutar `wlrcctl` de forma independiente.

Crate note-settings

note-settings es una aplicación GTK4 normal. Su función actual es editar preferencias propias y servir como centro de acceso a herramientas externas.

9.1 Estructura visual

La ventana tiene:

- HeaderBar;
- barra lateral con páginas;
- GtkStack con transición crossfade;
- ActionBar con restaurar, estado y aplicar.

9.2 Páginas

Página	Estado actual
Apariencia	Nativa: modo, acento, idioma, tamaño del dock, opacidades, animaciones, scroll y escritorios.
Idioma y región	Informativa; muestra locale y archivo generado.
Pantallas	Abre wdisplays o arandr.
Red	Abre nm-connection-editor.
Bluetooth	Abre blueman-manager.
Sonido	Abre pavucontrol.
Energía	Página informativa.
Teclado	Página informativa.
Privacidad	Página informativa.
Acerca de	Versión, icono y acceso a note-doctor.

9.3 AppearanceControls

La estructura conserva referencias a todos los widgets editables. Como los objetos GTK son referencias contadas, clonar la estructura es barato y permite capturarla

en varios closures.

Listing 9.1: Controles editables de Apariencia

```

1 #[derive(Clone)]
2 struct AppearanceControls {
3     page: gtk::Widget,
4     appearance: gtk::DropDown,
5     accent: gtk::DropDown,
6     language: gtk::DropDown,
7     dock_size: gtk::Scale,
8     panel_opacity: gtk::Scale,
9     dock_opacity: gtk::Scale,
10    animations: gtk::Switch,
11    natural_scroll: gtk::Switch,
12    workspaces: gtk::SpinButton,
13 }

```

9.4 Flujo de guardado

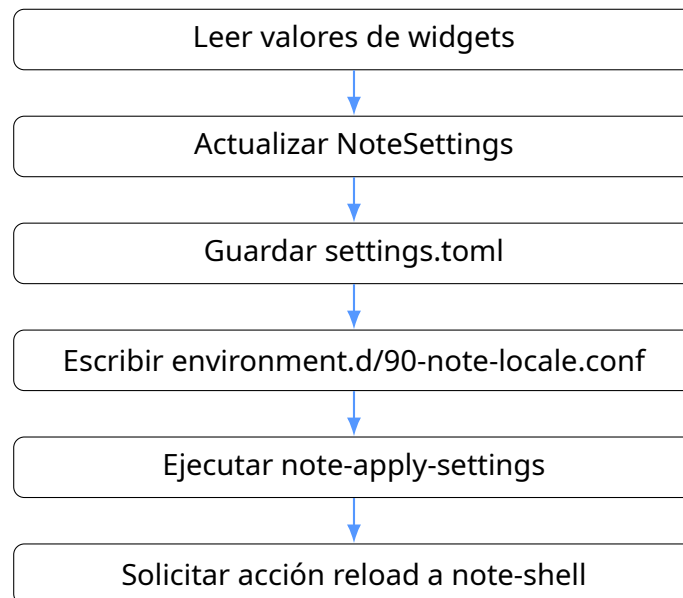


Figura 9.1: Aplicación de preferencias.

note-apply-settings genera un fragmento XML de usuario y llama `labwc --reconfigure`. Después la acción `GApplication` reconstruye panel y dock.

9.5 Error E0382 corregido

La fuente original de 0.2.3 entregaba `controls.page` al registrar la página. Después intentaba `controls.clone()`, pero el campo ya se había movido. La solución correcta es clonar la referencia GTK:

```

1 add_page(..., controls.page.clone());

```

La captura de la [Figura 9.2](#) muestra el diagnóstico del compilador.

```

Compiling gio-sys v0.21.5
Compiling cairo-sys-rs v0.21.5
Compiling gdk-pixbuf-sys v0.21.5
Compiling pango-sys v0.21.5
Compiling glib-macros v0.21.5
Compiling glib v0.21.5
Compiling graphene-sys v0.21.5
Compiling gdk4-sys v0.10.3
Compiling serde_core v1.0.228
Compiling gsk4-sys v0.10.3
Compiling serde v1.0.228
Compiling gio v0.21.5
Compiling serde_derive v1.0.228
Compiling gtk4-sys v0.10.3
Compiling gdk-pixbuf v0.21.5
Compiling pango v0.21.5
Compiling cairo-rs v0.21.5
Compiling hashbrown v0.17.1
Compiling equivalent v1.0.2
Compiling indexmap v2.14.0
Compiling gdk4 v0.10.3
Compiling serde_spanned v0.6.9
Compiling toml_datetime v0.6.11
Compiling graphene-rs v0.21.5
Compiling winnow v0.7.15
Compiling toml_write v0.1.2
Compiling toml_edit v0.22.27
Compiling gsk4 v0.10.3
Compiling gtk4-macros v0.10.3
Compiling gtk4-layer-shell-sys v0.5.2
Compiling gtk4 v0.10.3
Compiling toml v0.8.23
Compiling note-core v0.2.2 (/home/user/desknote/crates/note-core)
Compiling gtk4-layer-shell v0.7.1
Compiling note-shell v0.2.2 (/home/user/desknote/crates/note-shell)
Compiling note-settings v0.2.2 (/home/user/desknote/crates/note-settings)
error[E0382]: borrow of partially moved value: `controls`
  -> crates/note-settings/src/main.rs:86:24
   |
49 |         add_page(&sidebar, &stack, "appearance", "preferences-desktop-theme-symbolic", &tr.text("appearance"), controls.page);
   |                                                                                                     ----- value partially moved here
...
86 |         let controls = controls.clone();
   |         ^^^^^^^^^ value borrowed here after partial move

= note: partial move occurs because `controls.page` has type `gtk4::Widget`, which does not implement the `Copy` trait

For more information about this error, try `rustc --explain E0382`.
error: could not compile `note-settings` (bin "note-settings") due to 1 previous error
user@debian-vm:~/desknote$

```

Figura 9.2: Error E0382 encontrado durante la compilación de note-settings.

9.6 Limitaciones del diseño actual

- las preferencias no se validan mediante un esquema versionado;
- no existe botón de deshacer ni detección de cambios sin guardar;
- restaurar defaults sólo actualiza widgets; no guarda hasta pulsar Aplicar;
- varias páginas son lanzadores de herramientas ajenas;
- no hay separación entre configuración de shell, compositor, hardware y cuenta;
- cambiar idioma requiere recargar aplicaciones y a veces cerrar sesión.

9.7 Ruta de evolución

Conviene separar la UI de un servicio **note-settings-daemon**. La aplicación escribiría a través de D-Bus; el daemon validaría, migraría versiones, aplicaría cambios y notificaría a compositor/shell. Así también se podrían modificar preferencias desde CLI o pruebas automatizadas.

Configuración, CSS, tema y recursos visuales

10.1 Jerarquía de configuración

Note Desktop usa configuración global y de usuario. La global proporciona valores predeterminados; la del usuario debe prevalecer.

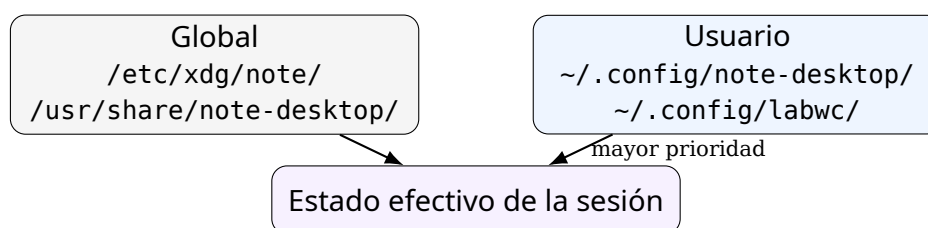


Figura 10.1: Fuentes de configuración y precedencia conceptual.

10.2 settings.toml

Un archivo típico:

Listing 10.1: Ejemplo de configuración de usuario

```
1 language = "es-MX"
2 appearance = "dark"
3 accent = "blue"
4 dock_size = 50
5 panel_opacity = 0.86
6 dock_opacity = 0.78
7 animations = true
8 natural_scroll = true
9 workspaces = 4
10 favorites = [
11     "foot.desktop",
12     "thunar.desktop",
13     "org.gnome.Epiphany.desktop",
14     "org.gnome.TextEditor.desktop"
15 ]
```

10.3 CSS del shell

El CSS se instala en `/usr/share/note-desktop/style/shell.css`. Las variables dinámicas se anteponen al cargar:

Listing 10.2: Variables generadas por NoteSettings

```
1 @define-color note_accent rgb(84, 151, 255);
2 @define-color note_panel_bg rgba(20, 22, 28, 0.860);
3 @define-color note_dock_bg rgba(26, 28, 35, 0.780);
```

Las clases principales son:

Clase CSS	Widget/función
<code>.panel</code>	Barra superior.
<code>.note-logo</code>	Botón de overview.
<code>.clock-button</code>	Reloj y calendario.
<code>.status-cluster</code>	Entrada al centro de control.
<code>.dock</code>	Contenedor flotante inferior.
<code>.dock-item</code>	Botones de aplicaciones y acciones.
<code>.running-indicator</code>	Punto bajo aplicaciones abiertas.
<code>.overview</code>	Fondo y contenedor del overview.
<code>.window-card</code>	Ventana listada.
<code>.app-tile</code>	Aplicación en la cuadrícula.
<code>.workspace-pill</code>	Selector de escritorio.
<code>.control-center</code>	Panel rápido de estado.
<code>.quick-tile</code>	Interruptor o acceso rápido.

10.4 Transparencia frente a blur

GTK puede dibujar fondos con alfa, pero el resultado depende de lo que el compositor haga detrás. El CSS actual usa `rgba(...)`; no captura ni desenfoca el fondo. Un blur verdadero requiere que el compositor procese la región detrás de la superficie.

10.5 Tema de ventanas

`configs/labwc/rc.xml` selecciona el tema Note. El directorio `/usr/share/themes/Note/labwc/` contiene la decoración. La disposición:

```
1 <titlebar><layout>close,max,iconify:</layout></titlebar>
```

coloca los controles al lado izquierdo. Esto se parece al orden visual de macOS, pero son decoraciones de labwc y no botones dibujados por cada aplicación.

10.6 Fondos

`note-wave.svg` es el fondo de sesión y `note-login.svg` el fondo conceptual del greeter. SVG permite escalar sin pérdida. swaybg se encarga de mostrarlo con modo fill.



Figura 10.2: Fondo principal note-wave incluido en el proyecto.

10.7 Agregar un color de acento

Se requieren tres cambios:

1. agregar el nombre y RGB en `accent_rgb()` de `config.rs`;
2. agregar el nombre al dropdown de `note-settings`;
3. agregarlo a la lista usada por `AppearanceControls::apply()`.

Para evitar listas duplicadas, una refactorización mejor es definir una estructura pública:

```
1 pub struct Accent {  
2     pub id: &'static str,  
3     pub label_key: &'static str,  
4     pub rgb: (u8, u8, u8),  
5 }  
6  
7 pub const ACCENTS: &[Accent] = &[ /* ... */ ];
```

Así `shell` y `settings` consumen la misma fuente.

10.8 Modificar estilos durante desarrollo

La versión instalada lee CSS desde `/usr/share/note-desktop/style`. Para iterar sin reinstalar, puedes temporalmente cambiar `install_css()` para buscar primero una variable:

```
1 let path = std::env::var("NOTE_SHELL_CSS")  
2     .unwrap_or_else(|_| "/usr/share/note-desktop/style/shell.css".into());
```

Y ejecutar:


```
1 NOTE_SHELL_CSS="$PWD/assets/style/shell.css" cargo run -p note-shell
```

Internacionalización, idioma y región

11.1 Idiomas incluidos

- es-MX: español de México;
- en-US: inglés de Estados Unidos;
- pt-BR: portugués de Brasil;
- fr-FR: francés;
- de-DE: alemán.

Cada idioma contiene locales/<locale>/shell.ftl. El mismo catálogo se usa para shell y settings.

11.2 Formato actual del catálogo

Listing 11.1: Ejemplo simplificado de catálogo

```
1 overview = Vista general
2 settings = Configuración
3 search-placeholder = Buscar aplicaciones...
4 power-off = Apagar
```

El parser divide cada línea por el primer signo igual. No soporta de manera real:

- selectores Fluent;
- plurales;
- atributos;
- valores multilínea;
- términos;
- aislamiento bidi;
- formatos de fecha y número dependientes de locale.

La extensión .ftl es engañosa

El sistema actual usa una sintaxis compatible sólo con el subconjunto más simple de Fluent. No copies ejemplos avanzados de Fluent esperando que funcionen. Antes de ampliar traducciones, migra a una biblioteca Fluent real o cambia la

extensión para no confundir mantenedores.

11.3 Selección y fallback

`Translator::new()` acepta códigos completos y abreviados. Cualquier idioma no reconocido cae a español de México; cada clave faltante cae a inglés; si tampoco existe en inglés, se muestra la clave.

Una política más coherente sería:

1. locale solicitado;
2. idioma base del locale;
3. español de México como idioma del proyecto o inglés como lingua franca;
4. clave visible con marcador de error.

11.4 Locale de procesos

Al guardar el idioma se genera:

Listing 11.2: Archivo de entorno generado

```
1 LANG=es_MX.UTF-8
2 LANGUAGE=es-MX
3 LC_MESSAGES=es_MX.UTF-8
```

Las aplicaciones nuevas reciben estas variables después de importar el entorno o cerrar sesión. El sistema no configura por separado moneda, fecha, decimal, papel, teclado ni zona horaria.

11.5 Agregar un idioma

Supongamos it-IT:

1. copiar un catálogo base a `locales/it-IT/shell.ftl`;
2. traducir todas las claves;
3. añadir `const IT_IT` en `i18n.rs`;
4. añadir el `match` en `Translator::new()`;
5. añadir el idioma al dropdown y a `language_code()`;
6. añadirlo al script de `locale-gen`;
7. agregarlo a la documentación y pruebas de correspondencia de claves.

11.6 Prueba automática de catálogos

Una prueba útil debe garantizar que todos los idiomas contienen las claves de inglés:

Listing 11.3: Prueba propuesta de claves de traducción

```
1 #[test]
2 fn all_locales_contain_reference_keys() {
3     let reference = parse(EN_US);
```

```
4     for (name, source) in [("es-MX", ES_MX), ("pt-BR", PT_BR)] {  
5         let locale = parse(source);  
6         for key in reference.keys() {  
7             assert!(locale.contains_key(key), "{name} no contiene {key}");  
8         }  
9     }  
10 }
```

11.7 Migración recomendada a Fluent real

Crea un módulo que use `fluent-bundle` y `unic-langid`. El API público puede conservar `text()` y `format()`, minimizando cambios en la UI. La migración habilitará plurales y mensajes correctos para idiomas complejos.

Scripts y comandos auxiliares

Los scripts son la capa de pegamento del proyecto. En la alpha cumplen un papel importante, pero deben mantenerse pequeños, predecibles y fáciles de sustituir.

12.1 Categorías

12.1.1 Entry points públicos

Instalados en `/usr/bin`:

- `note-session`;
- `note-overview`;
- `note-control-center`;
- `note-workspace`;
- `note-terminal`, `note-files`, `note-browser`, `note-editor`;
- `note-lock`, `note-logout`, `note-reboot`, `note-poweroff`;
- `note-screenshot`;
- `note-apply-settings`;
- `note-doctor`;
- `note-test-from-tty`;
- `note-nvidia-kms`.

12.1.2 Scripts internos

Instalados en `/usr/libexec/note-desktop`:

- `note-session-inner`;
- `note-greeter-session`;
- `note-autostart`.

El usuario no debería depender de estos paths internos; pueden cambiar entre versiones.

12.2 Wrappers de aplicaciones

note-terminal, note-files, note-browser y note-editor recorren una lista de candidatos. Esto permite que Note funcione en Debian y Ubuntu sin imponer una sola aplicación.

Listing 12.1: Patrón de selección de navegador

```
1 for command in epiphany firefox-esr firefox chromium chromium-browser; do
2     command -v "$command" >/dev/null 2>&1 && exec "$command" "$@"
3 done
```

A largo plazo, el orden debe provenir de preferencias y asociaciones MIME, no estar codificado en el script.

12.3 Capturas

note-screenshot crea ~/Pictures/Screenshots, obtiene una geometría con slurp y captura con grim. Si slurp no existe, captura toda la salida. Luego envía una notificación si notify-send está disponible.

12.4 Bloqueo

note-lock intenta en orden:

1. gtlck;
2. swaylock;
3. loginctl lock-session.

El tercer camino no garantiza una pantalla de bloqueo visible si no existe un lock manager. Para un escritorio propio se debe implementar ext-session-lock-v1 y un proceso de lock auditado.

12.5 Aplicación de preferencias

note-apply-settings usa Python y tomlib para leer workspaces y natural scroll, genera un XML de usuario y reconfigura labwc.

Compatibilidad de Python

tomlib está disponible desde Python 3.11. Debian 13 lo incluye, pero una distribución más antigua necesitaría tomli o un parser alternativo. El script deja datos vacíos si no existe tomlib, lo que silently aplica defaults.

12.6 Diagnóstico

note-doctor verifica comandos, sesión, GPU, Vulkan, grupos y greetd. En máquinas con vulkaninfo --summary incompatible puede imprimir más salida de la esperada. El doctor debería evolucionar a un formato estable con códigos de salida por categoría y opción JSON.

12.7 Instalación y empaquetado

install.sh instala dependencias, compila, copia archivos y habilita servicios.

install-files.sh copia artefactos a un root o DESTDIR; es la pieza reutilizada por paquetes.

build-deb.sh crea un paquete Debian binario local.

make-release.sh genera archivo de release.

uninstall.sh elimina archivos Note, restaura greetd y conserva dependencias compartidas.

preflight.sh realiza comprobaciones rápidas antes de instalar.

12.8 Principios para crear un script nuevo

1. usa `#!/bin/sh` si no necesitas Bash;
2. activa `set -eu`;
3. usa `exec` para reemplazar el wrapper por la aplicación final;
4. escribe errores a `stderr`;
5. devuelve códigos distintos para uso incorrecto y fallo operativo;
6. no tragues errores importantes con `|| true`;
7. documenta dependencias y fallback;
8. agrega validación shell al CI.

systemd -user e integración con servicios del sistema

13.1 Por qué usar servicios de usuario

Un escritorio moderno contiene procesos con ciclos de vida diferentes. systemd -user permite expresar dependencias, reinicios y pertenencia a la sesión sin mantener un script monolítico con procesos en background.

13.2 note-session.target

Listing 13.1: Target de sesión Note

```
1 [Unit]
2 Description=Note Desktop user session
3 Wants=note-shell.service note-notifications.service note-polkit-agent.service
4 After=graphical-session-pre.target
5 BindsTo=graphical-session.target
6 PartOf=graphical-session.target
7
8 [Install]
9 WantedBy=graphical-session.target
```

El target quiere tres servicios: shell, notificaciones y agente PolicyKit. Se enlaza a graphical-session.target, por lo que debe detenerse cuando termina la sesión gráfica.

13.3 note-shell.service

Listing 13.2: Servicio del shell

```
1 [Unit]
2 Description=Note Desktop shell
3 PartOf=note-session.target
4 After=graphical-session.target
5
6 [Service]
7 Type=simple
```



```
8 ExecStart=/usr/bin/note-shell
9 Restart=on-failure
10 RestartSec=1
11 Slice=session.slice
12
13 [Install]
14 WantedBy=note-session.target
```

`Restart=on-failure` permite recuperar panel y dock. `RestartSec=1` evita un reinicio inmediato demasiado agresivo, aunque todavía puede entrar en bucle si hay un fallo determinista.

13.4 Notificaciones y PolicyKit

`note-notifications.service` ejecuta `mako` en `background.slice`. `note-polkit-agent.service` ejecuta `lxpolkit`. El uso de slices permite separar recursos de procesos de sesión y servicios en `background`.

13.5 Importación del entorno

Los servicios de usuario se arrancan fuera del proceso directo del compositor. Necesitan conocer:

- `WAYLAND_DISPLAY`;
- `DISPLAY`;
- `XDG_CURRENT_DESKTOP`;
- `XDG_SESSION_TYPE`;
- `LANG` y variables relacionadas.

Por eso `note-autostart` y `note-session-inner` llaman `systemctl -user import-environment` y `dbus-update-activation-environment`.

13.6 PipeWire y WirePlumber

El shell usa `wpctl`, cliente de WirePlumber, para consultar y cambiar volumen. PipeWire y WirePlumber normalmente ya son servicios de usuario de la distribución; Note no duplica esas unidades.

13.7 NetworkManager

El estado rápido se obtiene con `nmcli radio wifi`; el centro de control abre `nm-connection-editor`. La sesión habilita `NetworkManager.service` a nivel sistema durante instalación.

13.8 BlueZ

El shell llama `bluetoothctl show` y `bluetoothctl power`. La interfaz completa se delega a BlueMan. BlueZ corre como servicio del sistema.

13.9 PolicyKit

Varias operaciones administrativas requieren un agente de autenticación. Debian 13 usa **lxpolkit** en este proyecto porque `policykit-1-gnome` no tiene candidato instalable. El agente debe ejecutarse una sola vez por sesión.

13.10 UPower y batería

Aunque `upower` se instala, el panel actual lee directamente `/sys/class/power_supply/BAT*/capacity`. Esto funciona para el porcentaje básico, pero no proporciona estado de carga, tiempo estimado, múltiples baterías ni eventos. La migración correcta es UPower D-Bus.

13.11 Portales XDG

Listing 13.3: Selección de backends de portal

```
1 [preferred]
2 default=gtk
3 org.freedesktop.impl.portal.Screenshot=wlr
4 org.freedesktop.impl.portal.ScreenCast=wlr
5 org.freedesktop.impl.portal.RemoteDesktop=wlr
```

El backend GTK sirve de predeterminado; `wlr` maneja captura, screencast y escritorio remoto. El valor `XDG_CURRENT_DESKTOP=Note:labwc:wlroots` ayuda a que `xdg-desktop-portal` seleccione el archivo apropiado.

Capturas y screencast

Que `grim` funcione no significa que Firefox, OBS o una aplicación Flatpak puedan compartir pantalla. Para eso deben arrancar correctamente PipeWire, `xdg-desktop-portal` y `xdg-desktop-portal-wlr` con el entorno de la sesión importado.

13.12 Comandos de inspección

```
1 systemctl --user status note-session.target
2 systemctl --user status note-shell.service
3 systemctl --user list-dependencies note-session.target
4 journalctl --user -u note-shell.service -b
5 systemctl --user show-environment | sort
6 busctl --user list
```

GPU, DRM, NVIDIA y máquinas virtuales

14.1 Ruta gráfica actual

Labwc y wlroots seleccionan dispositivos DRM, crean buffers mediante GBM/EGL y presentan en KMS. Note no implementa todavía esta ruta; únicamente prepara variables y fallbacks.

14.2 AMD e Intel

En sistemas Mesa, la ruta esperada es:

```
1 Aplicación -> Wayland/DMA-BUF -> labwc/wlroots -> Mesa -> DRM/KMS -> monitor
```

El instalador agrega Mesa DRI, drivers Vulkan y herramientas de diagnóstico. El firmware de GPU depende de la distribución y repositorios habilitados.

14.3 NVIDIA

El instalador no instala el controlador propietario. `note-nvidia-kms` escribe:

Listing 14.1: Configuración generada para NVIDIA DRM KMS

```
1 options nvidia_drm modeset=1 fbdev=1
```

Después ejecuta `update-initramfs -u`. Antes de usarlo, el controlador debe estar instalado correctamente. El diagnóstico principal es:

```
1 cat /sys/module/nvidia_drm/parameters/modeset
2 ls -l /dev/dri
3 nvidia-smi
4 note-doctor
```

14.4 Configuración gpu.conf

El sistema carga primero `/etc/note-desktop/gpu.conf` y después `~/.config/note-desktop/gpu.conf`. Puede contener:

```
1 export WLR_RENDERER=pixman
2 export WLR_NO_HARDWARE_CURSORS=1
3 # export WLR_DRM_DEVICES=/dev/dri/card1:/dev/dri/card0
```

No configures `WLR_DRM_DEVICES` por número de card sin revisar cada arranque; el orden puede cambiar. En el compositor futuro se deben usar render nodes y propiedades de udev.

14.5 Fallback a Pixman

Tanto greeter como sesión reintentan con Pixman si el compositor muere durante los primeros segundos. Pixman renderiza por CPU. Es útil para recuperación y VMs, pero lento para animaciones, video y resoluciones altas.

14.6 VirtualBox

La configuración recomendada es:

- controlador VMSVGA;
- 128 MB de memoria de video;
- aceleración 3D activada;
- 4 GiB de RAM;
- dos o más vCPU;
- Guest Additions compatibles con la versión del host.

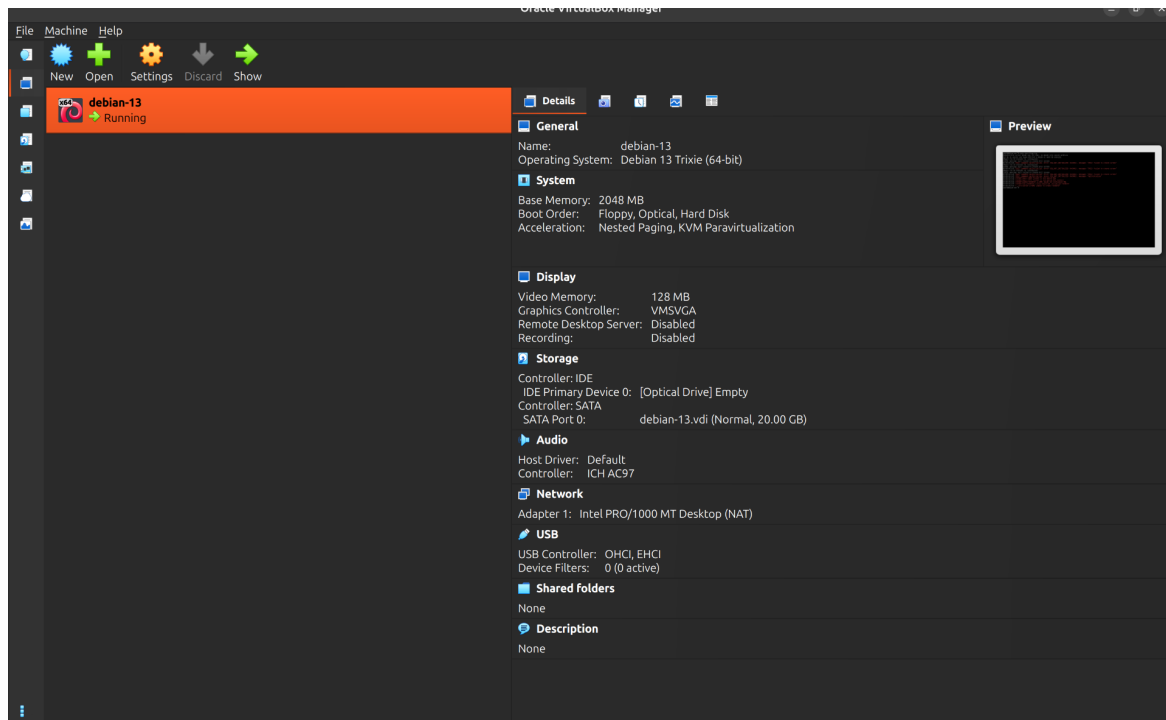


Figura 14.1: VirtualBox con VMSVGA y 128 MB; la opción 3D se habilita con la VM apagada.

14.7 Diagnóstico observado

En la VM de prueba, **note-doctor** identificó el adaptador VMware SVGA II y el módulo `vmwgfx`. La captura siguiente muestra un diagnóstico desde TTY; por eso el aviso de no estar dentro de Wayland es esperado.

```

Kernel modules: vmwgfx

Vulkan:
=====
VULKANINFO
=====

Vulkan Instance Version: 1.4.309

Instance Extensions: count = 24
-----
VK_EXT_acquire_drm_display : extension revision 1
VK_EXT_acquire_xlib_display : extension revision 10
VK_EXT_debug_report : extension revision 2
VK_EXT_debug_utils : extension revision 1
VK_EXT_direct_mode_display : extension revision 1
VK_EXT_display_surface_counter : extension revision 1
VK_EXT_headless_surface : extension revision 1
VK_EXT_surface_maintenance1 : extension revision 5
VK_EXT_swapchain_colorspace : extension revision 1
VK_KHR_device_group_creation : extension revision 23
VK_KHR_display : extension revision 1
VK_KHR_external_fence_capabilities : extension revision 1
VK_KHR_external_memory_capabilities : extension revision 1
VK_KHR_external_semaphore_capabilities : extension revision 1
VK_KHR_get_display_properties2 : extension revision 1
VK_KHR_get_physical_device_properties2 : extension revision 2
VK_KHR_get_surface_capabilities2 : extension revision 1
VK_KHR_portability_enumeration : extension revision 1
VK_KHR_surface : extension revision 25
VK_KHR_surface_protected_capabilities : extension revision 1
VK_KHR_wayland_surface : extension revision 6
VK_KHR_xcb_surface : extension revision 6
VK_KHR_xlib_surface : extension revision 6
VK_LUNARG_direct_driver_loading : extension revision 1

Instance Layers: count = 3
[OK] Usuario dentro de video
[OK] Usuario dentro de render
[WARN] No estás dentro de una sesión Wayland

Para recuperar una máquina que no muestre el login:
Ctrl+Alt+F2, inicia sesión y ejecuta:
sudo systemctl disable --now greed
user@debian-vm:~/desktop$ note-test-from-tty
Ejecuta este script desde una TTY real, no desde otra sesión gráfica.
Sal de la sesión con Super+Shift+E o desde el menú de energía.
user@debian-vm:~/desktop$ lost connection to Wayland compositor. ffer handle: Argumento inválido (-5648ms), your system is too slow

```

Figura 14.2: Salida real de note-doctor durante el desarrollo.

14.8 Errores EGL comunes

```
1 libEGL warning: failed to create dri2 screen
2 EGL_NOT_INITIALIZED
3 VMware: No 3D enabled
```

Secuencia de resolución:

1. apagar completamente la VM;
2. activar VMSVGA, 128 MB y 3D;
3. actualizar Guest Additions;
4. verificar `glxinfo -B` y `vulkaninfo --summary`;
5. probar con `WLR_RENDERER=pixman`;
6. revisar journal y coredumps.

14.9 Diseño futuro multi-GPU

note-compositor deberá distinguir:

- GPU de scanout por salida;
- GPU de render preferida;
- importación/exportación DMA-BUF;
- modifiers compatibles;
- sincronización explícita;
- direct scanout;
- hotplug y cambio de salida.

La política debe basarse en capacidades, no en condiciones `if vendor == NVIDIA` dispersas por el código.

Compilación e instalación

15.1 Tres formas de trabajar

Compilación local genera binarios en `target/` sin tocar el sistema.

Instalación de desarrollo usa `install.sh` y copia a `/usr`.

Paquete local usa `build-deb.sh` y permite instalar/desinstalar con `dpkg/APT`.

Para iterar código, usa la primera. Para probar login y sesión, usa la segunda en VM. Para validar mantenimiento, usa la tercera.

15.2 Preflight

`preflight.sh` comprueba distribución, APT, `greeterd`, `systemd` y `/dev/dri`. En 0.2.3 espera que `greeterd` ya esté instalado, por lo que es más una verificación posdependencias que un preflight completamente independiente.

15.3 Instalador directo

`install.sh` realiza estas fases:

1. detecta Debian o Ubuntu;
2. obtiene privilegios;
3. configura reintentos y evita pipelining problemático de APT;
4. actualiza índices;
5. habilita universe en Ubuntu;
6. filtra paquetes con candidato instalable usando `LC_ALL=C apt-cache policy`;
7. instala paquetes core y después opcionales;
8. genera locales;
9. valida comandos críticos y `pkg-config`;
10. compila el workspace;
11. ejecuta `install-files.sh`;
12. respalda y sustituye la configuración de `greeterd`;
13. agrega usuarios a grupos de dispositivos;

14. habilita servicios y graphical.target.

15.4 Por qué se fuerza LC_ALL=C

APT localiza sus etiquetas. En español, apt-cache policy imprime Candidato::; el script original buscaba Candidate: y concluyó que ningún paquete existía. La función corregida fuerza locale C sólo para el comando de consulta.

```
1 candidate() {  
2     LC_ALL=C apt-cache policy "$1" 2>/dev/null |  
3     awk '/^[[:space:]]*Candidate:/ {print $2; exit}'  
4 }
```

15.5 Paquetes core y opcionales

Los paquetes core incluyen compositor, XWayland, login, layer-shell, Rust, GTK, audio, red, Bluetooth, portales, herramientas gráficas y utilidades. Los opcionales incluyen lock screens, configuración de pantallas y aplicaciones de usuario.

El instalador debe abortar si falta una dependencia verdaderamente crítica. En cambio, una aplicación opcional ausente no debería impedir instalar el escritorio.

15.6 install-files.sh y DESTDIR

install-files.sh no instala dependencias ni habilita servicios. Sólo copia archivos. Acepta un destino para empaquetado:

```
1 DESTDIR=/tmp/pkg-root ./scripts/install-files.sh /tmp/pkg-root
```

Esta separación es correcta: el mismo procedimiento de instalación de archivos se reutiliza en .deb, RPM y PKGBUILD.

15.7 Instalación manual controlada

Para evitar que el script cambie greetd durante desarrollo:

```
1 cargo build --workspace --release  
2 sudo ./scripts/install-files.sh  
3 sudo systemctl daemon-reload
```

Después puedes arrancar la sesión desde TTY sin habilitar greetd:

```
1 note-test-from-tty
```

15.8 Instalación limpia en VM

```
1 unzip note-desktop-0.2.3.zip  
2 cd note-desktop-0.2.3  
3 sed -i 's/controls\.page)/controls.page.clone()/;' \  
4 crates/note-settings/src/main.rs
```



```
5 chmod +x scripts/*  
6 ./scripts/install.sh  
7 sudo reboot
```

15.9 Desinstalación

`uninstall.sh`:

- deshabilita `greetd`;
- cambia a `multi-user.target`;
- restaura el respaldo más reciente;
- elimina archivos `Note`;
- recarga `systemd`, escritorio e iconos;
- conserva paquetes compartidos.

No ejecutes `uninstall.sh` desde dentro de la única sesión gráfica sin una TTY

El script puede detener `greetd` y eliminar binarios en uso. Mantén otra TTY abierta. Para sistemas reales, la desinstalación debe delegarse al gestor de paquetes.

15.10 Build reproducible

Para mejorar reproducibilidad:

1. conserva `Cargo.lock` en releases;
2. usa `cargo build --locked`;
3. compila en contenedores limpios por distribución;
4. fija versiones mínimas de bibliotecas del sistema;
5. genera SBOM y checksums;
6. prueba instalación, actualización, downgrade y purga.

Empaquetado y repositorios

16.1 Por qué el instalador Bash no es el destino

Un gestor de paquetes resuelve dependencias, verifica firmas, registra archivos, aplica actualizaciones y ejecuta scripts de mantenimiento. Un instalador Bash propio tendría que reinventar todo eso y fallaría en casos de actualización o rollback.

16.2 Paquete Debian local

`build-deb.sh` compila, crea un staging temporal, ejecuta `install-files.sh`, genera control y scripts de mantenimiento y llama `dpkg-deb`.

```
1 ./scripts/build-deb.sh
2 sudo apt install ./dist/note-desktop_0.2.3_amd64.deb
```

16.3 Metadatos actuales

El paquete local declara dependencias de runtime y recomienda utilidades opcionales. También proporciona `x-window-manager`, aunque Note es un compositor Wayland; esta declaración debe revisarse para no confundir herramientas que esperan un WM X11 tradicional.

16.4 Scripts de mantenimiento Debian

postinst respalda `greetd`, activa locales y servicios.

prerm detiene/deshabilita `greetd` al quitar el paquete.

postrm restaura configuración previa.

Estos scripts realizan cambios globales agresivos. Un paquete para repositorio público debería preguntar menos, respetar display managers existentes y no cambiar el target predeterminado sin política clara.

16.5 Debian packaging formal

El directorio `packaging/debian/` contiene `control`, `rules` y `changelog`. Falta completar:

- `copyright`;
- `source format`;
- `watch`;
- pruebas `autopkgtest`;
- `manpages`;
- integración `debconf` si fuera necesaria;
- `lintian clean`;
- separación en paquetes binarios.

16.6 RPM

`note-desktop.spec` es un esqueleto. Requiere revisar nombres reales por Fedora, macros `Rust`, `ownership` de directorios, `scriptlets` de `systemd` y políticas `SELinux`.

16.7 Arch

El `PKGBUILD` compila desde un `tarball` y usa `sha256sums=('SKIP')`. Para AUR debe usar un `checksum` real. También debe decidir si depender de un paquete AUR para `gtk-greet`/`wlrcctl` cuando no estén en repositorios oficiales.

16.8 Paquetes planeados

Antes de 1.0 conviene dividir:

<code>note-desktop</code>	Metapaquete de experiencia completa.
<code>note-desktop-minimal</code>	Sesión mínima sin aplicaciones recomendadas.
<code>note-compositor</code>	Compositor <code>Smithay</code> .
<code>note-shell</code>	Panel, <code>dock</code> , <code>overview</code> y centro de control.
<code>note-settings</code>	Aplicación y <code>daemon</code> de preferencias.
<code>note-session</code>	Entry points, servicios y archivos <code>XDG</code> .
<code>note-greeter</code>	<code>Greeter</code> y tema.
<code>note-portal</code>	Backend de portal.
<code>note-themes</code>	CSS, iconos y decoración.
<code>note-wallpapers</code>	Fondos.
<code>note-l10n-*</code>	Catálogos separados o paquete único pequeño.

16.9 Repositorio APT propio

El flujo público esperado:

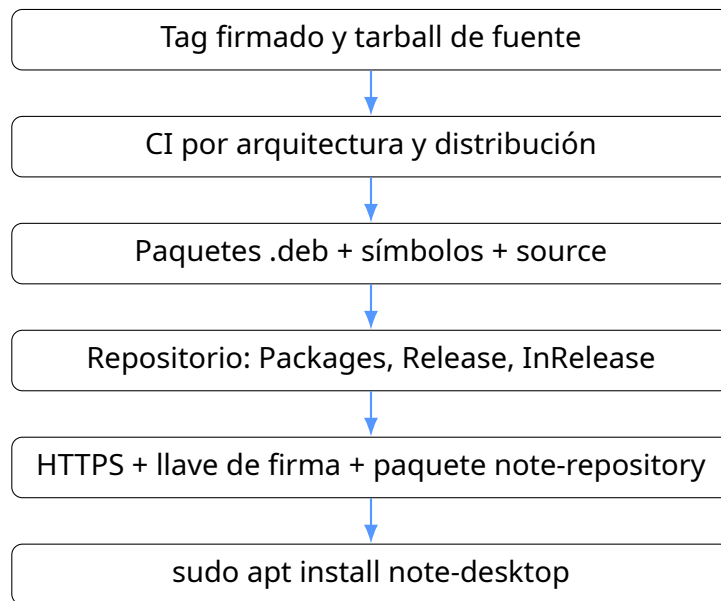


Figura 16.1: Camino para llegar a una instalación APT de una línea.

16.10 Compatibilidad multiplataforma

El código Rust puede compartirse, pero los nombres de paquetes, servicios y rutas varían. La instalación final debe tener adaptadores de empaquetado, no condiciones de distribución dentro del binario.

Pruebas, diagnóstico y depuración

17.1 Pirámide de pruebas

1. pruebas unitarias de parsers, configuración y selección de aplicaciones;
2. pruebas de integración de scripts con directorios temporales;
3. pruebas de UI GTK en compositor anidado;
4. pruebas de sesión completa en VM;
5. pruebas de hardware real y múltiples GPU.

17.2 CI incluida

El workflow usa un contenedor Debian 13 y ejecuta formato, check, tests y validación de shell. No inicia una sesión Wayland. Por tanto, puede detectar errores de tipos como los encontrados, pero no problemas de layer-shell, EGL o greetd.

17.3 Comandos básicos

```
1 cargo fmt --all -- --check
2 cargo check --workspace --all-targets
3 cargo clippy --workspace --all-targets -- -D warnings
4 cargo test --workspace
5 find scripts configs/labwc -type f -perm /111 -print0 |
6 xargs -0 -n1 sh -n
```

17.4 note-doctor

El doctor comprueba:

- comandos críticos;
- ubicación especial de greetd en/sbin;
- variables XDG y displays;
- GPU y módulo del kernel;
- resumen Vulkan;
- pertenencia a grupos video/render/input;

- estado de greetd;
- si la sesión actual es Wayland.

Un aviso de “No estás dentro de Wayland” es normal cuando el doctor se ejecuta desde TTY.

17.5 Logs

Listing 17.1: Comandos de diagnóstico de sesión

```
1 sudo journalctl -u greetd -b --no-pager
2 journalctl --user -u note-shell.service -b --no-pager
3 journalctl --user -b --no-pager | tail -n 200
4 sudo journalctl -b | grep -Ei 'drm|egl|gbm|wayland|labwc|vmwgfx|nvidia'
5 coredumpctl list
6 coredumpctl info labwc
```

17.6 Prueba desde TTY

note-test-from-tty rechaza ejecutarse si encuentra DISPLAY o WAYLAND_DISPLAY. Después crea un bus D-Bus y arranca note-session. Esto evita anidar accidentalmente la sesión real dentro de otra.

Para capturar salida:

```
1 note-test-from-tty > "$HOME/note-session.log" 2>&1
2 tail -n 200 "$HOME/note-session.log"
```

17.7 Pruebas unitarias prioritarias

- NoteSettings: defaults, roundtrip TOML, valores faltantes y archivo inválido;
- i18n: claves, fallback y reemplazos;
- desktop: localización, NoDisplay, códigos de campo, quoting y precedencia;
- system: parsers separados de ejecución de procesos;
- generación de XML de note-apply-settings;
- validación de que los catálogos tienen las mismas claves.

17.8 Pruebas de instalación

En CI o VM nueva:

1. instalar Debian mínimo;
2. ejecutar instalador;
3. confirmar archivos con dpkg -S o lista esperada;
4. arrancar greetd;
5. autenticar;
6. abrir app Wayland y app X11;
7. probar audio, red, screenshot y lock;
8. cerrar sesión;

9. desinstalar y verificar restauración.

17.9 Pruebas visuales

Conviene automatizar capturas de panel, dock, overview y settings a resoluciones conocidas. Las diferencias de imagen pueden detectar regresiones de CSS. Deben tolerar pequeñas variaciones tipográficas y antialiasing.

Diagnóstico estructurado

Una evolución útil de note-doctor es note-doctor --json. Cada comprobación debería producir ID, severidad, mensaje, evidencia y sugerencia. La UI de Configuración podría mostrarlo sin abrir una terminal.

Guía práctica para modificar y extender Note Desktop

Este capítulo es el más operativo del manual. Cada receta indica qué archivos tocar, por qué y cómo probar el cambio.

18.1 Agregar una nueva preferencia

Ejemplo: `dock_auto_hide: bool`.

18.1.1 1. Modelo persistente

En `crates/note-core/src/config.rs`:

```
1 pub struct NoteSettings {
2     // ...
3     pub dock_auto_hide: bool,
4 }
5
6 impl Default for NoteSettings {
7     fn default() -> Self {
8         Self {
9             // ...
10            dock_auto_hide: false,
11        }
12    }
13 }
```

Gracias a `#[serde(default)]`, archivos antiguos reciben el default.

18.1.2 2. Control en Configuración

Agrega un `gtk::Switch` a `AppearanceControls`, créalo en `build_appearance_page()`, actualízalo en `apply()` y léelo al guardar.

18.1.3 3. Uso en el shell

En `create_dock()`, consulta `settings.dock_auto_hide`. La alpha no tiene protocolo de revelado ni detección de ventanas; una primera implementación podría usar un tem-

porizador y `EventControllerMotion`, pero el `autohide` robusto debe coordinarse con el compositor.

18.1.4 4. Pruebas

```
1 cargo test -p note-core
2 cargo check -p note-settings
3 cargo check -p note-shell
```

Agrega un test de roundtrip TOML para evitar que la preferencia se pierda.

18.2 Agregar una página de Configuración

Ejemplo: “Aplicaciones predeterminadas”.

1. agrega claves al catálogo de todos los idiomas;
2. crea `build_default_apps_page()`;
3. llama `add_page()` en `build_ui()`;
4. evita ejecutar shell para leer asociaciones; usa `xdg-mime query default`;
5. para cambiar, usa `xdg-mime default ID tipo/mime`;
6. muestra éxito o error en la UI.

Estructura inicial:

```
1 fn build_default_apps_page(tr: &Translator) -> gtk::Widget {
2     let page = page_container(
3         &tr.text("default-apps"),
4         &tr.text("default-apps-description"),
5     );
6     // filas para navegador, correo, terminal, imágenes, video...
7     page
8 }
```

18.3 Agregar un idioma

La receta completa está en [Capítulo 11](#). Después ejecuta una herramienta que compare claves. Mientras el parser sea simple, evita valores multilínea y signos igual dentro del texto.

18.4 Cambiar favoritos del dock

El usuario puede editar `settings.toml`:

```
1 favorites = [
2     "org.gnome.Nautilus.desktop",
3     "firefox-esr.desktop",
4     "org.gnome.Terminal.desktop"
5 ]
```

Para una UI gráfica, crea una lista reordenable en Configuración que escriba desktop IDs. El shell ya recarga favoritos con la acción `reload`.

18.5 Agregar una acción al centro de control

Supongamos un botón de modo avión:

1. crea funciones `airplane_mode()` y `set_airplane_mode()` en `note-core/src/system.rs`;
2. construye un quick tile;
3. traduce título y descripción;
4. maneja fallo, porque apagar todas las radios puede requerir políticas;
5. prueba con y sin Bluetooth instalado.

No metas directamente cinco comandos shell en el callback del widget; la UI sólo debe llamar una abstracción.

18.6 Agregar un nuevo wrapper

Ejemplo: `note-images`.

```

1 #!/bin/sh
2 set -eu
3 for command in loupe eog gwenview ristretto; do
4     command -v "$command" >/dev/null 2>&1 && exec "$command" "$@"
5 done
6 echo "No se encontró visor de imágenes." >&2
7 exit 1

```

Después:

1. agrégalo a `bin_scripts` en `install-files.sh`;
2. agrégalo a `uninstall.sh`;
3. agrega pruebas shell;
4. documenta dependencias recomendadas.

18.7 Agregar un servicio `systemd -user`

Ejemplo: daemon de portapapeles.

Listing 18.1: Unidad de ejemplo

```

1 [Unit]
2 Description=Note Desktop clipboard history
3 PartOf=note-session.target
4 After=graphical-session.target
5
6 [Service]
7 Type=simple
8 ExecStart=/usr/bin/note-clipboard-daemon
9 Restart=on-failure
10 Slice=background.slice
11
12 [Install]
13 WantedBy=note-session.target

```

Agrega la unidad al array que instala archivos y a `Wants=` del target sólo si es esencial. Si es opcional, puede instalarse como paquete separado y habilitarse mediante `preset`.

18.8 Modificar el panel

`create_panel()` construye el widget completo. Para agregar un indicador:

1. crea un servicio de estado independiente;
2. crea el widget;
3. conecta señales o un temporizador;
4. agrega una clase CSS;
5. evita bloquear el hilo GTK;
6. considera multi-monitor.

Si la consulta puede tardar, usa `gio::spawn_blocking` o un hilo y devuelve resultados al `main` context. Nunca ejecutes una llamada lenta a D-Bus o disco directamente dentro de un callback frecuente.

18.9 Agregar una acción GApplication

```
1 let action = gio::SimpleAction::new("toggle-dock", None);
2 {
3     let state = state.clone();
4     action.connect_activate(move |_, _| {
5         // modificar visibilidad de superficies dock
6     });
7 }
8 app.add_action(&action);
```

Se invoca con:

```
1 gapplication action mx.note.desktop.shell toggle-dock
```

18.10 Crear un módulo nuevo en note-core

Ejemplo: `power.rs`.

1. define tipos de dominio independientes de D-Bus;
2. define un trait para consultar/aplicar estado;
3. implementa backend real;
4. exporta sólo la API necesaria desde `lib.rs`;
5. escribe tests con un backend falso.

```
1 pub trait PowerBackend {
2     fn profile(&self) -> Result<PowerProfile, PowerError>;
3     fn set_profile(&self, profile: PowerProfile) -> Result<(), PowerError>;
4 }
```

18.11 Trabajar sin reinstalar todo

Una rutina eficiente:

```
1 # Terminal 1
2 cargo watch -x 'check -p note-shell'
3
4 # Terminal 2, dentro de una sesión labwc de desarrollo
5 pkill note-shell || true
6 ./target/debug/note-shell
```

Si systemd reinicia el shell instalado, detén temporalmente el servicio:

```
1 systemctl --user stop note-shell.service
2 ./target/debug/note-shell
3 # Al terminar
4 systemctl --user start note-shell.service
```

18.12 Añadir logs

En vez de `eprintln!`, incorpora `tracing` y `tracing-subscriber`. Usa targets por módulo y niveles. Ejemplo:

```
1 tracing::info!(monitor = ?monitor.connector(), "creando panel");
2 tracing::warn!(command, error = %error, "no se pudo ejecutar comando");
```

Después los logs llegan a journal cuando el proceso es administrado por systemd.

Calidad, refactorización y seguridad

19.1 Deuda técnica principal

1. archivos `main.rs` demasiado grandes;
2. integración del sistema mediante procesos y parsing de texto;
3. errores ignorados o reducidos a `bool`;
4. polling frecuente;
5. parser parcial de `.desktop`;
6. internacionalización incompleta;
7. scripts de instalación con efectos globales;
8. ausencia de esquema y migraciones de preferencias;
9. pocas pruebas;
10. falta de logs estructurados.

19.2 Refactorización por etapas

19.2.1 Etapa 1: sin cambiar comportamiento

- dividir archivos en módulos;
- agregar tests de comportamiento actual;
- introducir tipos de error con `thiserror`;
- sustituir `eprintln` por `tracing`;
- centralizar listas de idiomas, acentos y aplicaciones fallback.

19.2.2 Etapa 2: backends abstractos

Crea traits para ventanas, audio, red, Bluetooth, brillo, batería y aplicaciones. La UI dependerá de traits o servicios, no de comandos.

19.2.3 Etapa 3: eventos

Sustituye polling por señales:

- `compositor` → foco y ventanas;
- `UPower` → batería;

- NetworkManager → conectividad;
- BlueZ → adaptadores/dispositivos;
- PipeWire/WirePlumber → volumen y streams.

19.3 Modelo de amenazas

Un escritorio opera con acceso a entrada, pantalla, archivos y servicios. Debe considerar:

- archivos desktop maliciosos;
- escalamiento mediante PolicyKit mal configurado;
- bloqueo de pantalla que puede omitirse;
- portales de captura demasiado permisivos;
- comandos contruidos con strings;
- scripts ejecutados como root;
- sobrescritura de configuración de greetd;
- symlinks y rutas controladas por usuario durante instalación;
- plugins o temas no confiables;
- sockets privados del futuro compositor.

19.4 Uso de shell

DesktopApp::launch() usa `sh -lc`. Reduce el uso de shell a los casos estrictamente necesarios. La especificación de archivos desktop define un campo Exec con reglas propias; un parser dedicado debe producir argv y eliminar códigos de campo.

19.5 Privilegios

Los binarios del shell deben ejecutarse como usuario. Las acciones privilegiadas deben pasar por servicios del sistema y PolicyKit. No hagas setuid a componentes de interfaz.

19.6 Bloqueo de sesión

Un lock seguro debe:

- obtener el protocolo de session lock del compositor;
- cubrir todas las salidas, incluidas las conectadas durante el bloqueo;
- impedir acceso a superficies previas;
- autenticar mediante PAM en un proceso cuidadosamente diseñado;
- limitar información visible;
- sobrevivir reinicios del shell sin desbloquear.

19.7 IPC privado

El protocolo futuro note-shell-v1 debe:

- ser versionado;
- validar la identidad del cliente privilegiado;
- no exponer control global a cualquier cliente Wayland;
- separar eventos de consulta y comandos;
- manejar reconexión del shell;
- documentar estados y errores.

19.8 Política de errores

Clasifica errores:

Recuperables falta una aplicación opcional; muestra aviso y continúa.

Degradables falla EGL; entra en modo Pixman y avisa.

Críticos de sesión no existe compositor o runtime dir; aborta con diagnóstico.

Críticos de seguridad lock o autenticación incompletos; no uses fallback inseguro silencioso.

19.9 Métricas útiles

- tiempo desde autenticación hasta panel visible;
- uso de memoria por proceso;
- número de procesos externos por minuto;
- latencia de overview;
- frames perdidos durante animaciones;
- tiempo de recuperación del shell;
- éxito de importación DMA-BUF por GPU;
- fallos por versión de distribución.

Roadmap hacia note-compositor con Smithay

20.1 Objetivo

La transición no debe reescribir todo al mismo tiempo. El shell, settings, sesión, empaquetado e idiomas pueden sobrevivir mientras se sustituye el compositor. La estrategia correcta es construir **note-compositor** en paralelo y activar funciones de forma incremental.

20.2 Fase 0.3: compositor anidado

Primero se ejecuta dentro de una sesión existente. Debe implementar:

- bucle de eventos calloop;
- display Wayland;
- xdg-shell;
- layer-shell;
- seat, teclado y puntero;
- foco, mover, redimensionar, maximizar y fullscreen;
- salida virtual;
- protocolo inicial para el shell;
- XWayland rootless.

No se toca DRM todavía. Esto permite depurar política de ventanas sin perder la TTY.

20.3 Estructura propuesta

Listing 20.1: Estructura propuesta para note-compositor

```
1 crates/note-compositor/src/|—  
2   main.rs|—  
3   state.rs|—  
4   backend/|  
5     |— mod.rs|  
6     |— nested.rs|
```



```

7  | tty.rs|
8  | udev.rs|
9  | drm.rs|
10 | renderer.rs|
11 | multi_gpu.rs|
12 | protocols/|
13 |   compositor.rs|
14 |   xdg_shell.rs|
15 |   layer_shell.rs|
16 |   dmabuf.rs|
17 |   explicit_sync.rs|
18 |   note_shell.rs|
19 | input/|
20 |   keyboard.rs|
21 |   pointer.rs|
22 |   touch.rs|
23 |   gestures.rs|
24 | window/|
25 |   model.rs|
26 |   layout.rs|
27 |   animation.rs|
28 |   rules.rs|
29 | xwayland/
30 |   server.rs
31 |   xwm.rs
32 |   selection.rs

```

20.4 Fase 0.4: backend DRM/KMS

Después se agrega ejecución directa desde TTY:

- sesión libseat/logind;
- udev y hotplug;
- DRM devices y outputs;
- GBM/EGL;
- renderer OpenGL ES y fallback Pixman;
- page flips y presentation feedback;
- DMA-BUF feedback;
- multi-GPU;
- sincronización explícita;
- direct scanout.

20.5 Fase 0.5: experiencia visual

Una vez estable la presentación:

- escena con transformaciones;
- animaciones por tiempo monotónico;
- redondeo y sombras;
- blur por pass de render;
- miniaturas en vivo;

- overview coordinado con el shell;
- escritorios dinámicos;
- gestos de touchpad;
- minimización hacia el dock.

20.6 Protocolo note-shell-v1

El shell necesita recibir:

- lista de ventanas, app ID, título, estado y escritorio;
- ventana activa;
- lista de escritorios y activo;
- salidas, escala y geometría;
- eventos de apertura/cierre/foco;
- thumbnails o handles de textura controlados;
- capacidad de enfocar, cerrar, mover, minimizar y cambiar escritorio;
- señal para iniciar/terminar overview y coordinar animaciones.

20.7 XWayland

El compositor debe lanzar XWayland bajo demanda, iniciar un XWM, mapear ventanas X11 al modelo común y manejar selecciones/portapapeles. El dock y overview deben tratar ventanas Wayland y X11 con la misma abstracción.

20.8 Migración del shell

Crea una capa `ToplevelBackend`. Durante transición:

- backend actual: `wlrctl`;
- backend nuevo: protocolo `note-shell-v1`.

Así el shell puede compilar y probarse contra ambos compositores.

20.9 Criterios para retirar labwc

No retires labwc hasta que note-compositor pueda:

1. arrancar en AMD, Intel, NVIDIA y VM;
2. abrir aplicaciones Wayland y X11;
3. manejar varias ventanas, popups y diálogos;
4. cambiar resolución y conectar monitores;
5. suspender/reanudar;
6. bloquear la sesión;
7. compartir pantalla mediante portal;
8. recuperarse o mostrar modo seguro;
9. pasar pruebas de estabilidad prolongadas.

No empieces por el blur

El blur luce chido, pero no sirve si el compositor pierde eventos, rompe popups o deja la pantalla negra al conectar HDMI. Primero corrección y estabilidad; después efectos.

Problemas conocidos, historial de fallos y soluciones

Este capítulo documenta fallos encontrados durante la instalación real en Debian 13. Son valiosos porque muestran dónde el proyecto asumía demasiado sobre distribución, locale o API.

21.1 policykit-1-gnome sin candidato

Debian 13 no ofreció candidato instalable para `policykit-1-gnome`. La dependencia se sustituyó por **lxpolkit**. Lección: los nombres de paquetes cambian y el instalador debe consultar candidatos reales antes de realizar una transacción.

21.2 Todos los paquetes aparecían sin candidato

El script buscaba la etiqueta inglesa `Candidate:`, pero APT respondía `Candidato:`. La corrección fue ejecutar `LC_ALL=C apt-cache policy`. Lección: nunca parsees salida localizada sin fijar locale.

21.3 Fallo repetitivo de descarga de APT

APT mostró múltiples mensajes `Tried to start delayed item ... but failed`. Se agregaron reintentos, limpieza de parciales y pipeline depth cero. El instalador debe distinguir error de mirror, DNS, almacenamiento y paquete inexistente.

21.4 greetd instalado pero no encontrado

Debian instala `/usr/sbin/greetd`; el PATH del usuario no incluía `sbin`. Se corrigió exportando un PATH completo y buscando rutas explícitas.

21.5 API de glib4-layer-shell

El código inicial usaba firmas distintas:

- `set_layer_shell_margin` en lugar de `set_margin`;
- `set_namespace(&str)` en lugar de `set_namespace(Some(&str))`;
- `set_monitor(&Monitor)` en lugar de `set_monitor(Some(&Monitor))`;
- `hide()` obsoleto en lugar de `set_visible(false)`.

```

--> crates/note-shell/src/main.rs:696:24
696 |         window.hide();
    |         ^^^^^
error[E0308]: mismatched types
--> crates/note-shell/src/main.rs:810:26
810 |         window.set_namespace(namespace);
    |                   ^^^^^^^^^ expected `Option<&str>`, found `&str`
    |
    = note: arguments to this method are incorrect
    = note: expected enum `std::option::Option<&str>`
            found reference `&str`
note: method defined here
--> /home/user/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/gtk4-layer-shell-0.7.1/src/lib.rs:291:8
291 |     fn set_namespace(&self, name_space: Option<&str>) {
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: try wrapping the expression in `Some`
810 |         window.set_namespace(Some(namespace));
    |                             ++++++
error[E0308]: mismatched types
--> crates/note-shell/src/main.rs:812:28
812 |         window.set_monitor(monitor);
    |                   ^^^^^^^^^ expected `Option<&Monitor>`, found `&Monitor`
    |
    = note: arguments to this method are incorrect
    = note: expected enum `std::option::Option<&Monitor>`
            found reference `&Monitor`
note: method defined here
--> /home/user/.cargo/registry/src/index.crates.io-1949cf8c6b5b557f/gtk4-layer-shell-0.7.1/src/lib.rs:274:8
274 |     fn set_monitor(&self, monitor: Option<&gtk::Monitor>) {
    |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: try wrapping the expression in `Some`
812 |         window.set_monitor(Some(monitor));
    |                             ++++++
Some errors have detailed explanations: E0308, E0599.
For more information about an error, try `rustc --explain E0308`.
warning: `note-shell` (bin "note-shell") generated 7 warnings
error: could not compile `note-shell` (bin "note-shell") due to 8 previous errors; 7 warnings emitted
user@debian-vm:~/desktop$

```

Figura 21.1: Errores de tipos provocados por diferencias de API de gtk4-layer-shell.

Lección: compila contra la versión mínima declarada y consulta documentación exacta de esa versión, no ejemplos de latest.

21.6 ApplicationFlags::DEFAULT_FLAGS

La constante no estaba disponible con las características/versión de gio usadas. Se sustituyó por `gio::ApplicationFlags::empty()`.

```

debian-13 [Running] - Oracle VirtualBox
Leyendo la información de estado... Hecho
blueman ya está en su versión más reciente (2.4.4-1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
epiphany-browser ya está en su versión más reciente (48.5-0+deb13u1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
firefox-esr ya está en su versión más reciente (140.12.0esr-1~deb13u1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
gnome-text-editor ya está en su versión más reciente (48.3-3).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
eog ya está en su versión más reciente (47.0-1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
evince ya está en su versión más reciente (48.1-3+deb13u1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
file-roller ya está en su versión más reciente (44.5-1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
gnome-calculator ya está en su versión más reciente (1:48.1-2+b1).
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 1 no actualizados.
[ok] greetd: /usr/sbin/greetd
Compiling note-core v0.2.2 (/home/user/desknote/crates/note-core)
Compiling note-shell v0.2.2 (/home/user/desknote/crates/note-shell)
error[E0599]: no associated item named `DEFAULT_FLAGS` found for struct `ApplicationFlags` in the current scope
--> crates/note-shell/src/main.rs:33:39
33 |         .flags(gio::ApplicationFlags::DEFAULT_FLAGS)
   |                                     ^^^^^^^^^^^^^^^ associated item not found in `ApplicationFlags`

For more information about this error, try `rustc --explain E0599`.
error: could not compile `note-shell` (bin "note-shell") due to 1 previous error
user@debian-vm:~/desknote$

```

Figura 21.2: Error E0599 por una constante no disponible.

21.7 Movimiento parcial de controls.page

Ya descrito en [Capítulo 9](#). La corrección es `controls.page.clone()`.

21.8 SnapToEdge combine

Una versión previa de la configuración usaba un argumento `combine` que la versión de `labwc` instalada no entendía. Se eliminó. Lección: la configuración XML también tiene compatibilidad por versión y necesita pruebas.

21.9 EGL en VirtualBox

Con 3D desactivado se observó `EGL_NOT_INITIALIZED`. Se resolvió activando `VMSVGA/3D` o usando `Pixman`. El mensaje `Lost connection to Wayland compositor` era consecuencia, no causa.

21.10 note-doctor muestra salida Vulkan extensa

El script usa `vulkaninfo --summary`, pero algunas versiones pueden ignorar o interpretar distinto la opción. Mejoras:

- detectar soporte de `--summary`;
- limitar por `timeout`;

- capturar salida y mostrar sólo driver/dispositivo;
- no bloquear el doctor por un driver defectuoso.

21.11 Herramientas externas ausentes

Si faltan wdisplays, pavucontrol, Blueman o gtklock, algunas páginas o acciones no funcionan. La UI debería mostrar “componente no instalado” y sugerir el paquete, no fallar silenciosamente.

21.12 Lista de limitaciones funcionales

- no hay blur real;
- overview sin miniaturas;
- escritorios fijos;
- indicadores de panel mayormente estáticos;
- no hay estado de Wi-Fi conectado, SSID ni dispositivos Bluetooth;
- no hay gestión nativa de monitores;
- no hay menú global de aplicaciones;
- no hay badges ni previews en dock;
- no hay autohide inteligente;
- no hay accesibilidad completa;
- greeter y lock no son propios.

La lección general

Cada error encontrado redujo una suposición oculta: idioma de APT, ubicación de sbin, nombre de paquete, firma de API o capacidad de la GPU virtual. Convierte esas lecciones en pruebas automatizadas para no volver a tropezar con la misma piedra.

Flujo de contribución y mantenimiento

22.1 Repositorio Git

Ramas sugeridas:

main código que compila y puede producir una alpha.

develop integración de cambios para la siguiente versión.

feature/* funciones aisladas.

fix/* correcciones reproducibles.

release/* preparación de paquete, changelog y migraciones.

Un proyecto pequeño también puede usar trunk-based development; lo importante es que main siempre pase CI.

22.2 Formato de commits

Ejemplos:

```
1 feat(shell): agregar indicador de red al panel
2 fix(settings): clonar widget antes de capturarlo
3 refactor(core): separar parser de archivos desktop
4 packaging(debian): dividir themes en paquete independiente
5 docs: documentar fallback Pixman
```

22.3 Checklist de pull request

- el cambio tiene objetivo claro;
- cargo fmt pasa;
- cargo clippy no agrega warnings;
- tests pasan;
- scripts pasan shellcheck o al menos `sh -n`;
- se probó en sesión Wayland;

- se documentaron variables/rutas nuevas;
- se actualizaron traducciones;
- no se introdujeron comandos shell con datos no confiables;
- se agregó entrada a changelog si afecta usuarios.

22.4 Versionado

Mientras la API y configuración cambian rápido, usa SemVer pre-1.0:

- 0.2.4: correcciones compatibles;
- 0.3.0: compositor anidado y APIs nuevas;
- 0.4.0: backend hardware;
- 1.0.0: primera sesión diaria estable.

Los archivos de configuración también necesitan `schema_version`. Ejemplo:

```
1 schema_version = 1
2 language = "es-MX"
3 # ...
```

22.5 Release

1. actualizar versión en Cargo.toml, VERSION y paquetes;
2. actualizar Cargo.lock;
3. ejecutar CI en todas las distribuciones soportadas;
4. probar instalación desde cero y actualización;
5. generar tarball y checksums;
6. firmar tag y artefactos;
7. publicar notas con cambios, problemas conocidos y recuperación;
8. conservar paquetes anteriores para rollback.

22.6 Matriz de pruebas recomendada

Sistema	Gráficos	Objetivo
Debian 13 VM	VirtualBox VMSVGA	Instalador, Pixman, login.
Ubuntu Server reciente VM	VirtualBox/VMware	Nombres de paquetes y universe.
Debian hardware	Intel	Mesa, suspensión, monitores.
Ubuntu hardware	AMD	Wayland, Vulkan, VRR.
Debian/Ubuntu hardware	NVIDIA propietario	DRM KMS, XWayland y explicit sync futura.
Laptop híbrida	Intel/AMD + NVIDIA	PRIME, outputs y consumo.

22.7 Documentación como requisito

Cada módulo nuevo debe incluir:

- propósito y frontera;
- API pública;
- dependencias externas;
- rutas y variables;
- fallbacks;
- errores esperados;
- prueba manual y automatizada;
- consideraciones de seguridad.

Referencia de rutas y variables

A.1 Rutas del repositorio

<code>crates/note-core</code>	Lógica independiente de interfaz.
<code>crates/note-shell</code>	Shell GTK4/layer-shell.
<code>crates/note-settings</code>	Aplicación de configuración.
<code>assets/style</code>	CSS.
<code>assets/themes</code>	Decoración labwc.
<code>assets/wallpapers</code>	Fondos SVG.
<code>configs/greetd</code>	Login.
<code>configs/labwc</code>	Política temporal de ventanas.
<code>configs/xdg-desktop-portal</code>	Selección de portales.
<code>systemd/user</code>	Servicios de sesión.
<code>packaging</code>	Metadatos de gestores de paquetes.

A.2 Rutas instaladas globales

<code>/usr/bin/note-shell</code>	Binario shell.
<code>/usr/bin/note-settings</code>	Configuración.
<code>/usr/bin/note-session</code>	Entry point de la sesión.
<code>/usr/libexec/note-desktop</code>	Scripts internos.
<code>/usr/share/note-desktop/style</code>	CSS instalado.
<code>/usr/share/note-desktop/wallpapers</code>	Fondos.
<code>/usr/share/note-desktop/locales</code>	Catálogos.
<code>/usr/share/themes/Note/labwc</code>	Tema de ventanas.
<code>/usr/share/wayland-sessions/note.desktop</code>	Entrada de sesión.
<code>/usr/share/xdg-desktop-portal/note-portals.conf</code>	Selección de portales.
<code>/etc/xdg/note/labwc</code>	Configuración global de labwc.

<code>/etc/greetd/note-config.toml</code>	Configuración Note conservada.
<code>/etc/greetd/config.toml</code>	Configuración activa de greetd.
<code>/etc/note-desktop/gpu.conf</code>	Overrides de GPU opcionales.
<code>/var/lib/note-desktop</code>	Respalos y estado del instalador.

A.3 Rutas de usuario

<code>~/.config/note-desktop/settings.toml</code>	Preferencias.
<code>~/.config/note-desktop/gpu.conf</code>	Overrides de GPU del usuario.
<code>~/.config/labwc/rc.xml</code>	Fragmento generado para labwc.
<code>~/.config/environment.d/90-note-locale.conf</code>	Locale de sesión.
<code>~/Pictures/Screenshots</code>	Capturas.
<code>\$XDG_RUNTIME_DIR/bus</code>	Bus D-Bus de usuario.
<code>\$XDG_RUNTIME_DIR/wayland-*</code>	Socket Wayland.

A.4 Variables de entorno

<code>XDG_SESSION_TYPE</code>	wayland.
<code>XDG_SESSION_DESKTOP</code>	note.
<code>XDG_CURRENT_DESKTOP</code>	Note:labwc:wlroots.
<code>WAYLAND_DISPLAY</code>	Socket Wayland asignado por compositor.
<code>DISPLAY</code>	Display XWayland, si está activo.
<code>GTK_USE_PORTAL</code>	Solicita portales.
<code>WLR_RENDERER</code>	Renderer forzado, por ejemplo pixman.
<code>WLR_NO_HARDWARE_CURSORS</code>	Cursor por software.
<code>WLR_DRM_DEVICES</code>	Orden explícito de devices DRM, usar con cuidado.
<code>MOZ_ENABLE_WAYLAND</code>	Preferencia Mozilla.
<code>QT_QPA_PLATFORM</code>	Backend Qt.
<code>SDL_VIDEODRIVER</code>	Backends SDL.
<code>ELECTRON_OZONE_PLATFORM_HINT</code>	Selección Electron/Ozone.
<code>LANG, LANGUAGE, LC_MESSAGES</code>	Idioma de procesos.

Referencia de comandos

B.1 Desarrollo

<code>cargo check --workspace --all-targets</code>	Comprobar tipos sin generar release.
<code>cargo build --workspace --release</code>	Compilar binarios optimizados.
<code>cargo test --workspace</code>	Ejecutar pruebas.
<code>cargo fmt --all</code>	Formatear.
<code>cargo clippy --workspace --all-targets</code>	Lint estricto.
<code>-- -D warnings</code>	
<code>make build / make check</code>	Atajos del Makefile.

B.2 Sesión

<code>note-test-from-tty</code>	Iniciar sesión sin greetd.
<code>note-overview</code>	Alternar overview.
<code>note-control-center</code>	Alternar centro de control.
<code>note-settings</code>	Abrir Configuración.
<code>note-lock</code>	Bloquear.
<code>note-logout</code>	Salir de labwc/sesión.
<code>note-screenshot</code>	Captura de región.
<code>note-doctor</code>	Diagnóstico.

B.3 systemd

```
1 systemctl status greetd
2 sudo systemctl restart greetd
3 sudo systemctl reset-failed greetd
4 systemctl --user status note-session.target
5 systemctl --user restart note-shell.service
6 journalctl --user -u note-shell.service -b
7 sudo journalctl -u greetd -b
```

B.4 Gráficos

```
1 ls -l /dev/dri
2 lspci -nnk | grep -EA3 'VGA|3D|Display'
3 glxinfo -B
4 vulkaninfo --summary
5 cat /sys/module/nvidia_drm/parameters/modeset
6 WLR_RENDERER=pixman WLR_NO_HARDWARE_CURSORS=1 note-test-from-tty
```

B.5 Audio, red y Bluetooth

```
1 wpctl status
2 wpctl get-volume @DEFAULT_AUDIO_SINK@
3 nmcli general status
4 nmcli radio wifi
5 bluetoothctl show
6 brightnessctl -m
```

B.6 Empaquetado

```
1 ./scripts/build-deb.sh
2 sudo apt install ./dist/note-desktop_0.2.3_amd64.deb
3 dpkg -L note-desktop
4 sudo apt purge note-desktop
```

B.7 Recuperación

```
1 # Desde una TTY
2 sudo systemctl disable --now greetd
3 sudo systemctl set-default multi-user.target
4 sudo reboot
```

Referencia exhaustiva de archivos

Este apéndice enumera todos los archivos del snapshot documentado. La columna de líneas es aproximada y sirve para localizar componentes grandes.

Ruta	Tipo	Responsabilidad	Líneas
.github/workflows/ci.yml	Texto/otro	Workflow de integración continua.	23
Cargo.toml	TOML	Manifiesto raíz del workspace y versiones compartidas.	21
LICENSE	Texto/otro	Licencia GPL-3.0-or-later.	674
Makefile	Texto/otro	Atajos de compilación, validación, instalación y empaquetado.	20
PATCHES_APLICADOS.md	Markdown	Correcciones posrelease incluidas en el snapshot documental.	3
README.md	Markdown	Introducción breve, instalación y limitaciones de la release.	234
VERSION	Texto/otro	Versión textual consumida por el instalador.	1
assets/icons/note-desktop-symbolic.svg	SVG	Icono simbólico de Note Desktop.	3
assets/note-settings.desktop	Desktop entry	Entrada de aplicación de Configuración.	11
assets/style/settings.css	CSS	Hoja CSS de la interfaz.	44
assets/style/shell.css	CSS	Hoja CSS de la interfaz.	186
assets/themes/Note/labwc/close-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/close-hover-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/close-hover-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/close-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/iconify-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/iconify-hover-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/iconify-hover-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/iconify-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max-hover-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max-hover-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max_toggled-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max_toggled-hover-active.svg	SVG	Archivos de decoración del tema Note para labwc.	1

Ruta	Tipo	Responsabilidad	Líneas
assets/themes/Note/labwc/max_toggled-hover-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/max_toggled-inactive.svg	SVG	Archivos de decoración del tema Note para labwc.	1
assets/themes/Note/labwc/themerc	Texto/otro	Archivos de decoración del tema Note para labwc.	36
assets/wallpapers/note-login.svg	SVG	Fondo vectorial SVG.	11
assets/wallpapers/note-wave.svg	SVG	Fondo vectorial SVG.	19
configs/gpu.conf.example	Texto/otro	Plantilla de configuración opcional.	9
configs/greetd/config.toml	TOML	Configuración o estilo del greeter.	6
configs/greetd/note-greet.css	CSS	Configuración o estilo del greeter.	34
configs/labwc/autostart	Texto/otro	Entry point de autostart de labwc.	2
configs/labwc/environment	Texto/otro	Variables de entorno de la sesión.	12
configs/labwc/menu.xml	XML	Menú contextual raíz.	14
configs/labwc/rc.xml	XML	Política de ventanas, entrada, escritorios y atajos.	97
configs/labwc/shutdown	Texto/otro	Acciones al cerrar labwc.	3
configs/labwc/themerc-override	Texto/otro	Ajustes del tema de decoración.	22
configs/wayland-session/note.desktop	Desktop entry	Entrada XDG de sesión Wayland.	9
configs/xdg-desktop-portal/note-portals.conf	Texto/otro	Selección de backends de portales.	5
crates/note-core/Cargo.toml	TOML	Manifiesto de un crate Rust.	10
crates/note-core/src/config.rs	Rust	Modelo de preferencias, carga/guardado, locale y variables CSS.	141
crates/note-core/src/desktop.rs	Rust	Descubrimiento, parsing y lanzamiento de archivos .desktop.	192
crates/note-core/src/i18n.rs	Rust	Traductor y parser simplificado de catálogos.	66
crates/note-core/src/lib.rs	Rust	Raíz de note-core; reexporta módulos y tipos públicos.	8
crates/note-core/src/system.rs	Rust	Adaptadores de ventanas, volumen, brillo, Wi-Fi y Bluetooth.	112
crates/note-settings/Cargo.toml	TOML	Manifiesto de un crate Rust.	10
crates/note-settings/src/main.rs	Rust	Aplicación gráfica de preferencias y lanzadores de configuración.	397
crates/note-shell/Cargo.toml	TOML	Manifiesto de un crate Rust.	12

Ruta	Tipo	Responsabilidad	Líneas
crates/note-shell/src/main.rs	Rust	Aplicación shell: panel, dock, overview y centro de control.	919
docs/ARCHITECTURE.md	Markdown	Documento breve incluido en el repositorio.	97
docs/CHANGELOG.md	Markdown	Documento breve incluido en el repositorio.	15
docs/PACKAGING.md	Markdown	Documento breve incluido en el repositorio.	47
docs/ROADMAP.md	Markdown	Documento breve incluido en el repositorio.	69
docs/VALIDATION.md	Markdown	Documento breve incluido en el repositorio.	26
locales/de-DE/shell.ftl	Texto/otro	Catálogo de traducción para el locale indicado.	71
locales/en-US/shell.ftl	Texto/otro	Catálogo de traducción para el locale indicado.	71
locales/es-MX/shell.ftl	Texto/otro	Catálogo de traducción para el locale indicado.	71
locales/fr-FR/shell.ftl	Texto/otro	Catálogo de traducción para el locale indicado.	71
locales/pt-BR/shell.ftl	Texto/otro	Catálogo de traducción para el locale indicado.	71
packaging/arch/PKGBUILD	Texto/otro	Receta PKGBUILD para Arch.	21
packaging/debian/changelog	Texto/otro	Metadato o regla de empaquetado Debian.	13
packaging/debian/control	Texto/otro	Metadato o regla de empaquetado Debian.	15
packaging/debian/rules	Texto/otro	Metadato o regla de empaquetado Debian.	9
packaging/debian/source/format	Texto/otro	Metadato o regla de empaquetado Debian.	1
packaging/rpm/note-desktop.spec	RPM spec	Especificación de paquete RPM.	71
scripts/build-deb.sh	Bash	Generador de paquete .deb local.	105
scripts/install-files.sh	Bash	Copia binarios y recursos a /usr o DESTDIR.	58
scripts/install.sh	Bash	Instalador de desarrollo para Debian/Ubuntu.	189
scripts/make-release.sh	Bash	Generación de archivos de release.	17
scripts/note-apply-settings	Texto/otro	Traduce preferencias a configuración de labwc.	31
scripts/note-autostart	Texto/otro	Fondo, entorno y target de servicios.	13
scripts/note-browser	Texto/otro	Selecciona navegador.	6
scripts/note-control-center	Texto/otro	Invoca la acción del centro de control.	6
scripts/note-doctor	Texto/otro	Diagnóstico de instalación, sesión y GPU.	46
scripts/note-editor	Texto/otro	Selecciona editor.	5
scripts/note-files	Texto/otro	Selecciona gestor de archivos.	6
scripts/note-greeter-session	Texto/otro	Ejecuta cage/gtkgreet con fallback Pixman.	23
scripts/note-lock	Texto/otro	Selecciona lock screen disponible.	4

Ruta	Tipo	Responsabilidad	Líneas
scripts/note-logout	Texto/otro	Finaliza labwc.	2
scripts/note-nvidia-kms	Texto/otro	Habilita options de nvidia_drm y reconstruye initramfs.	8
scripts/note-overview	Texto/otro	Invoca la acción GApplication del overview.	6
scripts/note-poweroff	Texto/otro	Apaga el sistema.	2
scripts/note-reboot	Texto/otro	Reinicia el sistema.	2
scripts/note-screenshot	Texto/otro	Captura con grim/slurp.	12
scripts/note-session	Texto/otro	Entry point de la sesión del usuario.	29
scripts/note-session-inner	Texto/otro	Importa entorno y ejecuta labwc con fallback.	29
scripts/note-terminal	Texto/otro	Selecciona terminal.	13
scripts/note-test-from-tty	Texto/otro	Prueba de sesión directa desde una TTY.	7
scripts/note-workspace	Texto/otro	Cambia de escritorio mediante wlrctl.	8
scripts/patch-0.2.1-build.sh	Bash	Parche de compatibilidad con gtk4-layer-shell 0.7.1.	24
scripts/patch-0.2.2-build.sh	Bash	Parche de ApplicationFlags.	30
scripts/preflight.sh	Bash	Comprobaciones previas rápidas.	28
scripts/uninstall.sh	Bash	Eliminación y restauración de greetd.	34
systemd/user/note-notifications.service	systemd	Unidad systemd -user de la sesión Note.	13
systemd/user/note-polkit-agent.service	systemd	Unidad systemd -user de la sesión Note.	13
systemd/user/note-session.target	systemd	Unidad systemd -user de la sesión Note.	9
systemd/user/note-shell.service	systemd	Unidad systemd -user de la sesión Note.	14

Listados técnicos seleccionados

Este apéndice reproduce archivos y secciones clave para poder estudiar el proyecto sin saltar constantemente al editor. El snapshot completo acompaña al manual.

D.1 Configuración persistente completa

```
1 use serde::{Deserialize, Serialize};
2 use std::{
3     env,
4     fs,
5     io,
6     path::{Path, PathBuf},
7 };
8
9 #[derive(Debug, Clone, Copy, Serialize, Deserialize, PartialEq, Eq)]
10 #[serde(rename_all = "kebab-case")]
11 pub enum AppearanceMode {
12     Auto,
13     Light,
14     Dark,
15 }
16
17 impl Default for AppearanceMode {
18     fn default() -> Self {
19         Self::Dark
20     }
21 }
22
23 #[derive(Debug, Clone, Serialize, Deserialize)]
24 #[serde(default)]
25 pub struct NoteSettings {
26     pub language: String,
27     pub appearance: AppearanceMode,
28     pub accent: String,
29     pub dock_size: i32,
30     pub panel_opacity: f64,
31     pub dock_opacity: f64,
32     pub animations: bool,
33     pub natural_scroll: bool,
34     pub workspaces: u8,
35     pub favorites: Vec<String>,
36 }
```

```

36 }
37
38 impl Default for NoteSettings {
39     fn default() -> Self {
40         Self {
41             language: "es-MX".to_owned(),
42             appearance: AppearanceMode::Dark,
43             accent: "blue".to_owned(),
44             dock_size: 50,
45             panel_opacity: 0.86,
46             dock_opacity: 0.78,
47             animations: true,
48             natural_scroll: true,
49             workspaces: 4,
50             favorites: vec![
51                 "foot.desktop".into(),
52                 "thunar.desktop".into(),
53                 "org.gnome.Epiphany.desktop".into(),
54                 "org.gnome.TextEditor.desktop".into(),
55                 "org.gnome.Settings.desktop".into(),
56             ],
57         }
58     }
59 }
60
61 impl NoteSettings {
62     pub fn load() -> Self {
63         let path = settings_path();
64         match fs::read_to_string(path) {
65             Ok(contents) => toml::from_str(&contents).unwrap_or_default(),
66             Err(_) => Self::default(),
67         }
68     }
69
70     pub fn save(&self) -> io::Result<> {
71         let path = settings_path();
72         if let Some(parent) = path.parent() {
73             fs::create_dir_all(parent)?;
74         }
75         let data = toml::to_string_pretty(self)
76             .map_err(|error| io::Error::new(io::ErrorKind::InvalidData, error))?;
77         fs::write(path, data)
78     }
79
80     pub fn write_locale_environment(&self) -> io::Result<> {
81         let path = environment_dir().join("90-note-locale.conf");
82         if let Some(parent) = path.parent() {
83             fs::create_dir_all(parent)?;
84         }
85         let locale = locale_to_posix(&self.language);
86         fs::write(
87             path,
88             format!(
89                 "LANG={locale}.UTF-8\nLANGUAGE={}{}\nLC_MESSAGES={locale}.UTF-8\n",
90                 locale
91             ),
92         )
93     }
94 }

```

```

95     pub fn css_variables(&self) -> String {
96         let (r, g, b) = accent_rgb(&self.accent);
97         format!(
98             "@define-color note_accent rgb({r}, {g}, {b});\n\
99             @define-color note_panel_bg rgba(20, 22, 28, {:.3});\n\
100            @define-color note_dock_bg rgba(26, 28, 35, {:.3});\n",
101            self.panel_opacity.clamp(0.25, 1.0),
102            self.dock_opacity.clamp(0.25, 1.0),
103        )
104     }
105 }
106
107 pub fn settings_path() -> PathBuf {
108     config_home().join("note-desktop/settings.toml")
109 }
110
111 pub fn config_home() -> PathBuf {
112     env::var_os("XDG_CONFIG_HOME")
113         .map(PathBuf::from)
114         .unwrap_or_else(|| home_dir().join(".config"))
115 }
116
117 pub fn environment_dir() -> PathBuf {
118     config_home().join("environment.d")
119 }
120
121 pub fn home_dir() -> PathBuf {
122     env::var_os("HOME")
123         .map(PathBuf::from)
124         .unwrap_or_else(|| Path::new("/").to_path_buf())
125 }
126
127 fn locale_to_posix(locale: &str) -> String {
128     locale.replace('-', "_")
129 }
130
131 fn accent_rgb(accent: &str) -> (u8, u8, u8) {
132     match accent {
133         "purple" => (175, 112, 255),
134         "pink" => (255, 99, 160),
135         "red" => (255, 86, 86),
136         "orange" => (255, 150, 62),
137         "green" => (79, 210, 132),
138         "teal" => (52, 205, 198),
139         _ => (84, 151, 255),
140     }
141 }

```

D.2 Traductor completo

```

1 use std::collections::HashMap;
2
3 const ES_MX: &str = include_str!("../../locales/es-MX/shell.ftl");
4 const EN_US: &str = include_str!("../../locales/en-US/shell.ftl");
5 const PT_BR: &str = include_str!("../../locales/pt-BR/shell.ftl");
6 const FR_FR: &str = include_str!("../../locales/fr-FR/shell.ftl");
7 const DE_DE: &str = include_str!("../../locales/de-DE/shell.ftl");

```

```

8
9 #[derive(Debug, Clone)]
10 pub struct Translator {
11     locale: String,
12     values: HashMap<String, String>,
13     fallback: HashMap<String, String>,
14 }
15
16 impl Translator {
17     pub fn new(locale: impl Into<String>) -> Self {
18         let locale = locale.into();
19         let source = match locale.as_str() {
20             "en-US" | "en" => EN_US,
21             "pt-BR" | "pt" => PT_BR,
22             "fr-FR" | "fr" => FR_FR,
23             "de-DE" | "de" => DE_DE,
24             _ => ES_MX,
25         };
26         Self {
27             locale,
28             values: parse(source),
29             fallback: parse(EN_US),
30         }
31     }
32
33     pub fn locale(&self) -> &str {
34         &self.locale
35     }
36
37     pub fn text(&self, key: &str) -> String {
38         self.values
39             .get(key)
40             .or_else(|| self.fallback.get(key))
41             .cloned()
42             .unwrap_or_else(|| key.to_owned())
43     }
44
45     pub fn format(&self, key: &str, replacements: &[(&str, &str)]) -> String {
46         let mut value = self.text(key);
47         for (name, replacement) in replacements {
48             value = value.replace(&format!("{{ ${name} }}"), replacement);
49         }
50         value
51     }
52 }
53
54 fn parse(source: &str) -> HashMap<String, String> {
55     source
56         .lines()
57         .filter_map(|line| {
58             let trimmed = line.trim();
59             if trimmed.is_empty() || trimmed.starts_with('#') || trimmed.starts_with
60 ('-') {
61                 return None;
62             }
63             let (key, value) = trimmed.split_once('=')?;
64             Some((key.trim().to_owned(), value.trim().to_owned()))
65         })
66         .collect()

```

66 }

D.3 Adaptadores del sistema completos

```

1 use std::{
2     process::{Command, Stdio},
3     str::FromStr,
4 };
5
6 #[derive(Debug, Clone)]
7 pub struct Toplevel {
8     pub app_id: String,
9     pub title: String,
10 }
11
12 pub fn run_quiet(command: &str, args: &[&str]) -> bool {
13     Command::new(command)
14         .args(args)
15         .stdin(Stdio::null())
16         .stdout(Stdio::null())
17         .stderr(Stdio::null())
18         .status()
19         .map(|status| status.success())
20         .unwrap_or(false)
21 }
22
23 pub fn command_output(command: &str, args: &[&str]) -> Option<String> {
24     let output = Command::new(command).args(args).output().ok()?;
25     if !output.status.success() {
26         return None;
27     }
28     Some(String::from_utf8_lossy(&output.stdout).trim().to_owned())
29 }
30
31 pub fn list_toplevels(filter: Option<&str>) -> Vec<Toplevel> {
32     let mut args = vec!["toplevel", "list"];
33     if let Some(filter) = filter {
34         args.push(filter);
35     }
36     let Some(output) = command_output("wlrctl", &args) else {
37         return Vec::new();
38     };
39     output
40         .lines()
41         .filter_map(|line| {
42             let (app_id, title) = line.split_once(':')?;
43             Some(Toplevel {
44                 app_id: app_id.trim().to_owned(),
45                 title: title.trim().to_owned(),
46             })
47         })
48         .collect()
49 }
50
51 pub fn focus_toplevel(toplevel: &Toplevel) -> bool {
52     run_quiet(
53         "wlrctl",

```



```

54     &[
55         "toplevel",
56         "focus",
57         &format!("app_id:{}", toplevel.app_id),
58         &format!("title:{}", toplevel.title),
59     ],
60 )
61 }
62
63 pub fn volume_percent() -> u8 {
64     let Some(output) = command_output("wpctl", &["get-volume", "@DEFAULT_AUDIO_SINK@"])
65     else {
66         return 0;
67     };
68     output
69     .split_whitespace()
70     .find_map(|token| f64::from_str(token).ok())
71     .map(|value| (value * 100.0).round().clamp(0.0, 150.0) as u8)
72     .unwrap_or(0)
73 }
74
75 pub fn set_volume(percent: f64) {
76     let value = format!("{}", percent.round().clamp(0.0, 150.0));
77     let _ = run_quiet("wpctl", &["set-volume", "@DEFAULT_AUDIO_SINK@", &value]);
78 }
79
80 pub fn brightness_percent() -> Option<u8> {
81     let output = command_output("brightnessctl", &["-m"])?;
82     output
83     .split(',')
84     .find(|part| part.trim_end().ends_with('%'))
85     .and_then(|part| part.trim().trim_end_matches('%').parse().ok())
86 }
87
88 pub fn set_brightness(percent: f64) {
89     let value = format!("{}", percent.round().clamp(1.0, 100.0));
90     let _ = run_quiet("brightnessctl", &["set", &value]);
91 }
92
93 pub fn wifi_enabled() -> bool {
94     command_output("nmcli", &["radio", "wifi"])
95     .map(|value| value.eq_ignore_ascii_case("enabled"))
96     .unwrap_or(false)
97 }
98
99 pub fn set_wifi(enabled: bool) {
100     let state = if enabled { "on" } else { "off" };
101     let _ = run_quiet("nmcli", &["radio", "wifi", state]);
102 }
103
104 pub fn bluetooth_enabled() -> bool {
105     command_output("bluetoothctl", &["show"])
106     .map(|output| output.lines().any(|line| line.trim() == "Powered: yes"))
107     .unwrap_or(false)
108 }
109
110 pub fn set_bluetooth(enabled: bool) {
111     let state = if enabled { "on" } else { "off" };
112     let _ = run_quiet("bluetoothctl", &["power", state]);

```

112 }

D.4 Arranque de note-shell y acciones

```

1 use chrono::{Local, Timelike};
2 use gtk::{gdk, gio, glib, prelude::*};
3 use gtk4_layer_shell::{Edge, KeyboardMode, Layer, LayerShell};
4 use note_core::{
5     DesktopApp, NoteSettings, Translator, discover_apps,
6     system::{
7         bluetooth_enabled, brightness_percent, focus_toplevel, list_toplevels,
8         set_bluetooth, set_brightness, set_volume, set_wifi, volume_percent,
9         wifi_enabled,
10     },
11 };
12 use std::{cell::RefCell, path::Path, process::{Command, Stdio}, rc::Rc};
13
14 const APP_ID: &str = "mx.note.desktop.shell";
15 const FALLBACK_CSS: &str = include_str!("../../assets/style/shell.css");
16
17 #[derive(Default)]
18 struct ShellState {
19     surfaces: Vec<gtk::ApplicationWindow>,
20     overview: Option<gtk::ApplicationWindow>,
21     control_center: Option<gtk::ApplicationWindow>,
22     settings: NoteSettings,
23     apps: Vec<DesktopApp>,
24 }
25
26 fn main() -> glib::ExitCode {
27     if std::env::args().any(|arg| arg == "--version" || arg == "-V") {
28         println!("note-shell {}", env!("CARGO_PKG_VERSION"));
29         return glib::ExitCode::SUCCESS;
30     }
31
32     let app = gtk::Application::builder()
33         .application_id(APP_ID)
34         .flags(gio::ApplicationFlags::empty())
35         .build();
36     let state = Rc::new(RefCell::new(ShellState::default()));
37
38     {
39         let state = state.clone();
40         app.connect_startup(move |app| {
41             register_actions(app, state.clone());
42         });
43     }
44     {
45         let state = state.clone();
46         app.connect_activate(move |app| {
47             if state.borrow().surfaces.is_empty() {
48                 rebuild_shell(app, &state);
49             }
50         });
51     }
52
53     app.run()

```

```

53 }
54
55 fn register_actions(app: &gtk::Application, state: Rc<RefCell<ShellState>>) {
56     let overview = gio::SimpleAction::new("overview", None);
57     {
58         let app = app.clone();
59         let state = state.clone();
60         overview.connect_activate(move |_, _| toggle_overview(&app, &state));
61     }
62     app.add_action(&overview);
63
64     let control_center = gio::SimpleAction::new("control-center", None);
65     {
66         let app = app.clone();
67         let state = state.clone();
68         control_center.connect_activate(move |_, _| toggle_control_center(&app, &state))
69     };
70     app.add_action(&control_center);
71
72     let reload = gio::SimpleAction::new("reload", None);
73     {
74         let app = app.clone();
75         let state = state.clone();
76         reload.connect_activate(move |_, _| rebuild_shell(&app, &state));
77     }
78     app.add_action(&reload);
79
80     let hide_overview = gio::SimpleAction::new("hide-overview", None);
81     {
82         let state = state.clone();
83         hide_overview.connect_activate(move |_, _| {
84             if let Some(window) = state.borrow().overview.as_ref() {
85                 window.set_visible(false);
86             }
87         });
88     }
89     app.add_action(&hide_overview);
90 }
91
92 fn rebuild_shell(app: &gtk::Application, state: &Rc<RefCell<ShellState>>) {
93     {
94         let mut state = state.borrow_mut();
95         for window in state-surfaces.drain(..) {
96             window.close();
97         }
98         if let Some(window) = state.overview.take() {
99             window.close();
100         }
101         if let Some(window) = state.control_center.take() {
102             window.close();
103         }
104         state.settings = NoteSettings::load();
105         state.apps = discover_apps(&state.settings.language);
106         install_css(&state.settings);
107         apply_gtk_preferences(&state.settings);
108     }
109
110     let Some(display) = gdk::Display::default() else {

```

```

111     eprintln!("note-shell: no se encontró un display Wayland");
112     return;
113 };
114 if !gtk4_layer_shell::is_supported() {
115     eprintln!("note-shell: el compositor no implementa wlr-layer-shell");
116     return;
117 }
118
119 let monitors = display.monitors();
120 if monitors.n_items() == 0 {
121     let panel = create_panel(app, state, None);
122     let dock = create_dock(app, state, None);
123     state.borrow_mut().surfaces.extend([panel, dock]);
124     return;
125 }
126
127 for index in 0..monitors.n_items() {
128     let Some(item) = monitors.item(index) else { continue };
129     let Ok(monitor) = item.downcast::(<gdk::Monitor>()) else { continue };
130     let panel = create_panel(app, state, Some(&monitor));
131     let dock = create_dock(app, state, Some(&monitor));
132     state.borrow_mut().surfaces.extend([panel, dock]);
133 }
134 }

```

D.5 Construcción del panel

```

1 fn create_panel(
2     app: &gtk::Application,
3     state: &Rc<RefCell<ShellState>>,
4     monitor: Option<&gdk::Monitor>,
5 ) -> gtk::ApplicationWindow {
6     let settings = state.borrow().settings.clone();
7     let tr = Translator::new(settings.language.clone());
8     let window = gtk::ApplicationWindow::builder()
9         .application(app)
10        .title("Note Panel")
11        .decorated(false)
12        .resizable(false)
13        .build();
14    configure_layer_window(&window, monitor, Layer::Top, "note-panel");
15    window.set_anchor(Edge::Top, true);
16    window.set_anchor(Edge::Left, true);
17    window.set_anchor(Edge::Right, true);
18    window.set_keyboard_mode(KeyboardMode::OnDemand);
19    window.auto_exclusive_zone_enable();
20
21    let left = gtk::Box::new(gtk::Orientation::Horizontal, 4);
22    left.set_margin_start(8);
23
24    let note_button = gtk::Button::builder()
25        .label("●")
26        .tooltip_text(tr.text("overview"))
27        .build();
28    note_button.add_css_class("note-logo");
29    {
30        let app = app.clone();

```

```

31     note_button.connect_clicked(move |_| app.activate_action("overview", None));
32 }
33 left.append(&note_button);
34
35 let active_app = gtk::Label::new(Some(&tr.text("desktop")));
36 active_app.add_css_class("active-application");
37 active_app.set_xalign(0.0);
38 left.append(&active_app);
39 update_active_application(&active_app, &tr);
40 {
41     let active_app = active_app.clone();
42     let tr = tr.clone();
43     glib::timeout_add_seconds_local(1, move || {
44         update_active_application(&active_app, &tr);
45         glib::ControlFlow::Continue
46     });
47 }
48
49 let clock_button = gtk::MenuButton::new();
50 clock_button.add_css_class("clock-button");
51 let clock = gtk::Label::new(None);
52 clock.add_css_class("panel-clock");
53 update_clock(&clock);
54 clock_button.set_child(Some(&clock));
55 clock_button.set_popover(Some(&calendar_popover(&tr)));
56 {
57     let clock = clock.clone();
58     glib::timeout_add_seconds_local(1, move || {
59         update_clock(&clock);
60         glib::ControlFlow::Continue
61     });
62 }
63
64 let right = gtk::Box::new(gtk::Orientation::Horizontal, 2);
65 right.set_margin_end(8);
66
67 let battery = gtk::Label::new(None);
68 battery.add_css_class("panel-status");
69 update_battery(&battery);
70 {
71     let battery = battery.clone();
72     glib::timeout_add_seconds_local(30, move || {
73         update_battery(&battery);
74         glib::ControlFlow::Continue
75     });
76 }
77 right.append(&battery);
78
79 let status_button = gtk::Button::new();
80 status_button.add_css_class("status-cluster");
81 status_button.set_tooltip_text(Some(&tr.text("control-center")));
82 let status_box = gtk::Box::new(gtk::Orientation::Horizontal, 7);
83 for icon in [
84     "network-wireless-signal-excellent-symbolic",
85     "audio-volume-high-symbolic",
86     "battery-good-symbolic",
87 ] {
88     let image = gtk::Image::from_icon_name(icon);
89     image.set_pixel_size(15);

```

```

90     status_box.append(&image);
91 }
92 status_button.set_child(Some(&status_box));
93 {
94     let app = app.clone();
95     status_button.connect_clicked(move |_| app.activate_action("control-center",
None));
96 }
97 right.append(&status_button);
98
99 let center = gtk::CenterBox::new();
100 center.add_css_class("panel");
101 center.set_start_widget(Some(&left));
102 center.set_center_widget(Some(&clock_button));
103 center.set_end_widget(Some(&right));
104 window.set_child(Some(&center));
105 window.present();
106 window
107 }

```

D.6 Construcción del dock

```

1  ●
2  fn create_dock(
3      app: &gtk::Application,
4      state: &Rc<RefCell<ShellState>>,
5      monitor: Option<&gdk::Monitor>,
6  ) -> gtk::ApplicationWindow {
7      let settings = state.borrow().settings.clone();
8      let apps = state.borrow().apps.clone();
9      let tr = Translator::new(settings.language.clone());
10     let window = gtk::ApplicationWindow::builder()
11         .application(app)
12         .title("Note Dock")
13         .decorated(false)
14         .resizable(false)
15         .build();
16     configure_layer_window(&window, monitor, Layer::Top, "note-dock");
17     window.set_anchor(Edge::Bottom, true);
18     window.set_margin(Edge::Bottom, 14);
19     window.set_keyboard_mode(KeyboardMode::None);
20     window.set_exclusive_zone(0);
21
22     let dock = gtk::Box::new(gtk::Orientation::Horizontal, 7);
23     dock.add_css_class("dock");
24
25     let overview = dock_action_button(
26         "view-app-grid-symbolic",
27         &tr.text("overview"),
28         settings.dock_size,
29     );
30     {
31         let app = app.clone();
32         overview.connect_clicked(move |_| app.activate_action("overview", None));
33     }
34     dock.append(&overview);
35     dock.append(&dock_separator());

```

```

36
37 let mut indicators: Vec<(String, gtk::Label)> = Vec::new();
38 for favorite in &settings.favorites {
39     let Some(desktop_app) = apps.iter().find(|candidate| &candidate.desktop_id ==
favorite).cloned() else {
40         continue;
41     };
42     let (button, indicator) = dock_app_button(&desktop_app, settings.dock_size);
43     indicators.push((desktop_app.app_id.clone(), indicator));
44     dock.append(&button);
45 }
46
47 dock.append(&dock_separator());
48 let settings_button = dock_action_button(
49     "preferences-system-symbolic",
50     &tr.text("settings"),
51     settings.dock_size,
52 );
53 settings_button.connect_clicked(|_| spawn("note-settings", &[]));
54 dock.append(&settings_button);
55
56 update_running_indicators(&indicators);
57 glib::timeout_add_seconds_local(2, move || {
58     update_running_indicators(&indicators);
59     glib::ControlFlow::Continue
60 });
61
62 window.set_child(Some(&dock));
63 window.present();
64 window
65 }
66
67 fn dock_app_button(app: &DesktopApp, size: i32) -> (gtk::Button, gtk::Label) {
68     let icon = gtk::Image::from_icon_name(&app.icon);
69     icon.set_pixel_size((size - 16).max(24));
70     let indicator = gtk::Label::new(Some("•"));
71     indicator.add_css_class("running-indicator");
72     indicator.set_opacity(0.0);
73
74     let content = gtk::Box::new(gtk::Orientation::Vertical, 0);
75     content.set_halign(gtk::Align::Center);
76     content.append(&icon);
77     content.append(&indicator);
78
79     let button = gtk::Button::builder().child(&content).build();
80     button.add_css_class("dock-item");
81     button.set_tooltip_text(Some(&app.name));
82     button.set_size_request(size, size + 6);
83
84     let base_icon_size = (size - 16).max(24);
85     let motion = gtk::EventControllerMotion::new();
86     {
87         let icon = icon.clone();
88         let button = button.clone();
89         motion.connect_enter(move |_, _, _| {
90             icon.set_pixel_size(base_icon_size + 8);
91             button.add_css_class("dock-item-hover");
92         });
93     }
94 }

```

```

94     {
95         let icon = icon.clone();
96         let button = button.clone();
97         motion.connect_leave(move |_| {
98             icon.set_pixel_size(base_icon_size);
99             button.remove_css_class("dock-item-hover");
100         });
101     }
102     button.add_controller(motion);
103
104     let app_info = app.clone();
105     button.connect_clicked(move |_| {
106         let open = list_toplevels(None);
107         if let Some(toplevel) = open.into_iter().find(|toplevel| {
108             toplevel.app_id.eq_ignore_ascii_case(&app_info.app_id)
109             || toplevel.app_id.eq_ignore_ascii_case(app_info.desktop_id.
110 trim_end_matches(".desktop"))
111         }) {
112             let _ = focus_toplevel(&toplevel);
113         } else if let Err(error) = app_info.launch() {
114             eprintln!("note-shell: no se pudo abrir {}: {error}", app_info.name);
115         }
116     });
117     (button, indicator)
118 }
119
120 fn dock_action_button(icon_name: &str, tooltip: &str, size: i32) -> gtk::Button {
121     let icon = gtk::Image::from_icon_name(icon_name);
122     icon.set_pixel_size((size - 18).max(24));
123     let button = gtk::Button::builder().child(&icon).build();
124     button.add_css_class("dock-item");
125     button.set_tooltip_text(Some(tooltip));
126     button.set_size_request(size, size + 6);
127     button
128 }
129
130 fn dock_separator() -> gtk::Separator {
131     let separator = gtk::Separator::new(gtk::Orientation::Vertical);
132     separator.add_css_class("dock-separator");
133     separator
134 }

```

D.7 Overview

```

1  ••
2  fn toggle_overview(app: &gtk::Application, state: &Rc<RefCell<ShellState>>) {
3      if let Some(window) = state.borrow().overview.as_ref() {
4          if window.is_visible() {
5              window.set_visible(false);
6          } else {
7              refresh_overview(window, state);
8              window.present();
9          }
10         return;
11     }
12
13     let window = create_overview(app, state);

```



```

14     window.present();
15     state.borrow_mut().overview = Some(window);
16 }
17
18 fn create_overview(app: &gtk::Application, state: &Rc<RefCell<ShellState>>) -> gtk::
    ApplicationWindow {
19     let settings = state.borrow().settings.clone();
20     let tr = Translator::new(settings.language.clone());
21     let window = gtk::ApplicationWindow::builder()
22         .application(app)
23         .title("Note Overview")
24         .decorated(false)
25         .build();
26     window.init_layer_shell();
27     window.set_layer(Layer::Overlay);
28     window.set_namespace(Some("note-overview"));
29     for edge in [Edge::Top, Edge::Bottom, Edge::Left, Edge::Right] {
30         window.set_anchor(edge, true);
31     }
32     window.set_keyboard_mode(KeyboardMode::Exclusive);
33
34     let root = gtk::Box::new(gtk::Orientation::Vertical, 22);
35     root.add_css_class("overview");
36     root.set_margin_top(56);
37     root.set_margin_bottom(82);
38     root.set_margin_start(72);
39     root.set_margin_end(72);
40
41     let search = gtk::SearchEntry::new();
42     search.set_placeholder_text(Some(&tr.text("search-placeholder")));
43     search.add_css_class("overview-search");
44     search.set_hexexpand(true);
45     root.append(&search);
46
47     let content = gtk::Paned::new(gtk::Orientation::Horizontal);
48     content.set_wide_handle(true);
49     content.set_resize_start_child(false);
50     content.set_shrink_start_child(false);
51
52     let windows_box = gtk::Box::new(gtk::Orientation::Vertical, 9);
53     windows_box.set_size_request(300, -1);
54     let windows_title = gtk::Label::new(Some(&tr.text("open-windows")));
55     windows_title.add_css_class("section-title");
56     windows_title.set_xalign(0.0);
57     windows_box.append(&windows_title);
58     let windows_list = gtk::Box::new(gtk::Orientation::Vertical, 6);
59     windows_list.set_widget_name("note-overview-window-list");
60     windows_box.append(&windows_list);
61     content.set_start_child(Some(&windows_box));
62
63     let apps_box = gtk::Box::new(gtk::Orientation::Vertical, 9);
64     let apps_title = gtk::Label::new(Some(&tr.text("applications")));
65     apps_title.add_css_class("section-title");
66     apps_title.set_xalign(0.0);
67     apps_box.append(&apps_title);
68     let scroll = gtk::ScrolledWindow::new();
69     scroll.set_hexexpand(true);
70     scroll.set_vexpand(true);
71     scroll.set_policy(gtk::PolicyType::Never, gtk::PolicyType::Automatic);

```

```

72 let grid = gtk::Grid::new();
73 grid.set_column_spacing(14);
74 grid.set_row_spacing(14);
75 grid.set_widget_name("note-overview-app-grid");
76 scroll.set_child(Some(&grid));
77 apps_box.append(&scroll);
78 content.set_end_child(Some(&apps_box));
79 content.set_position(330);
80 root.append(&content);
81
82 let workspaces = gtk::Box::new(gtk::Orientation::Horizontal, 8);
83 workspaces.set_halign(gtk::Align::Center);
84 for index in 1..=settings.workspaces.max(1) {
85     let button = gtk::Button::with_label(&index.to_string());
86     button.add_css_class("workspace-pill");
87     button.connect_clicked(move |_| {
88         spawn("note-workspace", &[&index.to_string()]);
89     });
90     workspaces.append(&button);
91 }
92 root.append(&workspaces);
93 window.set_child(Some(&root));
94
95 let apps = state.borrow().apps.clone();
96 rebuild_app_grid(&grid, &apps, "", &window);
97 rebuild_window_list(&windows_list, &tr, &window);
98 {
99     let grid = grid.clone();
100     let apps = apps.clone();
101     let window = window.clone();
102     search.connect_search_changed(move |entry| {
103         let query = entry.text().to_string();
104         rebuild_app_grid(&grid, &apps, &query, &window);
105     });
106 }
107
108 let controller = gtk::EventControllerKey::new();
109 {
110     let window = window.clone();
111     controller.connect_key_pressed(move |_, key, _, _| {
112         if key == gdk::Key::Escape {
113             window.set_visible(false);
114             return glib::Propagation::Stop;
115         }
116         glib::Propagation::Proceed
117     });
118 }
119 window.add_controller(controller);
120 window
121 }
122
123 fn refresh_overview(window: &gtk::ApplicationWindow, state: &Rc<RefCell<ShellState>>) {
124     let Some(root) = window.child() else { return };
125     if let Some(windows_list) = find_widget_by_name(&root, "note-overview-window-list")
126         .and_then(|widget| widget.downcast::<gtk::Box>().ok())
127     {
128         let tr = Translator::new(state.borrow().settings.language.clone());
129         rebuild_window_list(&windows_list, &tr, window);
130     }

```

```

131 }
132
133 fn rebuild_window_list(list: &gtk::Box, tr: &Translator, overview: &gtk::
  ApplicationWindow) {
134   while let Some(child) = list.first_child() {
135     list.remove(&child);
136   }
137   let windows = list_toplevels(None);
138   if windows.is_empty() {
139     let empty = gtk::Label::new(Some(&tr.text("no-open-windows")));
140     empty.add_css_class("muted");
141     empty.set_wrap(true);
142     list.append(&empty);
143     return;
144   }
145   for toplevel in windows {
146     if toplevel.app_id.contains("note-shell") {
147       continue;
148     }
149     let row = gtk::Button::new();
150     row.add_css_class("window-card");
151     let content = gtk::Box::new(gtk::Orientation::Horizontal, 10);
152     let icon = gtk::Image::from_icon_name("application-x-executable-symbolic");
153     icon.set_pixel_size(28);
154     content.append(&icon);
155     let text = gtk::Box::new(gtk::Orientation::Vertical, 2);
156     let title = gtk::Label::new(Some(&toplevel.title));
157     title.set_xalign(0.0);
158     title.set_ellipsize(gtk::pango::EllipsizeMode::End);
159     title.add_css_class("window-title");
160     text.append(&title);
161     let app_id = gtk::Label::new(Some(&toplevel.app_id));
162     app_id.set_xalign(0.0);
163     app_id.add_css_class("muted");
164     text.append(&app_id);
165     content.append(&text);
166     row.set_child(Some(&content));
167     let target = toplevel.clone();
168     let overview = overview.clone();
169     row.connect_clicked(move |_| {
170       let _ = focus_toplevel(&target);
171       overview.set_visible(false);
172     });
173     list.append(&row);
174   }
175 }
176
177 fn rebuild_app_grid(
178   grid: &gtk::Grid,
179   apps: &[DesktopApp],
180   query: &str,
181   overview: &gtk::ApplicationWindow,
182 ) {
183   while let Some(child) = grid.first_child() {
184     grid.remove(&child);
185   }
186   let query = query.trim().to_lowercase();
187   let filtered = apps.iter().filter(|app| {
188     query.is_empty()

```

```

189         || app.name.to_lowercase().contains(&query)
190         || app.generic_name.as_deref().unwrap_or_default().to_lowercase().contains(&
query)
191         || app.categories.iter().any(|category| category.to_lowercase().contains(&
query))
192     });
193     for (position, app) in filtered.take(80).enumerate() {
194         let button = gtk::Button::new();
195         button.add_css_class("app-tile");
196         let content = gtk::Box::new(gtk::Orientation::Vertical, 8);
197         content.set_halign(gtk::Align::Center);
198         let icon = gtk::Image::from_icon_name(&app.icon);
199         icon.set_pixel_size(42);
200         content.append(&icon);
201         let label = gtk::Label::new(Some(&app.name));
202         label.set_max_width_chars(15);
203         label.set_ellipsize(gtk::pango::EllipsizeMode::End);
204         content.append(&label);
205         button.set_child(Some(&content));
206         let app_info = app.clone();
207         let overview = overview.clone();
208         button.connect_clicked(move |_| {
209             if let Err(error) = app_info.launch() {
210                 eprintln!("note-shell: no se pudo abrir {}: {error}", app_info.name);
211             }
212             overview.set_visible(false);
213         });
214         grid.attach(&button, (position % 5) as i32, (position / 5) as i32, 1, 1);
215     }
216 }

```

D.8 Centro de control

```

1  ••
2  fn toggle_control_center(app: &gtk::Application, state: &Rc<RefCell<ShellState>>) {
3      if let Some(window) = state.borrow().control_center.as_ref() {
4          if window.is_visible() { window.set_visible(false); } else { window.present(); }
5          return;
6      }
7      let window = create_control_center(app, &state.borrow().settings);
8      window.present();
9      state.borrow_mut().control_center = Some(window);
10 }
11
12 fn create_control_center(app: &gtk::Application, settings: &NoteSettings) -> gtk::
ApplicationWindow {
13     let tr = Translator::new(settings.language.clone());
14     let window = gtk::ApplicationWindow::builder()
15         .application(app)
16         .title("Note Control Center")
17         .decorated(false)
18         .resizable(false)
19         .default_width(360)
20         .build();
21     window.init_layer_shell();
22     window.set_layer(Layer::Overlay);
23     window.set_namespace(Some("note-control-center"));

```

```

24 window.set_anchor(Edge::Top, true);
25 window.set_anchor(Edge::Right, true);
26 window.set_margin(Edge::Top, 42);
27 window.set_margin(Edge::Right, 12);
28 window.set_keyboard_mode(KeyboardMode::OnDemand);
29
30 let root = gtk::Box::new(gtk::Orientation::Vertical, 14);
31 root.add_css_class("control-center");
32
33 let header = gtk::Box::new(gtk::Orientation::Horizontal, 10);
34 let user_icon = gtk::Image::from_icon_name("avatar-default-symbolic");
35 user_icon.set_pixel_size(34);
36 header.append(&user_icon);
37 let username = std::env::var("USER").unwrap_or_else(|_| tr.text("user"));
38 let user_label = gtk::Label::new(Some(&username));
39 user_label.add_css_class("control-title");
40 user_label.set_hexpand(true);
41 user_label.set_xalign(0.0);
42 header.append(&user_label);
43 let settings_button = icon_button("preferences-system-symbolic", &tr.text("settings"
44 ));
45 settings_button.connect_clicked(|_| spawn("note-settings", &[]));
46 header.append(&settings_button);
47 root.append(&header);
48
49 let quick = gtk::Grid::new();
50 quick.set_column_homogeneous(true);
51 quick.set_column_spacing(10);
52 quick.set_row_spacing(10);
53 quick.attach(&toggle_tile(
54     "network-wireless-symbolic", &tr.text("wifi"), wifi_enabled(), set_wifi,
55     ), 0, 0, 1, 1);
56 quick.attach(&toggle_tile(
57     "bluetooth-active-symbolic", &tr.text("bluetooth"), bluetooth_enabled(),
58     set_bluetooth,
59     ), 1, 0, 1, 1);
60 let network = action_tile("network-vpn-symbolic", &tr.text("network-settings"));
61 network.connect_clicked(|_| spawn("nm-connection-editor", &[]));
62 quick.attach(&network, 0, 1, 1, 1);
63 let displays = action_tile("video-display-symbolic", &tr.text("display-settings"));
64 displays.connect_clicked(|_| spawn_first(&["wdisplays", "arandr"]));
65 quick.attach(&displays, 1, 1, 1, 1);
66 root.append(&quick);
67
68 root.append(&slider_row(
69     "audio-volume-high-symbolic",
70     &tr.text("volume"),
71     volume_percent() as f64,
72     0.0,
73     150.0,
74     set_volume,
75 ));
76 if let Some(brightness) = brightness_percent() {
77     root.append(&slider_row(
78         "display-brightness-symbolic",
79         &tr.text("brightness"),
80         brightness as f64,
81         1.0,
82         100.0,

```

```

81         set_brightness,
82     ));
83 }
84
85 let actions = gtk::Box::new(gtk::Orientation::Horizontal, 8);
86 actions.set_homogeneous(true);
87 for (icon, title, command) in [
88     ("system-lock-screen-symbolic", tr.text("lock"), "note-lock"),
89     ("system-log-out-symbolic", tr.text("logout"), "note-logout"),
90     ("system-reboot-symbolic", tr.text("restart"), "note-reboot"),
91     ("system-shutdown-symbolic", tr.text("power-off"), "note-poweroff"),
92 ] {
93     let button = icon_button(icon, &title);
94     button.add_css_class("power-button");
95     button.connect_clicked(move |_| spawn(command, &[]));
96     actions.append(&button);
97 }
98 root.append(&actions);
99 window.set_child(Some(&root));
100
101 let controller = gtk::EventControllerKey::new();
102 {
103     let window = window.clone();
104     controller.connect_key_pressed(move |_, key, _, _| {
105         if key == gdk::Key::Escape {
106             window.set_visible(false);
107             return glib::Propagation::Stop;
108         }
109         glib::Propagation::Proceed
110     });
111 }
112 window.add_controller(controller);
113 window
114 }
115
116 fn toggle_tile(
117     icon_name: &str,
118     title: &str,
119     active: bool,
120     callback: fn(bool),
121 ) -> gtk::Box {
122     let tile = gtk::Box::new(gtk::Orientation::Horizontal, 8);
123     tile.add_css_class("quick-tile");
124     let icon = gtk::Image::from_icon_name(icon_name);
125     icon.set_pixel_size(20);
126     tile.append(&icon);
127     let label = gtk::Label::new(Some(title));
128     label.set_hexpand(true);
129     label.set_xalign(0.0);
130     tile.append(&label);
131     let switch = gtk::Switch::new();
132     switch.set_active(active);
133     switch.connect_state_set(move |_, state| {
134         callback(*state);
135         glib::Propagation::Proceed
136     });
137     tile.append(&switch);
138     tile
139 }

```

```

140
141 fn action_tile(icon_name: &str, title: &str) -> gtk::Button {
142     let content = gtk::Box::new(gtk::Orientation::Horizontal, 8);
143     let icon = gtk::Image::from_icon_name(icon_name);
144     icon.set_pixel_size(20);
145     content.append(&icon);
146     let label = gtk::Label::new(Some(title));
147     label.set_xalign(0.0);
148     content.append(&label);
149     let button = gtk::Button::builder().child(&content).build();
150     button.add_css_class("quick-tile");
151     button
152 }
153
154 fn slider_row(
155     icon_name: &str,
156     title: &str,
157     value: f64,
158     min: f64,
159     max: f64,
160     callback: fn(f64),
161 ) -> gtk::Box {
162     let row = gtk::Box::new(gtk::Orientation::Horizontal, 10);
163     row.add_css_class("slider-row");
164     let icon = gtk::Image::from_icon_name(icon_name);
165     icon.set_pixel_size(20);
166     icon.set_tooltip_text(Some(title));
167     row.append(&icon);
168     let scale = gtk::Scale::with_range(gtk::Orientation::Horizontal, min, max, 1.0);
169     scale.set_value(value);
170     scale.set_hexpand(true);
171     scale.set_draw_value(false);
172     scale.connect_value_changed(move |scale| callback(scale.value()));
173     row.append(&scale);
174     row
175 }

```

D.9 Configuración layer-shell y actualización de estado

```

1 ••
2 fn icon_button(icon_name: &str, tooltip: &str) -> gtk::Button {
3     let icon = gtk::Image::from_icon_name(icon_name);
4     icon.set_pixel_size(18);
5     let button = gtk::Button::builder().child(&icon).tooltip_text(tooltip).build();
6     button.add_css_class("icon-button");
7     button
8 }
9
10 fn configure_layer_window(
11     window: &gtk::ApplicationWindow,
12     monitor: Option<&gdk::Monitor>,
13     layer: Layer,
14     namespace: &str,
15 ) {
16     window.init_layer_shell();
17     window.set_layer(layer);
18     window.set_namespace(Some(namespace));

```

```

19     if let Some(monitor) = monitor {
20         window.set_monitor(Some(monitor));
21     }
22 }
23
24 fn update_clock(label: &gtk::Label) {
25     label.set_text(&Local::now().format("%a %d %b   %H:%M").to_string());
26 }
27
28 fn update_active_application(label: &gtk::Label, tr: &Translator) {
29     let active = list_toplevels(Some("state:active"));
30     if let Some(window) = active.first() {
31         label.set_text(&window.app_id);
32     } else {
33         label.set_text(&tr.text("desktop"));
34     }
35 }
36
37 fn update_running_indicators(indicators: &[(String, gtk::Label)]) {
38     let open = list_toplevels(None);
39     for (app_id, indicator) in indicators {
40         let running = open.iter().any(|window| window.app_id.eq_ignore_ascii_case(app_id
41 ));
42         indicator.set_opacity(if running { 1.0 } else { 0.0 });
43     }
44 }
45
46 fn update_battery(label: &gtk::Label) {
47     let capacity = std::fs::read_dir("/sys/class/power_supply")
48         .ok()
49         .into_iter()
50         .flatten()
51         .flatten()
52         .find_map(|entry| {
53             if !entry.file_name().to_string_lossy().starts_with("BAT") { return None; }
54             std::fs::read_to_string(entry.path().join("capacity")).ok()
55         })
56         .and_then(|value| value.trim().parse::<u8>().ok());
57     match capacity {
58         Some(value) => { label.set_text(&format!("{value}%")); label.set_visible(true); }
59         None => label.set_visible(false),
60     }
61 }
62
63 fn install_css(settings: &NoteSettings) {
64     let provider = gtk::CssProvider::new();
65     let installed = "/usr/share/note-desktop/style/shell.css";
66     let base = std::fs::read_to_string(installed).unwrap_or_else(|_| FALLBACK_CSS.to_owned());
67     provider.load_from_data(&(settings.css_variables() + &base));
68     if let Some(display) = gdk::Display::default() {
69         gtk::style_context_add_provider_for_display(
70             &display,
71             &provider,
72             gtk::STYLE_PROVIDER_PRIORITY_APPLICATION,
73         );
74     }
75 }

```



```

75
76 fn apply_gtk_preferences(settings: &NoteSettings) {
77     if let Some(gtk_settings) = gtk::Settings::default() {
78         let dark = match settings.appearance {
79             note_core::AppearanceMode::Light => false,
80             note_core::AppearanceMode::Dark => true,
81             note_core::AppearanceMode::Auto => {
82                 let hour = Local::now().hour();
83                 !(7..19).contains(&hour)
84             }
85         };
86         gtk_settings.set_property("gtk-application-prefer-dark-theme", dark);
87         gtk_settings.set_property("gtk-enable-animations", settings.animations);
88     }
89 }
90
91 fn find_widget_by_name(root: &gtk::Widget, name: &str) -> Option<gtk::Widget> {
92     if root.widget_name() == name { return Some(root.clone()); }
93     let mut child = root.first_child();
94     while let Some(widget) = child {
95         if let Some(found) = find_widget_by_name(&widget, name) { return Some(found); }
96         child = widget.next_sibling();
97     }
98     None
99 }
100
101 fn spawn(command: &str, args: &[&str]) {
102     if let Err(error) = Command::new(command)
103         .args(args)
104         .stdin(Stdio::null())
105         .stdout(Stdio::null())
106         .stderr(Stdio::null())
107         .spawn()
108     {
109         eprintln!("note-shell: no se pudo ejecutar {command}: {error}");
110     }
111 }
112
113 fn spawn_first(commands: &[&str]) {
114     if let Some(command) = commands.iter().find(|command| command_exists(command)) {
115         spawn(command, &[]);
116     }
117 }
118
119 fn command_exists(command: &str) -> bool {
120     if command.contains('/') { return Path::new(command).exists(); }
121     Command::new("sh")
122         .args(["-lc", &format!("command -v {} >/dev/null 2>&1", command)])
123         .status()
124         .map(|status| status.success())
125         .unwrap_or(false)
126 }

```

D.10 Aplicación note-settings completa

```

1 use gtk::{gdk, glib, prelude::*};
2 use note_core::{AppearanceMode, NoteSettings, Translator};

```

```

3 use std::process::{Command, Stdio};
4
5 const APP_ID: &str = "mx.note.desktop.settings";
6 const FALLBACK_CSS: &str = include_str!("../../assets/style/settings.css");
7
8 fn main() -> glib::ExitCode {
9     let app = gtk::Application::builder().application_id(APP_ID).build();
10    app.connect_startup(|_| install_css());
11    app.connect_activate(build_ui);
12    app.run()
13 }
14
15 fn build_ui(app: &gtk::Application) {
16     if let Some(window) = app.active_window() {
17         window.present();
18         return;
19     }
20
21     let settings = NoteSettings::load();
22     let tr = Translator::new(settings.language.clone());
23     let window = gtk::ApplicationWindow::builder()
24         .application(app)
25         .title(tr.text("settings-title"))
26         .default_width(980)
27         .default_height(680)
28         .build();
29     window.add_css_class("settings-window");
30
31     let root = gtk::Box::new(gtk::Orientation::Vertical, 0);
32     let header = gtk::HeaderBar::new();
33     header.set_title_widget(Some(&gtk::Label::new(Some(&tr.text("settings-title")))));
34     root.append(&header);
35
36     let body = gtk::Box::new(gtk::Orientation::Horizontal, 0);
37     body.set_vexpand(true);
38     let sidebar = gtk::ListBox::new();
39     sidebar.add_css_class("settings-sidebar");
40     sidebar.set_selection_mode(gtk::SelectionMode::Single);
41     sidebar.set_size_request(230, -1);
42
43     let stack = gtk::Stack::new();
44     stack.set_hexpand(true);
45     stack.set_vexpand(true);
46     stack.set_transition_type(gtk::StackTransitionType::Crossfade);
47
48     let controls = build_appearance_page(&stack, &tr, &settings);
49     add_page(&sidebar, &stack, "appearance", "preferences-desktop-theme-symbolic", &tr.
50     text("appearance"), controls.page.clone());
51     add_page(&sidebar, &stack, "language", "preferences-desktop-locale-symbolic", &tr.
52     text("language-region"), build_language_page(&tr));
53     add_page(&sidebar, &stack, "displays", "video-display-symbolic", &tr.text("displays"
54     ), build_launcher_page(&tr, "video-display-symbolic", "display-description", &["
55     wdisplays", "arandr"]));
56     add_page(&sidebar, &stack, "network", "network-wireless-symbolic", &tr.text("network
57     "), build_launcher_page(&tr, "network-wireless-symbolic", "network-description", &["
58     nm-connection-editor"]));
59     add_page(&sidebar, &stack, "bluetooth", "bluetooth-active-symbolic", &tr.text("
60     bluetooth"), build_launcher_page(&tr, "bluetooth-active-symbolic", "bluetooth-
61     description", &["blueman-manager"]));

```

```

54 add_page(&sidebar, &stack, "sound", "audio-volume-high-symbolic", &tr.text("sound"),
    build_launcher_page(&tr, "audio-volume-high-symbolic", "sound-description", &["
pavucontrol"])));
55 add_page(&sidebar, &stack, "power", "battery-good-symbolic", &tr.text("power"),
    build_info_page(&tr, "battery-good-symbolic", "power-description"));
56 add_page(&sidebar, &stack, "keyboard", "input-keyboard-symbolic", &tr.text("keyboard
"), build_info_page(&tr, "input-keyboard-symbolic", "keyboard-description"));
57 add_page(&sidebar, &stack, "privacy", "security-high-symbolic", &tr.text("privacy"),
    build_info_page(&tr, "security-high-symbolic", "privacy-description"));
58 add_page(&sidebar, &stack, "about", "help-about-symbolic", &tr.text("about"),
    build_about_page(&tr));
59
60 let stack_clone = stack.clone();
61 sidebar.connect_row_selected(move |_, row| {
62     let Some(row) = row else { return };
63     let name = row.widget_name();
64     stack_clone.set_visible_child_name(name.as_str());
65 });
66 if let Some(row) = sidebar.row_at_index(0) {
67     sidebar.select_row(Some(&row));
68 }
69
70 body.append(&sidebar);
71 body.append(&stack);
72 root.append(&body);
73
74 let action_bar = gtk::ActionBar::new();
75 let status = gtk::Label::new(None);
76 status.add_css_class("save-status");
77 action_bar.set_center_widget(Some(&status));
78 let reset = gtk::Button::with_label(&tr.text("restore-defaults"));
79 let save = gtk::Button::with_label(&tr.text("apply"));
80 save.add_css_class("suggested-action");
81 action_bar.pack_start(&reset);
82 action_bar.pack_end(&save);
83 root.append(&action_bar);
84
85 {
86     let controls = controls.clone();
87     let status = status.clone();
88     let tr = tr.clone();
89     save.connect_clicked(move |_| {
90         let mut new_settings = NoteSettings::load();
91         new_settings.appearance = match controls.appearance.selected() {
92             1 => AppearanceMode::Light,
93             2 => AppearanceMode::Dark,
94             _ => AppearanceMode::Auto,
95         };
96         new_settings.accent = selected_string(&controls.accent).unwrap_or_else(|| "
blue".into());
97         new_settings.language = language_code(controls.language.selected()).to_owned
98         ();
99         new_settings.dock_size = controls.dock_size.value().round() as i32;
100         new_settings.panel_opacity = controls.panel_opacity.value();
101         new_settings.dock_opacity = controls.dock_opacity.value();
102         new_settings.animations = controls.animations.is_active();
103         new_settings.natural_scroll = controls.natural_scroll.is_active();
104         new_settings.workspaces = controls.workspaces.value() as u8;

```

```

105         match new_settings.save().and_then(|_| new_settings.write_locale_environment
106         ()) {
107             Ok(()) => {
108                 status.set_text(&tr.text("settings-saved"));
109                 spawn("note-apply-settings", &[]);
110                 spawn("gapplication", &["action", "mx.note.desktop.shell", "reload"
111             ]);
112             }
113             Err(error) => status.set_text(&format!("{}", tr.text("settings-
114         error"))),
115         }
116     });
117 }
118 {
119     let controls = controls.clone();
120     reset.connect_clicked(move |_| controls.apply(&NoteSettings::default()));
121 }
122 window.set_child(Some(&root));
123 window.present();
124 }
125
126 #[derive(Clone)]
127 struct AppearanceControls {
128     page: gtk::Widget,
129     appearance: gtk::DropDown,
130     accent: gtk::DropDown,
131     language: gtk::DropDown,
132     dock_size: gtk::Scale,
133     panel_opacity: gtk::Scale,
134     dock_opacity: gtk::Scale,
135     animations: gtk::Switch,
136     natural_scroll: gtk::Switch,
137     workspaces: gtk::SpinButton,
138 }
139
140 impl AppearanceControls {
141     fn apply(&self, settings: &NoteSettings) {
142         self.appearance.set_selected(match settings.appearance {
143             AppearanceMode::Auto => 0,
144             AppearanceMode::Light => 1,
145             AppearanceMode::Dark => 2,
146         });
147         let accents = ["blue", "purple", "pink", "red", "orange", "green", "teal"];
148         let accent_index = accents.iter().position(|value| *value == settings.accent).
149         unwrap_or(0);
150         self.accent.set_selected(accent_index as u32);
151         let locales = ["es-MX", "en-US", "pt-BR", "fr-FR", "de-DE"];
152         let language_index = locales.iter().position(|value| *value == settings.language
153         ).unwrap_or(0);
154         self.language.set_selected(language_index as u32);
155         self.dock_size.set_value(settings.dock_size as f64);
156         self.panel_opacity.set_value(settings.panel_opacity);
157         self.dock_opacity.set_value(settings.dock_opacity);
158         self.animations.set_active(settings.animations);
159         self.natural_scroll.set_active(settings.natural_scroll);
160         self.workspaces.set_value(settings.workspaces as f64);
161     }
162 }

```

```

159 }
160
161 fn build_appearance_page(stack: &gtk::Stack, tr: &Translator, settings: &NoteSettings)
162   -> AppearanceControls {
163   let page = page_container(&tr.text("appearance"), &tr.text("appearance-description")
164   );
165   let group = settings_group();
166
167   let appearance = dropdown(&[&tr.text("automatic"), &tr.text("light"), &tr.text("dark
168   ")]);
169   group.append(&setting_row(&tr.text("color-mode"), &tr.text("color-mode-description")
170   , &appearance));
171
172   let accent = dropdown(&["blue", "purple", "pink", "red", "orange", "green", "teal"])
173   ;
174   group.append(&setting_row(&tr.text("accent-color"), &tr.text("accent-color-
175   description"), &accent));
176
177   let language = dropdown(&["Español (México)", "English (US)", "Português (Brasil)",
178   "Français", "Deutsch"]);
179   group.append(&setting_row(&tr.text("interface-language"), &tr.text("language-relogin
180   "), &language));
181
182   let dock_size = scale(38.0, 72.0, settings.dock_size as f64);
183   group.append(&setting_row(&tr.text("dock-size"), &tr.text("dock-size-description"),
184   &dock_size));
185
186   let panel_opacity = scale(0.35, 1.0, settings.panel_opacity);
187   panel_opacity.set_digits(2);
188   group.append(&setting_row(&tr.text("panel-opacity"), &tr.text("opacity-description")
189   , &panel_opacity));
190
191   let dock_opacity = scale(0.35, 1.0, settings.dock_opacity);
192   dock_opacity.set_digits(2);
193   group.append(&setting_row(&tr.text("dock-opacity"), &tr.text("opacity-description"),
194   &dock_opacity));
195
196   let animations = gtk::Switch::new();
197   group.append(&setting_row(&tr.text("animations"), &tr.text("animations-description")
198   , &animations));
199
200   let natural_scroll = gtk::Switch::new();
201   group.append(&setting_row(&tr.text("natural-scroll"), &tr.text("natural-scroll-
202   description"), &natural_scroll));
203
204   let workspaces = gtk::SpinButton::with_range(1.0, 9.0, 1.0);
205   group.append(&setting_row(&tr.text("workspaces"), &tr.text("workspaces-description")
206   , &workspaces));
207
208   if let Ok(page_box) = page.clone().downcast::<gtk::Box>() {
209     page_box.append(&group);
210   }
211   stack.add_named(&page, Some("appearance"));
212
213   let controls = AppearanceControls {
214     page,
215     appearance,
216     accent,
217     language,

```

```

204     dock_size,
205     panel_opacity,
206     dock_opacity,
207     animations,
208     natural_scroll,
209     workspaces,
210 };
211 controls.apply(settings);
212 controls
213 }
214
215 fn build_language_page(tr: &Translator) -> gtk::Widget {
216     let page = page_container(&tr.text("language-region"), &tr.text("language-
description"));
217     let group = settings_group();
218     group.append(&info_row(&tr.text("current-language"), tr.locale()));
219     group.append(&info_row(&tr.text("locale-file"), "~/config/environment.d/90-note-
locale.conf"));
220     group.append(&info_row(&tr.text("available-languages"), "es-MX · en-US · pt-BR · fr-
FR · de-DE"));
221     if let Ok(page_box) = page.clone().downcast::<gtk::Box>() { page_box.append(&group);
    }
222     page
223 }
224
225 fn build_launcher_page(tr: &Translator, icon: &str, description_key: &str, commands: &
static [&static str]) -> gtk::Widget {
226     let page = build_info_page(tr, icon, description_key);
227     if let Ok(page_box) = page.clone().downcast::<gtk::Box>() {
228         let button = gtk::Button::with_label(&tr.text("open-configuration"));
229         button.add_css_class("suggested-action");
230         button.set_halign(gtk::Align::Start);
231         button.connect_clicked(move |_| spawn_first(commands));
232         page_box.append(&button);
233     }
234     page
235 }
236
237 fn build_info_page(tr: &Translator, icon: &str, description_key: &str) -> gtk::Widget {
238     let page = page_container(&tr.text(description_key.trim_end_matches("-description"))
, &tr.text(description_key));
239     if let Ok(page_box) = page.clone().downcast::<gtk::Box>() {
240         let image = gtk::Image::from_icon_name(icon);
241         image.set_pixel_size(96);
242         image.add_css_class("page-hero-icon");
243         page_box.append(&image);
244     }
245     page
246 }
247
248 fn build_about_page(tr: &Translator) -> gtk::Widget {
249     let page = page_container(&tr.text("about"), &tr.text("about-description"));
250     if let Ok(page_box) = page.clone().downcast::<gtk::Box>() {
251         let logo = gtk::Image::from_icon_name("note-desktop-symbolic");
252         logo.set_pixel_size(96);
253         page_box.append(&logo);
254         let version = gtk::Label::new(Some(&format!("Note Desktop {} ", env!("
CARGO_PKG_VERSION"))));
255         version.add_css_class("about-version");

```

```

256     page_box.append(&version);
257     let doctor = gtk::Button::with_label(&tr.text("run-diagnostics"));
258     doctor.connect_clicked(|_| spawn("note-terminal", &["-e", "note-doctor"]));
259     page_box.append(&doctor);
260 }
261 page
262 }
263
264 fn add_page(sidebar: &gtk::ListBox, stack: &gtk::Stack, name: &str, icon: &str, title: &
    str, page: gtk::Widget) {
265     if stack.child_by_name(name).is_none() {
266         stack.add_named(&page, Some(name));
267     }
268     let row = gtk::ListBoxRow::new();
269     row.set_widget_name(name);
270     let content = gtk::Box::new(gtk::Orientation::Horizontal, 12);
271     content.set_margin_top(10);
272     content.set_margin_bottom(10);
273     content.set_margin_start(14);
274     content.set_margin_end(14);
275     let image = gtk::Image::from_icon_name(icon);
276     image.set_pixel_size(20);
277     content.append(&image);
278     let label = gtk::Label::new(Some(title));
279     label.set_xalign(0.0);
280     content.append(&label);
281     row.set_child(Some(&content));
282     sidebar.append(&row);
283 }
284
285 fn page_container(title: &str, description: &str) -> gtk::Widget {
286     let outer = gtk::Box::new(gtk::Orientation::Vertical, 12);
287     outer.add_css_class("settings-page");
288     outer.set_margin_top(32);
289     outer.set_margin_bottom(32);
290     outer.set_margin_start(42);
291     outer.set_margin_end(42);
292     let title_label = gtk::Label::new(Some(title));
293     title_label.add_css_class("page-title");
294     title_label.set_xalign(0.0);
295     outer.append(&title_label);
296     let description_label = gtk::Label::new(Some(description));
297     description_label.add_css_class("page-description");
298     description_label.set_xalign(0.0);
299     description_label.set_wrap(true);
300     outer.append(&description_label);
301     outer.upcast()
302 }
303
304 fn settings_group() -> gtk::Box {
305     let group = gtk::Box::new(gtk::Orientation::Vertical, 0);
306     group.add_css_class("settings-group");
307     group
308 }
309
310 fn setting_row<T: IsA<gtk::Widget>>(title: &str, description: &str, control: &T) -> gtk
    ::Box {
311     let row = gtk::Box::new(gtk::Orientation::Horizontal, 18);
312     row.add_css_class("setting-row");

```

```

313     let text = gtk::Box::new(gtk::Orientation::Vertical, 3);
314     text.set_hexpand(true);
315     let title = gtk::Label::new(Some(title));
316     title.set_xalign(0.0);
317     title.add_css_class("setting-title");
318     text.append(&title);
319     let description = gtk::Label::new(Some(description));
320     description.set_xalign(0.0);
321     description.set_wrap(true);
322     description.add_css_class("setting-description");
323     text.append(&description);
324     row.append(&text);
325     control.set_valign(gtk::Align::Center);
326     row.append(control);
327     row
328 }
329
330 fn info_row(title: &str, value: &str) -> gtk::Box {
331     let value_label = gtk::Label::new(Some(value));
332     value_label.add_css_class("setting-description");
333     setting_row(title, "", &value_label)
334 }
335
336 fn dropdown(values: &[&str]) -> gtk::DropDown {
337     let list = gtk::StringList::new(values);
338     gtk::DropDown::builder().model(&list).build()
339 }
340
341 fn scale(min: f64, max: f64, value: f64) -> gtk::Scale {
342     let scale = gtk::Scale::with_range(gtk::Orientation::Horizontal, min, max, (max -
343     min) / 100.0);
344     scale.set_value(value);
345     scale.set_size_request(230, -1);
346     scale
347 }
348
349 fn selected_string(dropdown: &gtk::DropDown) -> Option<String> {
350     dropdown
351         .selected_item()?
352         .downcast::<gtk::StringObject>()
353         .ok()
354         .map(|value| value.string().to_string())
355 }
356
357 fn language_code(index: u32) -> &'static str {
358     match index {
359         1 => "en-US",
360         2 => "pt-BR",
361         3 => "fr-FR",
362         4 => "de-DE",
363         _ => "es-MX",
364     }
365 }
366
367 fn install_css() {
368     let provider = gtk::CssProvider::new();
369     let css = std::fs::read_to_string("/usr/share/note-desktop/style/settings.css")
370         .unwrap_or_else(|_| FALLBACK_CSS.to_owned());
371     provider.load_from_data(&css);

```



```

371     if let Some(display) = gdk::Display::default() {
372         gtk::style_context_add_provider_for_display(&display, &provider, gtk::
STYLE_PROVIDER_PRIORITY_APPLICATION);
373     }
374 }
375
376 fn spawn(command: &str, args: &[&str]) {
377     let _ = Command::new(command)
378         .args(args)
379         .stdin(Stdio::null())
380         .stdout(Stdio::null())
381         .stderr(Stdio::null())
382         .spawn();
383 }
384
385 fn spawn_first(commands: &[&str]) {
386     for command in commands {
387         let exists = Command::new("sh")
388             .args(["-lc", &format!("command -v {command} >/dev/null 2>&1")])
389             .status()
390             .map(|status| status.success())
391             .unwrap_or(false);
392         if exists {
393             spawn(command, &[]);
394             break;
395         }
396     }
397 }

```

D.11 Sesión

```

1  #!/bin/sh
2  set -eu
3
4  export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
5  export XDG_SESSION_TYPE=wayland
6  export XDG_SESSION_DESKTOP=note
7  export XDG_CURRENT_DESKTOP=Note:labwc:wlroots
8  export XDG_CONFIG_DIRS="/etc/xdg/note${XDG_CONFIG_DIRS:+:$XDG_CONFIG_DIRS}/etc/xdg"
9  export GTK_USE_PORTAL=1
10 export MOZ_ENABLE_WAYLAND=1
11 export QT_QPA_PLATFORM='wayland;xcb'
12 export SDL_VIDEODRIVER='wayland,x11'
13 export ELECTRON_OZONE_PLATFORM_HINT=auto
14 export _JAVA_AWT_WM_NONREParenting=1
15 export LABWC_UPDATE_ACTIVATION_ENV=1
16
17 if [ -r "$HOME/.config/environment.d/90-note-locale.conf" ]; then
18     set -a
19     . "$HOME/.config/environment.d/90-note-locale.conf"
20     set +a
21 fi
22 [ -r /etc/note-desktop/gpu.conf ] && . /etc/note-desktop/gpu.conf
23 [ -r "$HOME/.config/note-desktop/gpu.conf" ] && . "$HOME/.config/note-desktop/gpu.conf"
24
25 if [ -S "${XDG_RUNTIME_DIR:-/run/user/${id -u}}/bus" ]; then
26     export DBUS_SESSION_BUS_ADDRESS="unix:path=${XDG_RUNTIME_DIR:-/run/user/${id -u}}/bus"

```

```

27 exec /usr/libexec/note-desktop/note-session-inner
28 fi
29 exec dbus-run-session -- /usr/libexec/note-desktop/note-session-inner

```

```

1 #!/bin/sh
2 set -u
3
4 export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
5
6 dbus-update-activation-environment --systemd \
7     DBUS_SESSION_BUS_ADDRESS DISPLAY WAYLAND_DISPLAY \
8     XDG_CURRENT_DESKTOP XDG_SESSION_DESKTOP XDG_SESSION_TYPE \
9     XDG_RUNTIME_DIR LANG LANGUAGE LC_MESSAGES 2>/dev/null || true
10
11 systemctl --user import-environment \
12     DBUS_SESSION_BUS_ADDRESS DISPLAY WAYLAND_DISPLAY \
13     XDG_CURRENT_DESKTOP XDG_SESSION_DESKTOP XDG_SESSION_TYPE \
14     XDG_RUNTIME_DIR LANG LANGUAGE LC_MESSAGES 2>/dev/null || true
15
16 start_time=$(date +%s)
17 labwc --merge-config
18 status=$?
19 elapsed=$(( $(date +%s) - start_time ))
20
21 # A normal logout usually happens after the session has been alive for a while.
22 # Only retry with software rendering after an immediate graphics failure.
23 if [ "$status" -ne 0 ] && [ "$elapsed" -lt 12 ] && [ "${WLR_RENDERER:-}" != "pixmap" ];
24 then
25     echo "note-session: accelerated renderer failed; retrying with pixmap" >&2
26     export WLR_RENDERER=pixmap
27     export WLR_NO_HARDWARE_CURSORS=1
28     exec labwc --merge-config
29 fi
30 exit "$status"

```

```

1 #!/bin/sh
2 set -u
3
4 pkill -x swaybg 2>/dev/null || true
5 swaybg -i /usr/share/note-desktop/wallpapers/note-wave.svg -m fill >/dev/null 2>&1 &
6
7 systemctl --user import-environment WAYLAND_DISPLAY DISPLAY XDG_CURRENT_DESKTOP
8     XDG_SESSION_TYPE 2>/dev/null || true
9 systemctl --user --no-block start graphical-session.target note-session.target 2>/dev/
10     null || {
11     # Emergency fallback when the user systemd instance is unavailable.
12     mako >/dev/null 2>&1 &
13     lxdm >/dev/null 2>&1 &
14     note-shell >/dev/null 2>&1 &
15 }

```

D.12 Aplicación de preferencias a labwc

```

1 #!/bin/sh
2 set -eu
3 settings="${XDG_CONFIG_HOME:-$HOME/.config}/note-desktop/settings.toml"

```

```

4 labwc_dir="${XDG_CONFIG_HOME:-$HOME/.config}/labwc"
5 mkdir -p "$labwc_dir"
6
7 python3 - "$settings" "$labwc_dir/rc.xml" <<'PY'
8 import pathlib, sys
9 try:
10     import toml
11 except ModuleNotFoundError:
12     toml = None
13 settings_path = pathlib.Path(sys.argv[1])
14 out = pathlib.Path(sys.argv[2])
15 data = {}
16 if settings_path.exists() and toml:
17     with settings_path.open('rb') as handle:
18         data = toml.load(handle)
19 workspaces = max(1, min(9, int(data.get('workspaces', 4))))
20 natural = 'yes' if data.get('natural_scroll', True) else 'no'
21 out.write_text(f'<?xml version="1.0"?>
22 <labwc_config xmlns="http://openbox.org/3.4/rc">
23     <desktops number="{workspaces}" />
24     <libinput>
25         <device category="touchpad"><naturalScroll>{natural}</naturalScroll></device>
26     </libinput>
27 </labwc_config>
28 ''')
29 PY
30
31 labwc --reconfigure >/dev/null 2>&1 || true

```

D.13 Configuración completa de labwc

```

1 <?xml version="1.0"?>
2 <labwc_config xmlns="http://openbox.org/3.4/rc">
3     <core>
4         <decoration>server</decoration>
5         <gap>10</gap>
6         <adaptiveSync>yes</adaptiveSync>
7         <reuseOutputMode>yes</reuseOutputMode>
8         <xwaylandPersistence>no</xwaylandPersistence>
9     </core>
10
11     <theme>
12         <name>Note</name>
13         <icon>Adwaita</icon>
14         <cornerRadius>12</cornerRadius>
15         <titlebar><layout>close,max,iconify:</layout></titlebar>
16         <font place="ActiveWindow"><name>Inter</name><size>10</size><weight>Semibold</weight>
17         </font>
18         <font place="InactiveWindow"><name>Inter</name><size>10</size></font>
19         <font place="MenuItem"><name>Inter</name><size>10</size></font>
20         <font place="OnScreenDisplay"><name>Inter</name><size>11</size><weight>Bold</weight>
21         </font>
22     </theme>
23
24     <desktops number="4">
25         <popupTime>650</popupTime>
26         <prefix>Escritorio</prefix>

```

```

25 </desktops>
26
27 <resize><popupShow>Always</popupShow><drawContents>yes</drawContents></resize>
28 <focus><followMouse>no</followMouse><raiseOnFocus>no</raiseOnFocus></focus>
29 <snapping>
30   <screenEdgeStrength>18</screenEdgeStrength>
31   <windowEdgeStrength>10</windowEdgeStrength>
32 </snapping>
33
34 <keyboard>
35   <default />
36   <keybind key="Super_L" onRelease="yes"><action name="Execute" command="note-overview
37     " /></keybind>
38   <keybind key="W-Space"><action name="Execute" command="note-overview" /></keybind>
39   <keybind key="W-Return"><action name="Execute" command="note-terminal" /></keybind>
40   <keybind key="W-e"><action name="Execute" command="note-files" /></keybind>
41   <keybind key="W-b"><action name="Execute" command="note-browser" /></keybind>
42   <keybind key="W-comma"><action name="Execute" command="note-settings" /></keybind>
43   <keybind key="W-l"><action name="Execute" command="note-lock" /></keybind>
44   <keybind key="W-q"><action name="Close" /></keybind>
45   <keybind key="W-f"><action name="ToggleFullscreen" /></keybind>
46   <keybind key="W-m"><action name="Iconify" /></keybind>
47   <keybind key="W-Up"><action name="ToggleMaximize" /></keybind>
48   <keybind key="W-Left"><action name="SnapToEdge" direction="left" /></keybind>
49   <keybind key="W-Right"><action name="SnapToEdge" direction="right" /></keybind>
50   <keybind key="W-Down"><action name="SnapToEdge" direction="down" /></keybind>
51   <keybind key="W-C-Left"><action name="GoToDesktop" to="left" wrap="yes" /></keybind>
52   <keybind key="W-C-Right"><action name="GoToDesktop" to="right" wrap="yes" /></keybind>
53   <keybind key="W-C-S-Left"><action name="SendToDesktop" to="left" wrap="yes" follow="
54     yes" /></keybind>
55   <keybind key="W-C-S-Right"><action name="SendToDesktop" to="right" wrap="yes" follow
56     ="yes" /></keybind>
57   <keybind key="W-1"><action name="GoToDesktop" to="1" /></keybind>
58   <keybind key="W-2"><action name="GoToDesktop" to="2" /></keybind>
59   <keybind key="W-3"><action name="GoToDesktop" to="3" /></keybind>
60   <keybind key="W-4"><action name="GoToDesktop" to="4" /></keybind>
61   <keybind key="W-5"><action name="GoToDesktop" to="5" /></keybind>
62   <keybind key="W-6"><action name="GoToDesktop" to="6" /></keybind>
63   <keybind key="W-7"><action name="GoToDesktop" to="7" /></keybind>
64   <keybind key="W-8"><action name="GoToDesktop" to="8" /></keybind>
65   <keybind key="W-9"><action name="GoToDesktop" to="9" /></keybind>
66   <keybind key="Print"><action name="Execute" command="note-screenshot" /></keybind>
67   <keybind key="XF86AudioRaiseVolume"><action name="Execute" command="wpctl set-volume
68     -l 1.5 @DEFAULT_AUDIO_SINK@ 5%+" /></keybind>
69   <keybind key="XF86AudioLowerVolume"><action name="Execute" command="wpctl set-volume
70     @DEFAULT_AUDIO_SINK@ 5%-" /></keybind>
71   <keybind key="XF86AudioMute"><action name="Execute" command="wpctl set-mute
72     @DEFAULT_AUDIO_SINK@ toggle" /></keybind>
73   <keybind key="XF86AudioPlay"><action name="Execute" command="playerctl play-pause"
74     /></keybind>
75   <keybind key="XF86AudioNext"><action name="Execute" command="playerctl next" /></keybind>
76   <keybind key="XF86AudioPrev"><action name="Execute" command="playerctl previous"
77     /></keybind>
78   <keybind key="XF86MonBrightnessUp"><action name="Execute" command="brightnessctl set
79     +5%" /></keybind>
80   <keybind key="XF86MonBrightnessDown"><action name="Execute" command="brightnessctl
81     set 5%-" /></keybind>

```

```

72 </keyboard>
73
74 <mouse>
75   <default />
76   <context name="Root">
77     <mousebind button="Right" action="Press"><action name="ShowMenu" menu="root-menu"
78     /></mousebind>
79     <mousebind direction="Up" action="Scroll"><action name="GoToDesktop" to="left"
80     wrap="yes" /></mousebind>
81     <mousebind direction="Down" action="Scroll"><action name="GoToDesktop" to="right"
82     wrap="yes" /></mousebind>
83   </context>
84 </mouse>
85
86 <libinput>
87   <device category="default">
88     <naturalScroll>no</naturalScroll>
89     <pointerSpeed>0.0</pointerSpeed>
90     <tap>yes</tap>
91     <tapButtonMap>lr</tapButtonMap>
92     <disableWhileTyping>yes</disableWhileTyping>
93   </device>
94   <device category="touchpad">
95     <naturalScroll>yes</naturalScroll>
96     <clickMethod>clickfinger</clickMethod>
97     <middleEmulation>yes</middleEmulation>
98   </device>
99 </libinput>
100 </labwc_config>

```

D.14 Unidades systemd -user

```

1 [Unit]
2 Description=Note Desktop user session
3 Wants=note-shell.service note-notifications.service note-polkit-agent.service
4 After=graphical-session-pre.target
5 BindsTo=graphical-session.target
6 PartOf=graphical-session.target
7
8 [Install]
9 WantedBy=graphical-session.target

```

```

1 [Unit]
2 Description=Note Desktop shell
3 PartOf=note-session.target
4 After=graphical-session.target
5
6 [Service]
7 Type=simple
8 ExecStart=/usr/bin/note-shell
9 Restart=on-failure
10 RestartSec=1
11 Slice=session.slice
12
13 [Install]
14 WantedBy=note-session.target

```

```
1 [Unit]
2 Description=Note Desktop notifications
3 PartOf=note-session.target
4 After=graphical-session.target
5
6 [Service]
7 Type=simple
8 ExecStart=/usr/bin/mako
9 Restart=on-failure
10 Slice=background.slice
11
12 [Install]
13 WantedBy=note-session.target
```

```
1 [Unit]
2 Description=Note Desktop PolicyKit agent
3 PartOf=note-session.target
4 After=graphical-session.target
5
6 [Service]
7 Type=simple
8 ExecStart=/usr/bin/lxpolkit
9 Restart=on-failure
10 Slice=background.slice
11
12 [Install]
13 WantedBy=note-session.target
```

Glosario

Compositor Proceso que recibe superficies, administra entrada y produce la imagen final de cada monitor.

Wayland Protocolo entre compositor y clientes gráficos.

XWayland Servidor X compatible que se ejecuta como cliente Wayland.

DRM Subsistema del kernel para dispositivos gráficos y acceso controlado a recursos.

KMS Parte de DRM encargada de modos de pantalla, conectores, CRTC's y planos.

GBM API para administrar buffers gráficos sobre DRM.

EGL API para crear contextos y superficies de renderizado.

DMA-BUF Mecanismo Linux para compartir buffers entre dispositivos y procesos.

Explicit sync Sincronización explícita de acceso a buffers entre GPU, cliente y compositor.

Direct scanout Presentar un buffer del cliente directamente en un plano de pantalla sin composición intermedia.

wlroots Biblioteca C modular usada por varios compositores Wayland, incluida la base de labwc.

Smithay Biblioteca Rust para construir compositores Wayland.

layer-shell Protocolo para paneles, docks, fondos y overlays.

xdg-shell Protocolo principal para ventanas y popups de aplicaciones de escritorio.

GApplication Modelo de aplicación GLib con instancia única, acciones y activación D-Bus.

GObject Sistema de objetos usado por GTK; los wrappers Rust suelen ser referencias contadas.

PolicyKit Framework de autorización para operaciones privilegiadas.

Portal XDG Interfaz mediada para archivos, capturas, screencast y otras operaciones sensibles.

PipeWire Servidor multimedia para audio y video.

WirePlumber Session manager de PipeWire; wpctl es su cliente CLI.

NetworkManager Servicio de red y API D-Bus.

BlueZ Pila Bluetooth de Linux.

UPower Servicio de energía y batería.

greetd Daemon de inicio de sesión genérico.

gtk-greet Greeter GTK para greetd.

cage Compositor Wayland de una sola aplicación.

labwc Compositor Wayland apilado, compatible con configuración tipo Openbox.

TTY Terminal virtual del kernel; vía de recuperación fuera del escritorio.

systemd -user Instancia de systemd para procesos de un usuario.

DESTDIR Raíz temporal usada al construir paquetes sin instalar directamente en el sistema.

Desktop entry Archivo .desktop que describe nombre, icono y comando de una aplicación.

App ID Identificador usado para relacionar ventanas con aplicaciones.

Pixman Biblioteca de composición por CPU usada como fallback de software.

PRIME Mecanismos de offload y compartición en sistemas multi-GPU.

VRR Frecuencia de refresco variable.

CI Integración continua: compilación y pruebas automatizadas por cada cambio.

Cierre

Note Desktop 0.2.3 ya demuestra una vertical completa: login, sesión Wayland, XWayland, compositor temporal, shell propio, configuración, servicios, instalación y empaquetado inicial. La siguiente etapa no consiste en seguir pegando efectos encima de labwc, sino en mejorar la calidad del shell, convertir integraciones CLI en servicios y construir note-compositor de forma incremental.

El proyecto tiene un valor especial como laboratorio de ingeniería de sistemas: toca UI, procesos, IPC, gráficos, seguridad, paquetes y compatibilidad de hardware. Para que crezca sin hacerse un desmadre, mantén fronteras claras, pruebas por capa, recuperación por TTY y documentación al día.

Siguiente movimiento recomendado

Crea una rama de refactorización, divide note-shell en módulos, agrega pruebas a note-core y configura CI para compilar el snapshot corregido. Después abre un crate note-compositor mínimo anidado. Ese orden produce progreso visible sin sacrificar la base que ya jala.